République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Poly technique SOUSSE
EUR-ACE®

RAPPORT
DE PROJET DE FIN D'ÉTUDES

Pour l'obtention du Diplôme National d'Ingénieur
en Génie Informatique

Sujet :

Conception et développement d'une plateforme bancaire intelligente intégrant un agent intelligent de support client

Réalisé par : Raef Gaied

Encadrant Académique : M. Haythem SAOUDI

Encadrante Professionnelle : Mme Feriel JRIDI

VERMEG

2025 - 2026
Année Universitaire

Dédicaces

Je dédie ce mémoire à tous ceux qui ont été mes sources de courage dans les moments d'adversité.

À mes parents, mes oncles, ma grand-mère, mes frères,

Pour votre support inébranlable et votre soutien indéfectible, pour chaque sacrifice fait sans hésitation pour m'aider à atteindre mes rêves. Vous m'avez enseigné que la persévérance et la passion sont les clés de toute réussite. Ce mémoire est le fruit de votre éducation et de votre confiance en moi.

À mes amis,

Nous avons partagé des défis, des succès et des moments inoubliables. Chaque jour passé avec vous a été une leçon de collaboration, d'innovation et d'amitié. Merci d'avoir rendu ce voyage mémorable.

À tous ceux qui, de près ou de loin,

Ont contribué à ma formation et à ce mémoire. Votre influence a façonné le professionnel que je deviens.

Raef GAIED

Remerciements

Au terme de ce projet de fin d'études, je souhaite exprimer ma profonde gratitude envers toutes les personnes qui, de près ou de loin, ont contribué à l'aboutissement de ce travail.

Je tiens tout d'abord à adresser mes sincères remerciements à **Mme Feriel JRIDI**, mon encadrante professionnelle au sein de **VERMEG**, pour sa disponibilité constante, sa bienveillance et ses précieux conseils tout au long de ce projet. Son accompagnement rigoureux et son expertise m'ont permis de progresser et de mener à bien ce travail dans les meilleures conditions.

Je remercie également l'ensemble des équipes de **VERMEG** pour leur accueil chaleureux, leur professionnalisme et pour m'avoir offert un environnement de travail stimulant et enrichissant. Travailler au sein d'une entreprise internationale de premier plan dans le secteur des technologies financières a été une expérience particulièrement formatrice.

J'exprime ma reconnaissance la plus sincère envers **M. Haythem SAOUDI**, mon encadrant académique, pour son suivi attentif, ses orientations méthodologiques et ses remarques constructives. Ses conseils avisés ont largement contribué à la qualité scientifique et technique de ce rapport.

Mes remerciements s'adressent également à l'ensemble du corps professoral de l'**École Polytechnique de Sousse**, pour la qualité de l'enseignement dispensé tout au long de mon cursus, ainsi que pour les compétences et les valeurs transmises. Leur engagement et leur exigence ont été déterminants dans ma formation d'ingénieur.

Je remercie chaleureusement ma famille pour son soutien indéfectible, sa patience et ses encouragements constants. Leur présence bienveillante a été une source de motivation essentielle dans les moments exigeants de ce parcours.

Enfin, j'adresse mes remerciements les plus respectueux aux **membres du jury** pour l'honneur qu'ils me font en acceptant d'évaluer ce travail, ainsi que pour le temps et l'attention qu'ils y consacrent.

Table des matières

1 Cadre général du projet **3**

Introduction . . . 3

1.0.1 VERMEG . . . 3

1.0.2 Expertises et clients . . . 5

1.1 Contexte du projet . . . 6

1.1.1 Évolution des attentes des clients bancaires . . . 6

1.1.2 Limites des solutions traditionnelles . . . 6

1.1.3 Émergence des assistants intelligents . . . 7

1.1.4 Positionnement du projet FinBank . . . 7

1.1.5 Problématique . . . 8

1.1.6 Étude de l'existant . . . 9

1.1.7 Analyse comparative de l'existant . . . 9

1.1.8 Critique de l'existant . . . 10

1.1.9 Solution proposée . . . 12

1.1.10 Comparaison des solutions . . . 13

1.2 Méthodologie de travail . . . 13

1.2.1 Framework Scrum . . . 13

1.2.2 Équipe et rôles . . . 14

1.2.3 Chronologie . . . 14

1.2.4 Conclusion . . . 16

2 Sprint 0 : Analyse et spécification des besoins **17**

Introduction . . . 17

2.1 Langages et outils de modélisation . . . 17

2.1.1 Unified Modeling Language (UML) . . . 18

2.1.2 Business Process Model and Notation (BPMN) . . . 18

2.1.3 Complémentarité des deux langages . . . 19

2.2 Analyse et spécification des besoins 20
2.2.1 Identification des acteurs 20
2.2.2 Identification des besoins 21
2.3 Diagrammes UML 25
2.3.1 Diagramme de cas d'utilisation global 25
2.3.2 Diagramme de classes global 27
2.4 Modélisation des processus métier avec BPMN 29
2.4.1 Processus de validation d'un virement 29
2.4.2 Processus de traitement d'un signalement de fraude 29
2.4.3 Processus de validation d'une mise à jour des données 30
2.5 Conduite du projet avec Scrum 31
2.5.1 Équipe et rôles 31
2.5.2 Le Product Backlog 31
2.5.3 Planification des sprints 42
2.6 Architecture et outils 44
2.6.1 Architecture globale de l'application 44
2.6.2 Architecture logicielle 45
2.7 Environnement de travail 47
2.7.1 Environnement matériel 47
2.7.2 Environnement logiciel 47
Conclusion 50

3 Sprint 1 : Mise en place du backend et sécurisation des accès 51

3.1 Introduction 51
3.2 Backlog du Sprint 52
3.3 Conception 52
3.3.1 Diagramme de cas d'utilisation 53
3.3.2 Diagramme de classes 54
3.3.3 Diagramme de séquence : authentification sans 2FA 55
3.3.4 Diagramme de séquence : authentification avec 2FA 55
3.3.5 Description textuelle des cas d'utilisation 56
3.4 Réalisation 58

3.4.1 Interface d'inscription . . . 59
3.4.2 Interface de connexion . . . 59
3.4.3 Interface de vérification 2FA . . . 60
3.4.4 Interface d'activation de la 2FA . . . 61
3.4.5 Interface de gestion du profil . . . 62
3.4.6 Interface de changement de mot de passe . . . 63
3.4.7 Interface d'administration des utilisateurs . . . 63
3.5 Conclusion . . . 64

4 Sprint 2 : Développement des fonctionnalités bancaires 65
4.1 Introduction . . . 65
4.2 Backlog du Sprint . . . 66
4.3 Conception . . . 66
4.3.1 Diagramme de cas d'utilisation . . . 67
4.3.2 Diagramme de classes . . . 68
4.3.3 Diagramme de séquence : effectuer un virement . . . 69
4.3.4 Diagramme de séquence : consulter le compte et le tableau de bord . . . 69
4.3.5 Diagramme de séquence : consulter ses notifications . . . 70
4.3.6 Description textuelle des cas d'utilisation . . . 71
4.4 Réalisation . . . 73
4.4.1 Interface du tableau de bord et du compte . . . 74
4.4.2 Interface des dépôts et des retraits . . . 75
4.4.3 Interface des virements . . . 76
4.4.4 Interface de l'historique et des reçus . . . 79
4.4.5 Interface de gestion des cartes . . . 81
4.4.6 Interface de gestion des investissements . . . 81
4.4.7 Interface des clés de paiement . . . 82
4.5 Conclusion . . . 83

5 Sprint 3 : Assistant bancaire et base de connaissances RAG 84
5.1 Introduction . . . 84
5.2 Backlog du Sprint . . . 84

5.3 Conception . . . 85
5.3.1 Diagramme de cas d'utilisation . . . 85
5.3.2 Diagramme de classes . . . 87
5.3.3 Description textuelle des cas d'utilisation . . . 89
5.3.4 Diagrammes de séquence . . . 90
5.4 Réalisation . . . 92
5.4.1 Architecture générale de l'agent . . . 92
5.4.2 Intégration du protocole MCP . . . 93
5.4.3 Détection d'intention et choix du modèle . . . 93
5.4.4 Traitement conversationnel et outils bancaires . . . 95
5.4.5 Service Python FastAPI . . . 96
5.4.6 Gestion des sessions et des messages . . . 97
5.4.7 Préparation et indexation des données RAG . . . 98
5.4.8 Récupération du contexte et génération de la réponse . . . 98
5.4.9 Deux niveaux de recherche augmentée . . . 99
5.4.10 Administration des documents et extraction . . . 99
5.5 Conclusion . . . 101

6 Sprint 4 : Workflows BPMN et validation des opérations sensibles 103
6.1 Introduction . . . 103
6.2 Backlog du Sprint . . . 104
6.3 Conception . . . 105
6.3.1 Diagramme de cas d'utilisation . . . 105
6.3.2 Diagramme de classes . . . 107
6.3.3 Description textuelle des cas d'utilisation . . . 109
6.3.4 Diagramme de séquence . . . 113
6.3.5 Modèles BPMN des processus . . . 114
6.4 Réalisation . . . 117
6.4.1 Intégration de Flowable . . . 117
6.4.2 Parcours : validation d'un virement important . . . 118
6.4.3 Parcours : traitement d'une alerte de fraude . . . 119
6.4.4 Parcours : modification de données sensibles . . . 121

6.4.5 Espace administrateur : supervision des processus . . . 122
6.4.6 Suivi des demandes et gestion des états . . . 125
6.5 Conclusion . . . 126

7 Sprint 5 : Tests, validation et préparation de la livraison 127
7.1 Introduction . . . 127
7.2 Backlog du Sprint . . . 128
7.3 Stratégie de test . . . 129
7.4 Mise en œuvre des tests . . . 130
7.4.1 Tests unitaires du backend . . . 131
7.4.2 Tests des contrôleurs et de la sécurité . . . 131
7.4.3 Tests des processus BPMN et de l'assistant . . . 131
7.4.4 Tests frontend et parcours utilisateur . . . 132
7.5 Flux de développement et automatisation . . . 133
7.5.1 Gestion des versions avec Git . . . 133
7.5.2 Intégration continue avec GitHub Actions . . . 133
7.5.3 Analyse statique avec SonarCloud . . . 134
7.6 Correction des anomalies et résultats de la validation . . . 135
7.6.1 Démarche de correction . . . 135
7.6.2 Anomalies détectées et corrections appliquées . . . 136
7.6.3 Résultats de la validation . . . 136
7.6.4 Limites de la validation . . . 137
7.7 Conteneurisation et déploiement . . . 137
7.7.1 Conteneurisation et orchestration des services . . . 138
7.7.2 Publication des images et déploiement distant . . . 139
7.7.3 Vérification du déploiement . . . 140
7.8 Finalisation et préparation de la soutenance . . . 140
7.9 Conclusion . . . 141

Conclusion Générale et Perspectives 142

Bibliographie 144

Table des figures

1.1 Logo Vermeg 4
1.2 Exemples de clients de VERMEG 5
1.3 Bankin' 9
1.4 Revolut 9
1.5 Erica 9
1.6 Cycle de développement selon Scrum 14
1.7 Diagramme de Gantt du projet FinBank (février – juillet 2026) 15

2.1 Diagramme de cas d'utilisation global – FinBank 26
2.2 Diagramme de classes global – FinBank 28
2.3 Processus BPMN de validation d'un virement 29
2.4 Processus BPMN de traitement d'un signalement de fraude 30
2.5 Processus BPMN de validation d'une mise à jour des données 30
2.6 Architecture globale de la plateforme FinBank 46
2.7 Architecture logicielle : principe modèle-vue-contrôleur 47

3.1 Diagramme de cas d'utilisation du Sprint 1 53
3.2 Diagramme de classes du Sprint 1 54
3.3 Diagramme de séquence – Authentification sans 2FA 55
3.4 Diagramme de séquence – Authentification avec 2FA 56
3.5 Interface d'inscription 59
3.6 Interface de connexion 60
3.7 Interface de vérification 2FA 61
3.8 Interface d'activation de la 2FA 61
3.9 Confirmation de l'activation de la double authentification 62
3.10 Interface de gestion du profil 62
3.11 Interface de changement de mot de passe 63

3.12 Interface d'administration des utilisateurs 64

4.1 Diagramme de cas d'utilisation du Sprint 2 67
4.2 Diagramme de classes du Sprint 2 68
4.3 Diagramme de séquence – Effectuer un virement 69
4.4 Diagramme de séquence – Consultation du compte et du tableau de bord . . 70
4.5 Diagramme de séquence – Consulter ses notifications 70
4.6 Interface de consultation du compte 74
4.7 Tableau de bord financier de l'utilisateur 75
4.8 Interface de dépôt 76
4.9 Interface de retrait 76
4.10 Interface des virements 77
4.11 Notification reçue après un virement 78
4.12 Architecture de fonctionnement de Kafka et Redis 79
4.13 Interface de l'historique et des reçus 80
4.14 Exemple de reçu PDF d'une transaction 80
4.15 Interface de gestion des cartes 81
4.16 Interface de gestion des investissements 82
4.17 Interface des clés de paiement 83

5.1 Diagramme de cas d'utilisation du Sprint 3 86
5.2 Diagramme de classes du Sprint 3 88
5.3 Diagramme de séquence – Question et recherche RAG 91
5.4 Diagramme de séquence – Création et réindexation d'un document RAG . . 91
5.5 Architecture de l'agent bancaire conversationnel 92
5.6 Agents bancaires associés aux sessions et décisions affichées 92
5.7 Détection d'intention et mécanisme de repli 94
5.8 Collecte des informations et confirmation – création de carte 95
5.9 Confirmation de la création et vérification de la carte enregistrée 95
5.10 Historique des décisions associées à un agent 96
5.11 Traitement d'une demande par l'agent et ses outils 97
5.12 Interface de gestion des sessions conversationnelles 98

5.13 Test du moteur RAG – correspondances documentaires récupérées 99
5.14 Administration des documents de connaissance 100
5.15 Étapes de gestion des documents de connaissance 101

6.1 Diagramme de cas d'utilisation du Sprint 4 106
6.2 Diagramme de classes des demandes et workflows du Sprint 4 108
6.3 Supervision et complétion d'une tâche Flowable 114
6.4 Processus BPMN de validation d'un virement important 115
6.5 Processus BPMN de traitement d'une alerte de fraude 116
6.6 Processus BPMN de validation d'une modification sensible 117
6.7 État de disponibilité du moteur Flowable intégré 118
6.8 Soumission et mise en attente d'un virement important 119
6.9 Résolution et notification du virement validé 119
6.10 Signalement d'une fraude et création de la demande 120
6.11 Analyse et décision sur une alerte de fraude 120
6.12 Résultat du traitement d'une alerte de fraude 120
6.13 Ouverture d'une demande de modification sensible 121
6.14 Transmission du justificatif par le client 121
6.15 Examen et approbation de la demande sensible 122
6.16 Résolution de la demande de modification et retour au client 122
6.17 Liste des tâches de validation destinées à l'administrateur 123
6.18 Tableau de bord administratif et indicateurs de supervision 123
6.19 Notifications administratives relatives aux tickets et aux tâches BPMN . . . 124
6.20 Trace d'audit des décisions administratives 124
6.21 Vue administrateur des événements de sécurité 125
6.22 Suivi d'une demande depuis l'espace d'aide et support 125

7.1 Stratégie de test de la plateforme FinBank 130
7.2 Validation des workflows BPMN et de l'assistant bancaire 132
7.3 Tests des interfaces et des parcours utilisateur 132
7.4 Historique GitHub Actions et blocage actuel de facturation 134
7.5 Tableau de bord SonarCloud du projet banking-api 135

7.6 Dockerfile de l'agent IA 138
7.7 Orchestration des services avec Docker Compose 139
7.8 Services FinBank démarrés dans Docker Desktop 140
7.9 Préparation de la livraison de la plateforme 140

Liste des tableaux

1.1 Chiffres clés de VERMEG 5
1.2 Comparaison des assistants bancaires existants 11
1.3 Planification des sprints – Projet FinBank 15

2.1 Complémentarité d'UML et BPMN dans FinBank 19
2.2 Répartition des besoins fonctionnels par acteur 23
2.3 Synthèse des besoins non fonctionnels 25
2.4 Backlog produit de la plateforme FinBank 32
2.5 Planification des sprints – Projet FinBank 42
2.6 Principaux composants de l'architecture de FinBank 44

3.1 Backlog du Sprint 1 52
3.2 Description textuelle – S'authentifier 57
3.3 Description textuelle – Gérer son profil 58

4.1 Backlog du Sprint 2 66
4.2 Description textuelle – Consulter ses informations bancaires 71
4.3 Description textuelle – Consulter le tableau de bord financier 72
4.4 Description textuelle – Effectuer un virement 73

5.1 Backlog du Sprint 3 85
5.2 Description textuelle – Interagir avec l'assistant bancaire 89
5.3 Description textuelle – Gérer les documents de connaissance 90

6.1 Backlog du Sprint 4 104
6.2 Description textuelle – Traiter une tâche de workflow 110
6.3 Description textuelle – Valider un virement important 111
6.4 Description textuelle – Traiter une alerte de fraude 112
6.5 Description textuelle – Valider une modification de données sensibles 113

7.1 Backlog du Sprint 5 128
7.2 Niveaux de test et outils utilisés dans le projet 129
7.3 Anomalies détectées et corrections appliquées 136
7.4 Catégories de vérification finale 137

Liste des acronymes

2FA Authentification à deux facteurs (*Two-Factor Authentication*)

API Interface de programmation applicative (*Application Programming Interface*)

BPMN Modèle et notation des processus métier (*Business Process Model and Notation*)

IA Intelligence artificielle

JPA API de persistance Java (*Java Persistence API*)

JWT Jeton Web JSON (*JSON Web Token*)

OMG Groupe de gestion des objets (*Object Management Group*)

OpenAPI Spécification ouverte de description des API

PFE Projet de fin d'études

PIX Système de paiement instantané

PO Responsable produit (*Product Owner*)

RAG Génération augmentée par la recherche (*Retrieval-Augmented Generation*)

REST Transfert d'état représentatif (*Representational State Transfer*)

RGPD Règlement général sur la protection des données

SA Société anonyme

SQL Langage de requête structuré (*Structured Query Language*)

TOTP Mot de passe à usage unique basé sur le temps (*Time-Based One-Time Password*)

UML Langage de modélisation unifié (*Unified Modeling Language*)

UX Expérience utilisateur (*User Experience*)

Introduction Générale

Le secteur bancaire connaît une transformation profonde, portée par l'évolution des technologies numériques et par les nouvelles attentes des clients. Ces derniers souhaitent accéder à leurs services bancaires de manière simple, rapide et sécurisée. Cependant, la gestion des opérations, le traitement des demandes et l'accompagnement des clients nécessitent encore de nombreuses interventions manuelles, rendant certains processus longs et complexes.

À l'ère de la transformation digitale, les plateformes web et l'intelligence artificielle offrent de nouvelles possibilités pour moderniser les services bancaires. Elles permettent de centraliser les fonctionnalités, de faciliter les échanges avec les clients et d'automatiser certaines tâches. L'intégration de l'intelligence artificielle dans ce domaine doit toutefois respecter les exigences de **sécurité**, de **traçabilité** et de **contrôle** propres aux opérations financières.

C'est dans ce contexte que s'inscrit notre projet de fin d'études, réalisé au sein de **VERMEG** du **2 février au 19 juillet 2026**, sur une durée d'environ **six mois**. La mission qui nous a été confiée consiste à concevoir et à développer **FinBank**, une plateforme bancaire numérique intégrant un agent bancaire basé sur l'intelligence artificielle. Le projet vise à centraliser plusieurs services destinés aux clients et aux administrateurs, tout en explorant l'automatisation de certaines tâches traditionnellement prises en charge par un employé. Les opérations sensibles restent encadrées par des mécanismes de contrôle et de validation.

FinBank se distingue des plateformes bancaires classiques par l'intégration d'un **assistant conversationnel intelligent** capable d'interpréter les demandes en **langage naturel** et d'orienter automatiquement les opérations vers le traitement approprié. Cette approche combine **automatisation**, **sécurité** et **traçabilité**, trois exigences fondamentales du secteur bancaire.

Sur le plan technique, FinBank repose sur une architecture web moderne combinant un front-end **React / TypeScript**, un back-end **Spring Boot** et une base de données **PostgreSQL**. L'orchestration des opérations sensibles est assurée par le moteur BPMN **Flowable**, tandis que les fonctionnalités d'intelligence artificielle s'appuient principalement sur **Ollama** pour la détection d'intention et sur un mécanisme de **génération augmentée**

par la recherche (RAG) pour l'assistance documentaire.

La problématique à laquelle répond ce projet peut être formulée ainsi : *comment concevoir une plateforme bancaire numérique capable de centraliser les services destinés aux clients et d'automatiser le traitement de certaines demandes à l'aide d'un agent intelligent, tout en garantissant la sécurité, la traçabilité et la validation des opérations sensibles ?*

Pour répondre à cette problématique, nous proposons FinBank, une plateforme qui réunit des fonctionnalités bancaires, des outils d'administration et un assistant intelligent. L'agent peut accompagner le traitement des demandes et s'appuyer sur une base documentaire pour fournir des réponses pertinentes. Les processus nécessitant un contrôle particulier sont orchestrés à l'aide de workflows BPMN exécutés par Flowable. Cette approche vise à améliorer l'efficacité du traitement tout en maintenant un cadre de supervision adapté.

Ce rapport est structuré en quatre chapitres. Le premier présente l'entreprise d'accueil, le contexte du projet et la méthodologie adoptée. Le deuxième est consacré à l'analyse des besoins et à la spécification fonctionnelle. Le troisième traite de la conception de la plateforme, de son architecture et de ses processus métier. Le quatrième détaille la réalisation, l'intégration de l'agent intelligent et les tests effectués. Enfin, la conclusion générale synthétise les résultats et présente les perspectives d'amélioration.

1

Cadre général du projet

Introduction

Ce chapitre présente le cadre général du projet de fin d'études, réalisé au sein de **VERMEG**, entreprise internationale spécialisée dans le développement de solutions logicielles pour les secteurs **financier**, **bancaire** et **assurantiel**. M , le projet **FinBank** consiste à concevoir une plateforme **bancaire digitale** intégrant un **assistant bancaire intelligent** capable d'interpréter les demandes en **langage naturel**. La solution repose sur une architecture moderne combinant un backend **Java / Spring Boot**, un frontend **React / TypeScript**, une base de données **PostgreSQL**, un moteur BPMN **Flowable** pour les opérations sensibles, un service d'IA basé sur **Ollama** avec un mécanisme **RAG** pour la recherche documentaire, ainsi qu'une fonctionnalité d'**analyse financière**. Le projet a été développé selon la méthodologie **Agile Scrum**, favorisant une livraison itérative et une amélioration continue au fil des **sprints**.

1.0.1 VERMEG

VERMEG [1] est une entreprise internationale spécialisée dans le développement de logiciels informatiques, fondée en 1993 par **Badreddine Ouali**, entrepreneur tunisien reconnu

dans le secteur des technologies financières. Constituée sous la forme d'une **société anonyme**, elle intervient principalement sur le marché **B2B** (Business-to-Business) en proposant des solutions logicielles destinées aux institutions bancaires, financières et assurantielles. Au fil des années, VERMEG s'est imposée comme un **acteur de référence** dans trois domaines stratégiques : la **Banque**, les **Marchés de Capitaux** et l'**Assurance**.

Spécialisée dans l'**édition de logiciels**, l'entreprise conçoit et commercialise à la fois des **solutions standards** (produits packagés, prêts à l'emploi) et des **solutions sur mesure** (développements spécifiques adaptés aux besoins de chaque client). Ces solutions ont pour vocation d'accompagner les institutions financières dans leur **transformation numérique**, de **moderniser leurs systèmes d'information**, de renforcer leur **conformité réglementaire** et d'**optimiser leurs coûts** ainsi que leurs **délais de mise sur le marché**.

Forte de plus de **1 300 collaborateurs** répartis à travers le monde, VERMEG dispose d'une présence internationale solide avec des implantations en **Europe**, en **Afrique**, au **Moyen-Orient** et en **Asie**. Elle accompagne aujourd'hui plus de **500 clients** répartis dans **40 pays**. Cette couverture géographique et ce portefeuille client témoignent de la **confiance** que lui accordent les plus grandes institutions du secteur.

Enfin, VERMEG se distingue par sa **notoriété à l'échelle internationale**, par la qualité de ses solutions primées à plusieurs reprises, ainsi que par son **expertise reconnue** dans la conception de systèmes d'information dédiés aux secteurs bancaire, financier et assurantiel. Son engagement constant en matière de **recherche et développement** et sa capacité d'**innovation** lui permettent de maintenir une position de leader sur un marché en constante évolution.



Fig 1.1 : Logo Vermeg

Tab 1.1 : Chiffres clés de VERMEG

Critère	**Information**
Fondation	1993, par Badreddine Ouali
Forme juridique	Société anonyme (SA)
Secteur	Édition de logiciels B2B
Domaines clés	Banque, Marchés de Capitaux, Assurance
Effectif	Plus de 1 300 collaborateurs
Clients	Plus de 500 clients dans 40 pays
Présence	Europe, Afrique, Moyen-Orient, Asie

1.0.2 Expertises et clients

VERMEG accompagne de nombreux acteurs clés des secteurs de la finance et de l'assurance à l'échelle internationale, en leur fournissant des solutions technologiques sur mesure pour la **gestion des risques**, la **conformité réglementaire**, la **transformation digitale** et l'**optimisation opérationnelle**. Sa clientèle regroupe aussi bien des **banques commerciales**, des **banques centrales**, des **sociétés d'assurance**, des **gestionnaires d'actifs** que des **infrastructures de marché**. La figure 1.2 illustre quelques exemples de clients majeurs ayant fait confiance à VERMEG pour les accompagner dans leur transformation numérique.



Fig 1.2 : Exemples de clients de VERMEG

Les activités de VERMEG s'articulent autour de plusieurs axes stratégiques :

— **Gestion des risques et conformité réglementaire** : mise en place de solutions répondant aux exigences des régulateurs (Bâle III, Solvabilité II, IFRS, etc.).

— **Gestion des actifs et investissements** : plateformes de gestion de portefeuille, de conservation de titres et de reporting financier.

— **Transformation digitale** : modernisation des systèmes d'information, migration vers le cloud et digitalisation des processus métiers.

— **Cybersécurité et protection des données** : sécurisation des transactions financières et conformité au RGPD.

— **Automatisation des opérations back-office** : réduction des tâches manuelles et optimisation des délais de traitement.

Grâce à son expertise métier et technologique, VERMEG se positionne comme un **partenaire stratégique** pour les institutions financières en quête de **performance**, de **sécurité** et d'**innovation**. Son portefeuille client, composé de banques de premier plan telles que **Santander**, **BNP Paribas**, **Deutsche Bank**, **Société Générale**, **AXA**, **Natixis**, **Nordea** ou encore **la Banque de France**, confirme son positionnement de leader sur le marché des solutions logicielles financières.

1.1 Contexte du projet

1.1.1 Évolution des attentes des clients bancaires

Avec la généralisation des services bancaires en ligne, les clients attendent désormais une assistance **instantanée**, disponible **24h/24 et 7j/7**, pour des opérations courantes telles que la consultation du solde, les virements, l'historique des transactions ou la gestion de leurs cartes bancaires. Cette évolution des usages, portée par la **transformation digitale** du secteur financier, a profondément modifié la relation entre les banques et leurs clients.

Toutefois, toutes les demandes ne peuvent pas être traitées de la même manière. Certaines situations, comme les **situations de fraude**, la **perte ou le vol de carte**, les **oppositions**, ou encore les **opérations financières sensibles** (virements importants, modifications de coordonnées bancaires, etc.), nécessitent un traitement **sécurisé**, **contrôlé** et **traçable**. Ces exigences découlent directement des obligations réglementaires auxquelles les institutions financières sont soumises.

1.1.2 Limites des solutions traditionnelles

Les solutions traditionnelles de support client, souvent basées sur des **arbres de décision** ou des **réponses prédéfinies**, présentent aujourd'hui des limites importantes :

— Elles ne permettent pas toujours de comprendre des demandes formulées en **langage**

naturel, ce qui oblige le client à s'adapter au fonctionnement du système plutôt que l'inverse.

— Elles ne gèrent pas efficacement les **cas complexes** ou ambigus, qui nécessitent une interprétation fine du besoin réel.

— Elles ne garantissent pas un **niveau de sécurité suffisant** pour les transactions critiques, faute d'une validation adaptée au niveau de risque.

Dans un secteur soumis à des exigences fortes de **conformité** et de **responsabilité**, ces limites peuvent entraîner des **retards** de traitement, des **erreurs** opérationnelles, voire des **risques de fraude**. Elles nuisent également à la satisfaction client et à la compétitivité des établissements bancaires.

1.1.3 Émergence des assistants intelligents

Face à ces constats, les institutions financières cherchent à moderniser leur relation client en intégrant des **assistants intelligents** capables de dialoguer naturellement avec les utilisateurs, tout en respectant les contraintes réglementaires, de sécurité et de traçabilité. L'**intelligence artificielle générative** apparaît aujourd'hui comme une solution pertinente pour **interpréter les demandes clients**, **identifier leur intention** et **orienter l'action à mener** de manière fiable et contextuelle.

Cette approche permet d'offrir une expérience utilisateur fluide, personnalisée et disponible en permanence, tout en réduisant la charge supportée par les équipes humaines pour les demandes de premier niveau.

1.1.4 Positionnement du projet FinBank

Le projet **FinBank** s'inscrit pleinement dans cette dynamique. Il vise à concevoir une **plateforme bancaire digitale** qui combine :

— un **assistant bancaire intelligent** capable d'interpréter les demandes en langage naturel ;

— un **moteur de workflows métier** permettant de sécuriser, valider et tracer les opérations sensibles.

L'application permet ainsi à l'utilisateur d'interagir avec le système en langage naturel, de consulter ses informations bancaires, de lancer certaines opérations et de bénéficier d'une

validation adaptée selon le niveau de risque de chaque demande.

Sur le plan technique, l'architecture du projet repose sur l'intégration de plusieurs technologies complémentaires : l'**IA générative** est utilisée pour analyser les intentions de l'utilisateur, tandis qu'un **moteur BPMN**, tel que **Flowable**, assure le pilotage, la validation et la traçabilité des processus métier sensibles. Cette combinaison permet de répondre simultanément aux exigences de **rapidité**, de **personnalisation** et de **sécurité** imposées par les services bancaires modernes.

1.1.5 Problématique

Malgré l'essor des assistants virtuels dans les services financiers, les solutions actuelles peinent à concilier deux exigences essentielles : la **flexibilité conversationnelle** offerte par l'intelligence artificielle et la **rigueur procédurale** imposée par les opérations bancaires sensibles.

D'un côté, les clients souhaitent interagir naturellement avec le système, en langage courant, pour consulter leur solde, effectuer un virement ou gérer leurs cartes. De l'autre, les institutions bancaires doivent garantir la **sécurité**, la **traçabilité** et la **conformité** des actions réalisées, notamment pour les demandes à fort risque telles que les transferts importants, les fraudes ou la perte de carte.

Cette tension entre **simplicité d'usage** et **robustesse métier** constitue le cœur du problème : un assistant intelligent ne doit pas seulement comprendre les intentions de l'utilisateur, mais aussi décider si l'opération peut être exécutée automatiquement ou si elle doit être soumise à une **validation humaine** et à un **processus BPMN**.

La question centrale du projet peut ainsi être formulée comme suit :

> *« Comment concevoir un assistant bancaire intelligent capable de comprendre les demandes formulées en langage naturel grâce à l'IA générative, tout en garantissant l'exécution sécurisée, traçable et conforme des opérations sensibles à travers un moteur BPMN ? »*

1.1.6 Étude de l'existant

Cette section présente trois solutions numériques liées aux services bancaires et à la gestion financière : **Bankin'**, **Revolut** et **Erica (Bank of America)**. Leurs fonctions sont étudiées afin d'orienter la conception de FinBank, en distinguant les applications financières des assistants conversationnels.

Bankin' **Bankin'** [2] est une application de gestion financière qui regroupe les comptes de plusieurs banques dans une même interface. Elle permet de suivre les dépenses, de les classer par catégorie et de visualiser le budget à l'aide de graphiques.

Revolut **Revolut** [3] propose une application bancaire permettant de gérer un compte, d'effectuer des transferts et d'utiliser des cartes physiques ou virtuelles. Elle regroupe également des services d'épargne et d'investissement, selon les conditions d'accès applicables.

Erica (Bank of America) **Erica** [4] est l'assistant financier virtuel intégré à l'application mobile de Bank of America. Il interprète les demandes en langage naturel pour aider à consulter les informations du compte, rechercher des transactions et suivre les dépenses ; il peut aussi orienter l'utilisateur vers un conseiller.



Fig 1.3 : Bankin'



Fig 1.4 : Revolut



Fig 1.5 : Erica

1.1.7 Analyse comparative de l'existant

Les assistants comme **Bankin'**, **Revolut** ou **Erica (BoA)** proposent des outils de base pour l'assistance bancaire conversationnelle et la réalisation de certaines opérations courantes.

Cependant, ils présentent des limites importantes : absence de gestion de workflows métier (BPMN) pour les cas complexes, traçabilité partielle des actions, seuils de sensibilité non paramétrables, et notifications incomplètes pour tous les acteurs (client, conseiller, système de validation).

Notre plateforme **FinBank** se distingue en intégrant les fonctionnalités clés que nous avons définies comme des critères fonctionnels pour évaluer la couverture des besoins utilisateurs. Ces critères, notés **C1 à C12**, sont les suivants :

- **C1** : Compréhension du langage naturel des demandes client.
- **C2** : Exécution automatisée des virements et opérations courantes.
- **C3** : Gestion des cas complexes via un moteur BPMN.
- **C4** : Traçabilité et auditabilité complètes des actions.
- **C5** : Interface conversationnelle fluide et intuitive.
- **C6** : Sécurisation des échanges via JWT / OAuth2.
- **C7** : Escalade automatique vers un agent humain si nécessaire.
- **C8** : Seuil de sensibilité paramétrable (ex : 5 000 €).
- **C9** : Détection intelligente de fraude.
- **C10** : Historique complet des conversations.
- **C11** : Intégration avec les services bancaires existants.
- **C12** : Extensibilité et personnalisation de la plateforme.

Légende : ✓ = critère pleinement satisfait | $\sim$ = critère partiellement satisfait | $\times$ = critère non satisfait.

Ainsi, **FinBank** répond aux besoins spécifiques des clients, des conseillers et des systèmes de validation, en offrant un service bancaire plus efficace, transparent, sécurisé et traçable. La solution se distingue par sa capacité à combiner **intelligence conversationnelle** et **rigueur procédurale**, ce qui la positionne comme une alternative crédible aux solutions existantes.

1.1.8 Critique de l'existant

Les assistants étudiés offrent globalement une **bonne expérience conversationnelle**, mais ils présentent plusieurs limites importantes au regard des besoins d'une plateforme bancaire moderne :

- **Absence de moteur BPMN** : aucun des assistants analysés n'intègre de moteur

Tab 1.2 : Comparaison des assistants bancaires existants

Critère	**Bankin'**	**Revolut**	**Erica (BoA)**	**FinBank**
C1 : Langage naturel	∼	×	✓	✓
C2 : Exécution de virements	×	✓	✓	✓
C3 : Gestion des cas complexes (BPMN)	×	×	×	✓
C4 : Traçabilité / Audit	×	∼	∼	✓
C5 : Interface conversationnelle	✓	✓	✓	✓
C6 : Sécurisation JWT / OAuth2	×	✓	✓	✓
C7 : Escalade vers agent humain	×	∼	∼	✓
C8 : Seuil de sensibilité (5 000 €)	×	×	×	✓
C9 : Détection de fraude	×	∼	✓	✓
C10 : Historique de conversation	×	✓	✓	✓
C11 : Intégration aux services bancaires	∼	✓	✓	✓
C12 : Extensibilité / Personnalisation	×	∼	∼	✓

de workflow métier permettant d'orchestrer, de valider et de tracer les opérations sensibles de bout en bout.

— **Gestion limitée des cas complexes** : les situations telles que la fraude, les réclamations ou les oppositions restent souvent traitées manuellement ou en dehors du chatbot, ce qui nuit à la cohérence et à la rapidité du traitement.

— **Seuils de sensibilité non paramétrables** : aucune solution ne permet de définir un seuil (par exemple, un virement supérieur à 5 000 €) qui déclenche automatiquement un workflow structuré et une validation renforcée.

— **Traçabilité partielle** : la plupart des solutions ne garantissent pas un historique complet et auditable des actions réalisées, ce qui constitue un frein majeur dans un

contexte réglementaire bancaire.

Ces limites justifient pleinement la conception d'une solution hybride originale, **FinBank**, qui combine un **assistant conversationnel basé sur l'IA générative** et un **moteur BPMN** pour sécuriser, valider et tracer les opérations bancaires sensibles.

1.1.9 Solution proposée

La solution proposée, **FinBank**, repose sur une architecture modulaire combinant plusieurs composants complémentaires :

- **Frontend** : interface web développée avec React et TypeScript pour les clients et les administrateurs ;
- **Backend** : application développée avec Java et Spring Boot, exposant des services REST sécurisés par JWT ;
- **Intelligence artificielle** : intégration d'un modèle local exécuté avec Ollama pour analyser les intentions exprimées en langage naturel et produire des réponses structurées. Des fournisseurs externes tels qu'OpenAI ou Azure OpenAI peuvent également être utilisés de manière optionnelle lorsque Ollama est désactivé et qu'une configuration appropriée est disponible ;
- **Moteur BPMN** : Flowable pour modéliser et exécuter les processus métier nécessitant une validation ou un suivi particulier ;
- **Base de données** : PostgreSQL pour centraliser les données bancaires, les transactions, les journaux d'audit et les informations relatives aux processus Flowable ;
- **Services complémentaires** : Redis pour la mise en cache, Apache Kafka pour la communication événementielle et un service de notification pour informer les utilisateurs.

La *Sensitive Line*, ou seuil de sensibilité, détermine le mode de traitement d'une requête. Les opérations à faible risque, telles que la consultation du solde ou de l'historique, peuvent être exécutées automatiquement. En revanche, les opérations sensibles, comme un virement supérieur à 5000 €, une suspicion de fraude, une carte volée ou la modification de données personnelles, déclenchent un processus de validation avec le moteur BPMN Flowable.

Trois processus principaux sont ainsi modélisés : la validation des virements importants, la gestion des suspicions de fraude et la modification des données personnelles sensibles.

L'analyse des intentions est principalement assurée par un modèle local exécuté avec Ollama. Un traitement de secours fondé sur des règles déterministes est également disponible, tandis qu'OpenAI ou Azure OpenAI peuvent être utilisés de manière optionnelle selon la configuration du système.

L'architecture globale de la plateforme est présentée dans le chapitre 2, à la figure 2.6.

1.1.10 Comparaison des solutions

Le tableau 1.2 montre que notre solution *BankMate* couvre l'ensemble des critères attendus, en particulier l'orchestration BPMN, la traçabilité et le seuil de sensibilité, absents des solutions concurrentes. Elle se distingue également par son caractère modulaire et open source, permettant une évolution future.

1.2 Méthodologie de travail

Afin d'organiser le développement du projet sur une période de six mois, nous avons adopté la méthodologie **Agile Scrum**. Cette approche itérative et incrémentale permet de réaliser le projet progressivement, de suivre régulièrement son avancement et d'adapter la solution au fur et à mesure de l'évolution des besoins.

1.2.1 Framework Scrum

Scrum est un cadre de travail léger, facile à comprendre mais difficile à maîtriser, qui permet d'aborder des problèmes complexes et adaptatifs tout en livrant régulièrement des produits de la plus grande valeur possible. Il repose sur trois rôles clés :

- **Product Owner (PO)** : représente les clients et les utilisateurs finaux ; il définit et priorise les fonctionnalités du produit.
- **Scrum Master (SM)** : garant de l'application de la méthodologie ; il accompagne l'équipe et lève les obstacles rencontrés.
- **Development Team** : équipe de développement chargée de concevoir, implémenter et livrer les fonctionnalités à chaque sprint.

Le cycle Scrum se déroule en itérations courtes appelées **sprints**, au cours desquelles l'équipe réalise un ensemble de tâches issues du **Product Backlog**. Chaque sprint se conclut

par une revue et une rétrospective permettant d'améliorer continuellement le processus.



Fig 1.6 : Cycle de développement selon Scrum

La figure 1.6 illustre le cycle de développement Scrum, depuis le **Product Backlog** jusqu'à la **revue** et la **rétrospective de sprint**, en passant par la **réunion de planification**, le **Sprint Backlog** et la **mêlée quotidienne** (*Daily Scrum*).

1.2.2 Équipe et rôles

Pour le projet **FinBank**, les rôles Scrum ont été répartis comme suit :

— **Product Owner** : le client de VERMEG, qui a défini les besoins fonctionnels et validé les livrables ;

— **Scrum Master** : l'encadrante professionnelle, qui a supervisé l'avancement et assuré le respect de la méthodologie ;

— **Development Team** : l'étudiante stagiaire, qui a assuré la conception, le développement et les tests de la plateforme.

1.2.3 Chronologie

Le projet **FinBank** s'est déroulé sur une période d'environ **six mois**, du **2 février 2026** au **19 juillet 2026**, soit **24 semaines**. Afin de structurer progressivement le développement, les travaux ont été répartis en **six sprints** principaux.

Le tableau 1.3 présente la planification détaillée des sprints, et la figure 1.7 illustre leur répartition temporelle.

Tab 1.3 : Planification des sprints – Projet FinBank

Sprint	**Objectif**	**Début**	**Fin**
Sprint 0	Analyse et conception (UML, BPMN, base de données)	02/02/2026	01/03/2026
Sprint 1	Backend Spring Boot, sécurité, JWT, JPA et API REST	02/03/2026	29/03/2026
Sprint 2	Fonctionnalités bancaires et opérations sur les comptes	30/03/2026	26/04/2026
Sprint 3	Assistant bancaire et base de connaissances RAG	27/04/2026	24/05/2026
Sprint 4	Workflows BPMN et interfaces React/TypeScript	25/05/2026	21/06/2026
Sprint 5	Tests, documentation, déploiement Docker et workflow Git	22/06/2026	19/07/2026
	Durée totale	**24 semaines**	

Chaque colonne du diagramme correspond à une semaine ; les repères mensuels indiquent le mois du lundi qui débute la semaine concernée.

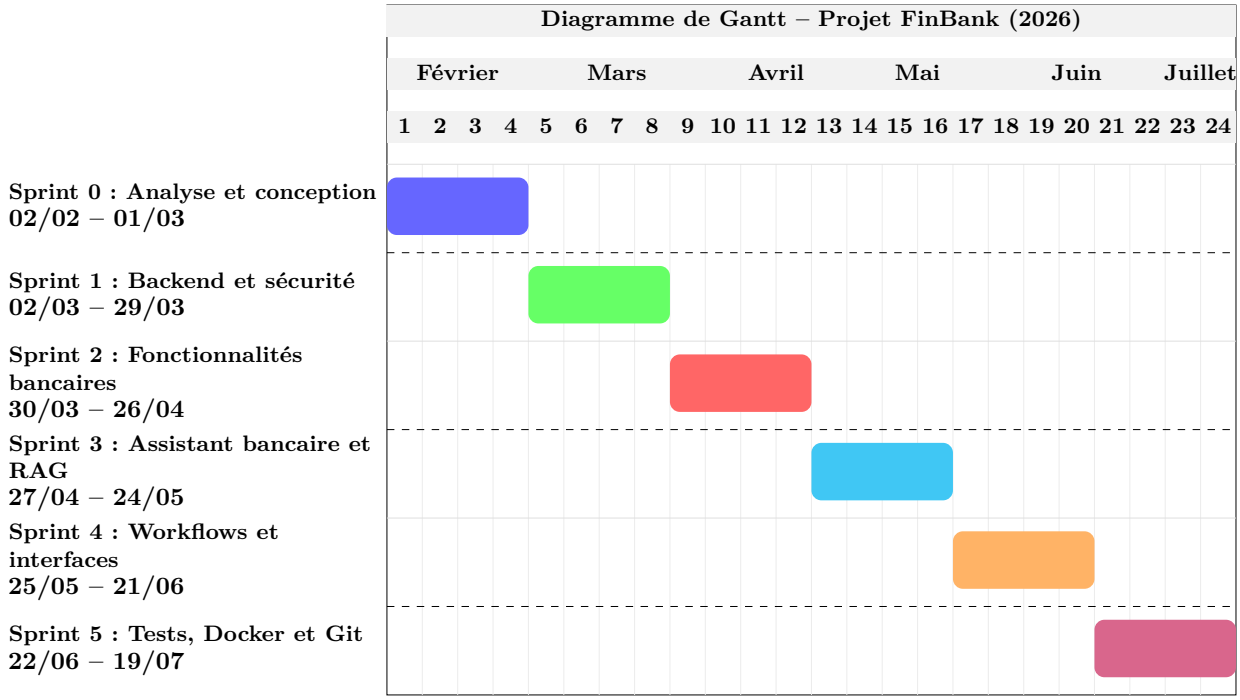


Fig 1.7 : Diagramme de Gantt du projet FinBank (février – juillet 2026)

1.2.4 Conclusion

La méthodologie **Agile Scrum** a ainsi fourni un cadre de travail itératif et adapté à la durée de **six mois** du projet. Elle a permis de livrer régulièrement des versions fonctionnelles de la plateforme, de recueillir les retours du Product Owner à chaque fin de sprint et d'améliorer continuellement la solution.

2

Sprint 0 : Analyse et spécification des besoins

Introduction

Après avoir présenté le cadre général du projet, ce chapitre est consacré à l'**analyse** et à la **spécification des besoins** de la plateforme *FinBank*. Cette étape fondamentale permet d'identifier les acteurs du système, de définir les fonctionnalités attendues et de préciser les contraintes fonctionnelles et non fonctionnelles auxquelles la solution doit répondre.

Nous présentons d'abord les **langages de modélisation** utilisés pour représenter la structure et les processus du système. Nous décrivons ensuite les **acteurs** ainsi que les **besoins fonctionnels** et **non fonctionnels** de la plateforme. Enfin, nous présentons l'organisation du projet selon la méthodologie **Agile Scrum**.

2.1 Langages et outils de modélisation

Pour concevoir la plateforme *FinBank*, nous avons utilisé deux langages de modélisation complémentaires : **UML** pour la modélisation orientée objet, et **BPMN** pour la modélisation des processus métier.

2.1.1 Unified Modeling Language (UML)

L'*Unified Modeling Language* (UML) [5] est un langage graphique de modélisation objet, standardisé par l'*Object Management Group* (OMG). Il permet de représenter les fonctionnalités, la structure et les interactions d'un système logiciel à l'aide de différents types de diagrammes normalisés. Sa nature standardisée facilite la communication entre les différents acteurs du projet et garantit une documentation technique pérenne.

Dans le cadre du projet *FinBank*, UML a été utilisé pour produire les diagrammes suivants :

- le **diagramme de cas d'utilisation global**, présentant les acteurs et les fonctionnalités du système ;
- les **diagrammes de cas d'utilisation détaillés** par module (client, administrateur, agent IA) ;
- le **diagramme de classes global**, décrivant les entités métier (Compte, Transaction, Carte, Investissement, Document RAG, etc.) et leurs relations ;
- les **diagrammes de séquence**, illustrant les interactions entre les composants pour chaque scénario clé (authentification, virement, traitement RAG, validation BPMN) ;
- le **diagramme de déploiement**, présentant l'architecture technique et l'organisation des conteneurs.

Ces diagrammes ont été réalisés à l'aide de l'outil **Visual Paradigm**, qui offre un support complet d'UML 2.5 et permet l'export des diagrammes dans des formats de haute qualité adaptés à la documentation technique.

2.1.2 Business Process Model and Notation (BPMN)

Le *Business Process Model and Notation* (BPMN) [6] est un standard de modélisation graphique des processus métier, également maintenu par l'OMG. Il permet de représenter de manière précise les différentes étapes d'un processus, les décisions, les intervenants (utilisateurs ou systèmes) et les flux d'information. BPMN est particulièrement adapté aux processus nécessitant une **validation** et une **traçabilité** complètes.

Dans le projet *FinBank*, BPMN a été utilisé pour modéliser trois processus métier sensibles :

- la **validation des virements importants** (au-delà d'un seuil de sensibilité) ;

— la **gestion des suspicions de fraude**, déclenchée par le système ou par une réclamation client ;

— la **modification des données personnelles sensibles** (numéro de téléphone, adresse, coordonnées bancaires).

La conception de ces processus a été réalisée avec l'outil **Camunda Modeler**, un éditeur graphique BPMN 2.0 qui génère automatiquement des fichiers XML standardisés. Ces fichiers sont ensuite déployés et exécutés par le moteur **Flowable**, intégré au backend Spring Boot de *FinBank*. Cette séparation entre **conception** (Camunda Modeler) et **exécution** (Flowable) permet de garantir la conformité au standard BPMN 2.0 tout en assurant une intégration native avec l'écosystème Java.

2.1.3 Complémentarité des deux langages

L'utilisation conjointe d'UML et de BPMN permet de couvrir l'ensemble des besoins de modélisation du projet :

Tab 2.1 : Complémentarité d'UML et BPMN dans FinBank

Aspect	**UML**	**BPMN**
Objectif	Structure et comportement du système	Processus métier
Public cible	Développeurs, architectes	Analystes métier, administrateurs
Standard	OMG UML 2.5	OMG BPMN 2.0
Outil de conception	Visual Paradigm	Camunda Modeler
Exécution	Non exécutable	Exécutable par Flowable
Utilisation dans FinBank	Cas d'utilisation, classes, séquences	Validation des virements, fraude, données sensibles

Cette complémentarité permet de bénéficier à la fois d'une modélisation rigoureuse du système (via UML) et d'une orchestration exécutable des processus sensibles (via BPMN et

Flowable).

2.2 Analyse et spécification des besoins

L'application *FinBank* doit être **sécurisée**, **évolutive**, **performante** et **conviviale**. Elle doit permettre aux clients d'effectuer leurs opérations bancaires courantes, de gérer leurs cartes et leurs investissements, ainsi que d'interagir avec un **assistant bancaire intelligent**.

Les opérations sensibles doivent faire l'objet d'un **traitement spécifique** afin de garantir leur validation, leur traçabilité et leur conformité. Cette section présente les acteurs du système ainsi que les besoins fonctionnels et non fonctionnels identifiés.

2.2.1 Identification des acteurs

Un **acteur** est une personne ou un système qui interagit avec l'application afin d'utiliser ses services ou de participer à l'exécution d'un processus. Pour le projet *FinBank*, les acteurs identifiés sont les suivants. Les **acteurs principaux** utilisent directement la plateforme et déclenchent des cas d'utilisation pour atteindre un objectif métier. Les **acteurs secondaires** sont des systèmes ou composants sollicités par FinBank pour contribuer au traitement d'une demande, sans l'initier eux-mêmes :

- **Acteur principal – le client** : utilisateur principal de la plateforme. Il peut consulter son compte et son solde, consulter l'historique de ses transactions, effectuer des virements, gérer ses cartes et ses paiements PIX, suivre ses investissements, consulter les analyses, gérer son profil, soumettre une réclamation et utiliser l'assistant bancaire.
- **Acteur principal – l'administrateur ou l'agent de support** : utilisateur chargé de superviser la plateforme. Il peut gérer les utilisateurs, traiter les tâches relatives aux opérations sensibles, suivre les réclamations et consulter les journaux d'audit.
- **Acteur secondaire – le service d'intelligence artificielle** : composant chargé d'analyser les messages formulés en langage naturel et d'identifier l'intention du client. Dans la configuration principale, cette fonction est assurée par un modèle local exécuté avec **Ollama**. Un traitement de secours fondé sur des règles déterministes est également disponible ; OpenAI ou Azure OpenAI peuvent être utilisés de manière **optionnelle** selon la configuration du système.

— **Acteur secondaire – le moteur Flowable BPMN** : moteur intégré au backend, chargé d'orchestrer les processus nécessitant une validation, notamment les virements importants, les suspicions de fraude et les modifications de données personnelles sensibles.

2.2.2 Identification des besoins

Les besoins du système *FinBank* ont été identifiés à partir des objectifs fonctionnels du projet, des attentes des utilisateurs cibles (clients et administrateurs) et des contraintes propres au secteur bancaire. Ils sont classés en deux catégories : les **besoins fonctionnels**, qui décrivent les services rendus par la plateforme, et les **besoins non fonctionnels**, qui définissent les exigences de qualité à respecter.

2.2.2.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les services que la plateforme *FinBank* doit fournir à ses utilisateurs. Ils sont organisés selon les principaux modules fonctionnels du système.

Gestion des comptes et authentification

— création de compte et inscription des nouveaux utilisateurs ;
— authentification sécurisée par jetons **JWT** ;
— gestion des sessions et de la sécurité du profil ;
— récupération et modification du mot de passe.

Services bancaires

— consultation des comptes, des soldes et de l'historique des transactions ;
— réalisation de dépôts, retraits et virements ;
— gestion des cartes bancaires et des moyens de paiement ;
— gestion des paiements **PIX** ;
— gestion des investissements et suivi de leur évolution.

Assistant bancaire intelligent

— interaction en **langage naturel** avec l'assistant ;

— détection d'intention et analyse sémantique des demandes ;
— recherche d'informations dans la base de connaissances (**RAG**) ;
— exécution automatique des opérations courantes confirmées ;
— redirection des opérations sensibles vers une validation.

Validation et orchestration BPMN

— validation des opérations sensibles via des workflows **Flowable** ;
— gestion des tâches de validation (virements importants, suspicions de fraude, modifications de données sensibles) ;
— suivi de l'état des processus en cours.

Administration et supervision

— gestion des utilisateurs et des rôles ;
— administration de la base de connaissances **RAG** (création, modification, réindexation des documents) ;
— supervision des journaux d'audit et des événements de sécurité ;
— traitement des tickets et des demandes de support.

Communication et notifications

— envoi de notifications aux clients (transactions, entretiens, validations) ;
— notification des administrateurs pour les tâches sensibles ;
— suivi et gestion des notifications depuis l'interface.

Le tableau 2.2 synthétise la répartition des besoins fonctionnels par acteur.

2.2.2.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les **contraintes de qualité** que la plateforme *FinBank* doit respecter. Ils sont regroupés en cinq catégories principales.

Performance

— le temps de réponse des opérations courantes doit rester court et adapté à une utilisation interactive ;

Tab 2.2 : Répartition des besoins fonctionnels par acteur

Besoin fonctionnel	**Client**	**Admin**	**Agent IA**
Authentification	✓	✓	—
Consultation du compte	✓	—	—
Virements / opérations	✓	✓	✓
Gestion des cartes / PIX	✓	—	—
Gestion des investissements	✓	—	—
Utilisation de l'assistant	✓	—	✓
Détection d'intention	—	—	✓
Recherche RAG	—	✓	✓
Validation BPMN	—	✓	✓
Gestion des utilisateurs	—	✓	—
Journaux d'audit	—	✓	—
Notifications	✓	✓	—

— les soldes et les informations fréquemment consultées sont mis en cache à l'aide de **Redis** ;

— les journaux d'audit et certains traitements secondaires sont réalisés de manière **asynchrone** afin de ne pas ralentir les opérations principales ;

— les champs fréquemment utilisés (identifiant utilisateur, date de création) sont correctement **indexés** dans PostgreSQL.

Disponibilité et robustesse

— la plateforme doit rester fonctionnelle même lorsqu'un service d'intelligence artificielle externe n'est pas disponible ;

— **Ollama** constitue le fournisseur IA principal dans l'environnement local, avec un **fallback** basé sur des règles déterministes ;

— les appels vers les services externes sont protégés par des délais d'expiration et une gestion explicite des erreurs ;
— les opérations sensibles sont traitées par des processus BPMN afin de garantir leur validation et leur suivi.

Sécurité

— l'accès aux ressources protégées est sécurisé par une authentification basée sur des jetons **JWT** ;
— les mots de passe sont stockés sous forme chiffrée ;
— les opérations sensibles sont soumises à des règles de validation avant leur exécution ;
— les actions importantes sont enregistrées dans les journaux d'audit ;
— les clés et paramètres sensibles sont externalisés dans des variables d'environnement.

Ergonomie et interface utilisateur

— l'interface web doit être claire, responsive et facile à utiliser ;
— elle doit permettre aux clients de consulter leurs comptes, leurs transactions, leurs cartes et leurs investissements ;
— l'interface conversationnelle doit afficher clairement les demandes, les réponses et les éventuelles demandes de confirmation ;
— les messages d'erreur doivent être compréhensibles et ne pas exposer de détails techniques inutiles.

Maintenabilité et évolutivité

— le backend est organisé en composants et services clairement séparés ;
— la configuration de l'application est externalisée dans `application.yml` et dans des variables d'environnement ;
— les API sont documentées à l'aide de **OpenAPI/Swagger** ;
— l'architecture permet l'ajout de nouveaux services bancaires, de nouveaux intents et de nouveaux processus BPMN ;
— le code et les processus métier sont accompagnés d'une documentation technique facilitant leur maintenance.

Le tableau 2.3 résume les catégories de besoins non fonctionnels et leurs indicateurs associés.

Tab 2.3 : Synthèse des besoins non fonctionnels

Catégorie	**Exigence principale**
Performance	Temps de réponse court, cache Redis, traitements asynchrones
Disponibilité	Fallback IA, gestion des erreurs, tolérance aux pannes
Sécurité	JWT, chiffrement, audit, gestion des secrets
Ergonomie	Interface responsive, messages clairs, UX fluide
Maintenabilité	Séparation des services, OpenAPI, documentation

2.3 Diagrammes UML

2.3.1 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation (figure 2.1) présente les interactions entre les acteurs et le système FinBank. Il distingue clairement les rôles du client et de l'administrateur, ainsi que les systèmes secondaires (Agent IA, Moteur BPMN Flowable).

Les cas d'utilisation sont organisés en trois packages :

— **Services client** : authentification, consultation, opérations bancaires, gestion du profil ;

— **Services intelligents** : interprétation de la demande, exécution d'opération, recherche RAG, soumission à validation ;

— **Administration** : gestion des utilisateurs, gestion de la base RAG, orchestration BPMN, traitement des tâches sensibles.

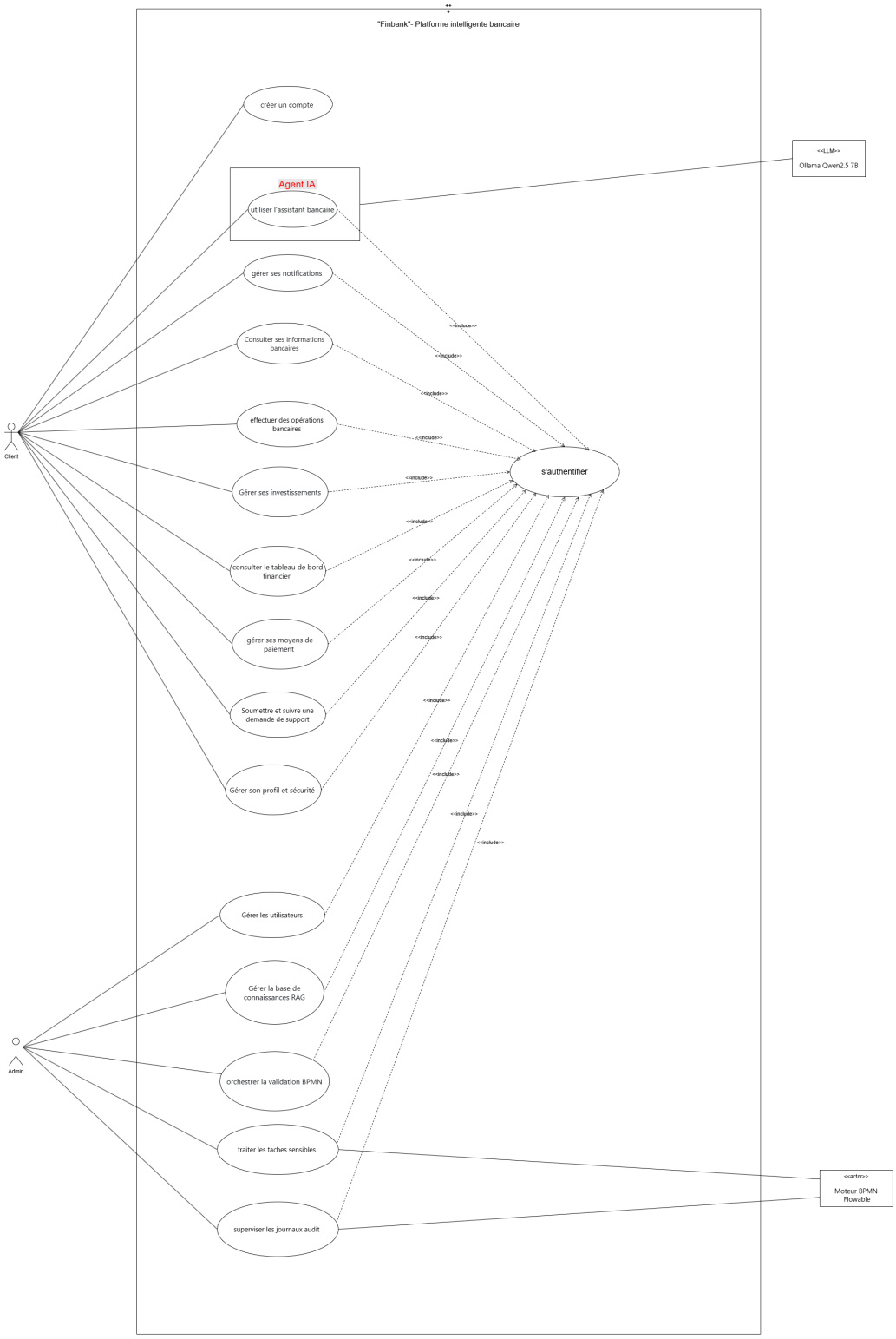


Fig 2.1 : Diagramme de cas d'utilisation global – FinBank

2.3.2 Diagramme de classes global

Le diagramme de classes (figure 2.2) présente les entités principales du système et leurs relations :

- `User` : entité centrale avec les rôles `USER` et `ADMIN` ;
- `Account` et `Transaction` : gestion des comptes et des opérations bancaires ;
- `Card` et `Investment` : moyens de paiement et investissements ;
- `SupportTicket` et `WorkflowTask` : gestion des tâches sensibles, liées à une instance Flowable via `flowableProcessId` ;
- `Message`, `Session` et `BankingAgent` : interactions avec l'assistant intelligent ;
- `RagDocument` : base de connaissances du mécanisme RAG.

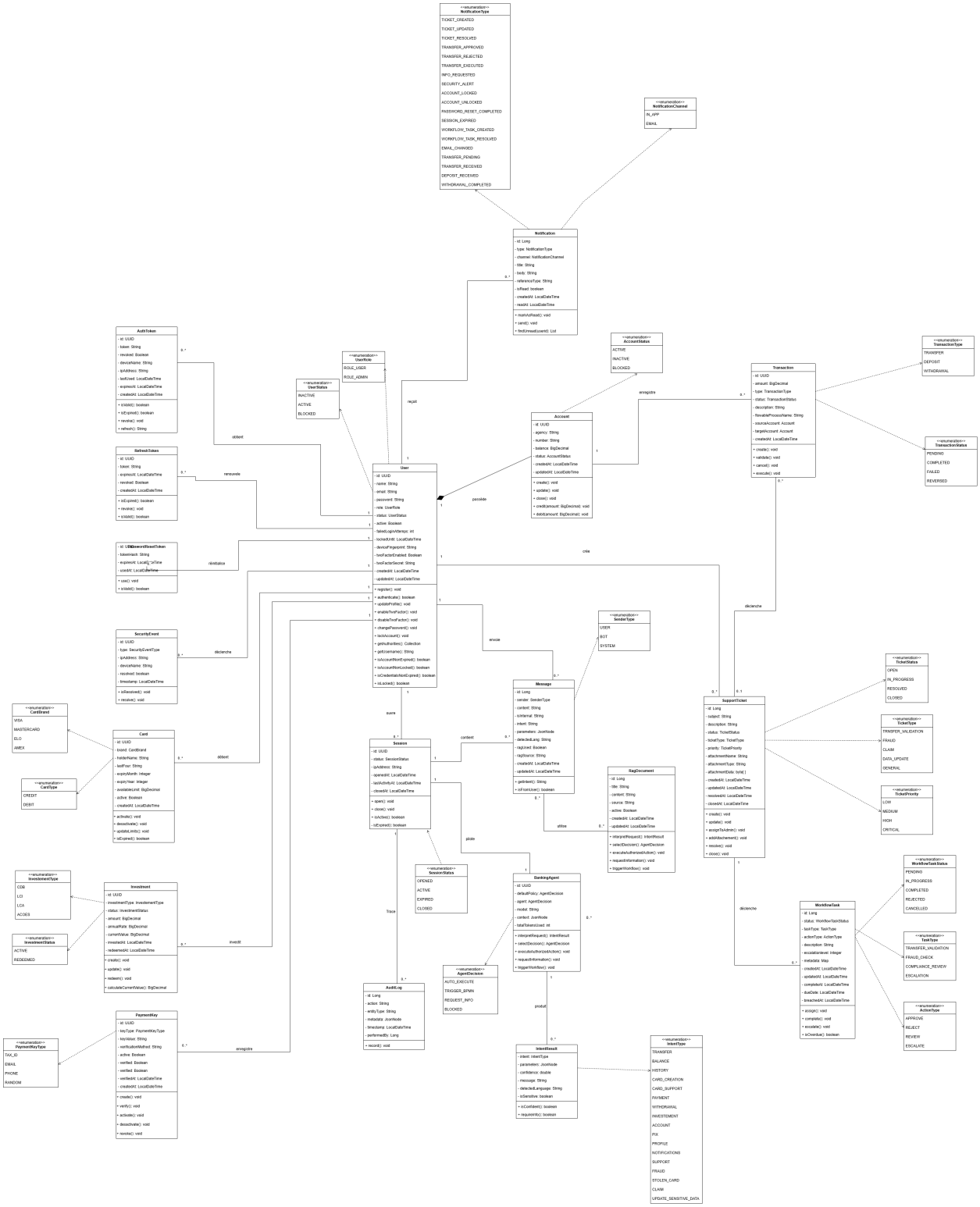


Fig 2.2 : Diagramme de classes global – FinBank

2.4 Modélisation des processus métier avec BPMN

En complément des diagrammes UML, les processus métier nécessitant plusieurs étapes de traitement sont modélisés avec la notation BPMN (*Business Process Model and Notation*). Cette modélisation permet de représenter l'enchaînement des activités, les décisions et les interactions entre les acteurs et le système.

Trois processus métier sont considérés dans le projet FinBank : la validation d'un virement, le traitement d'un signalement de fraude et la validation d'une demande de mise à jour des données.

2.4.1 Processus de validation d'un virement

Ce processus décrit le traitement d'un virement soumis à validation. Après la détection d'une situation nécessitant un contrôle, la demande est enregistrée et transmise au processus de validation. Une tâche est ensuite examinée par un administrateur, qui prend une décision selon les règles prévues par le processus. Le résultat du traitement est alors communiqué aux parties concernées. La figure 2.3 illustre les étapes de ce traitement.

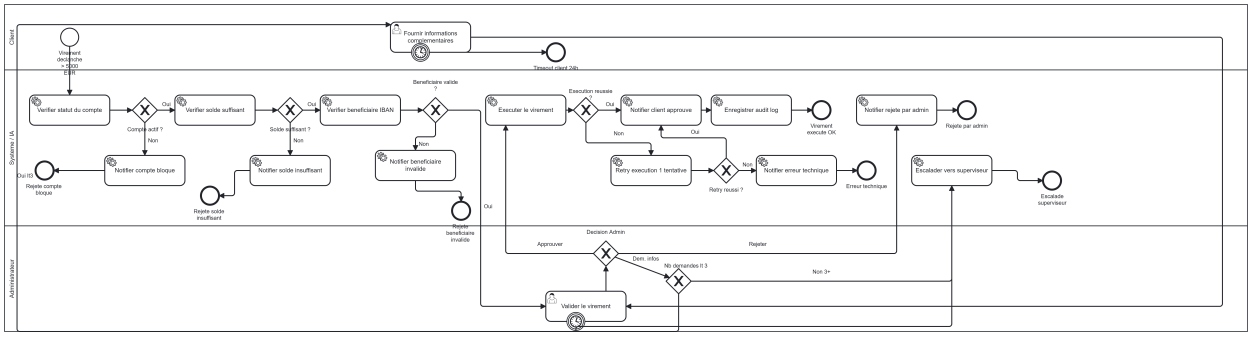


Fig 2.3 : Processus BPMN de validation d'un virement

2.4.2 Processus de traitement d'un signalement de fraude

Ce processus représente le traitement d'un signalement lié à une suspicion de fraude. Le signalement est enregistré sous la forme d'une demande à examiner, puis transmis au processus correspondant. Les tâches et décisions de traitement permettent de suivre l'évolution du dossier jusqu'à sa résolution. La figure 2.4 présente l'enchaînement de ces étapes.

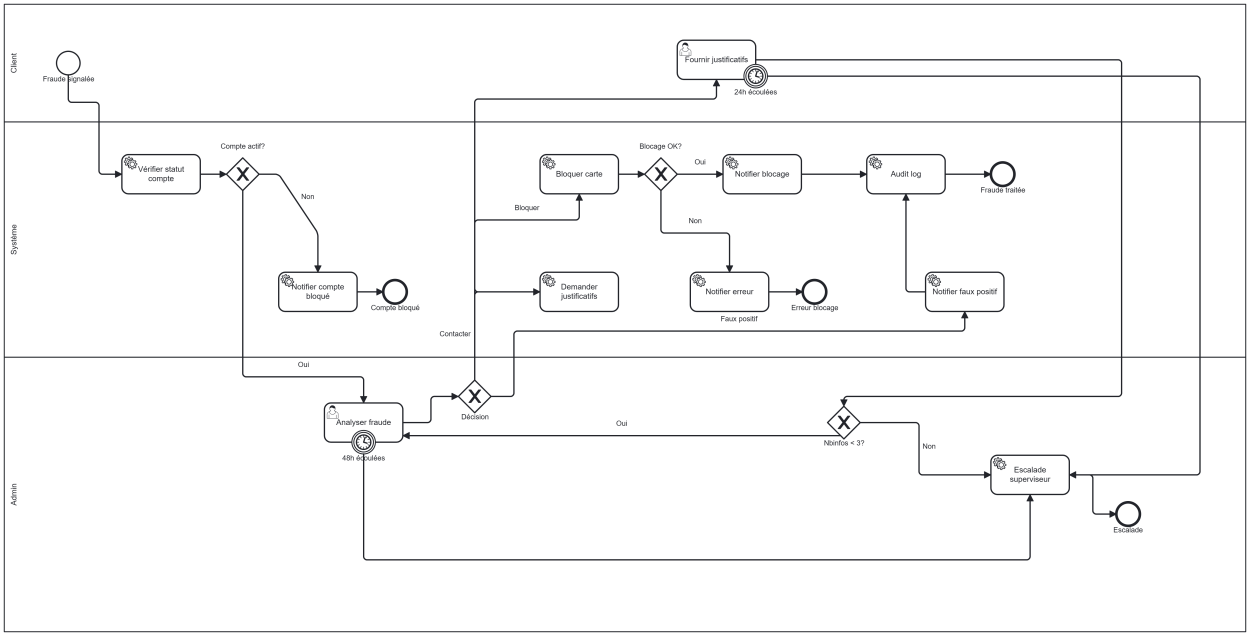


Fig 2.4 : Processus BPMN de traitement d'un signalement de fraude

2.4.3 Processus de validation d'une mise à jour des données

Ce processus concerne les demandes de mise à jour de données nécessitant une vérification, notamment lorsqu'une modification doit être approuvée avant d'être appliquée. La demande est enregistrée, examinée par un administrateur et traitée selon la décision prise. Le processus assure ainsi le suivi de la demande et de son résultat. La figure 2.5 présente ce processus de validation.

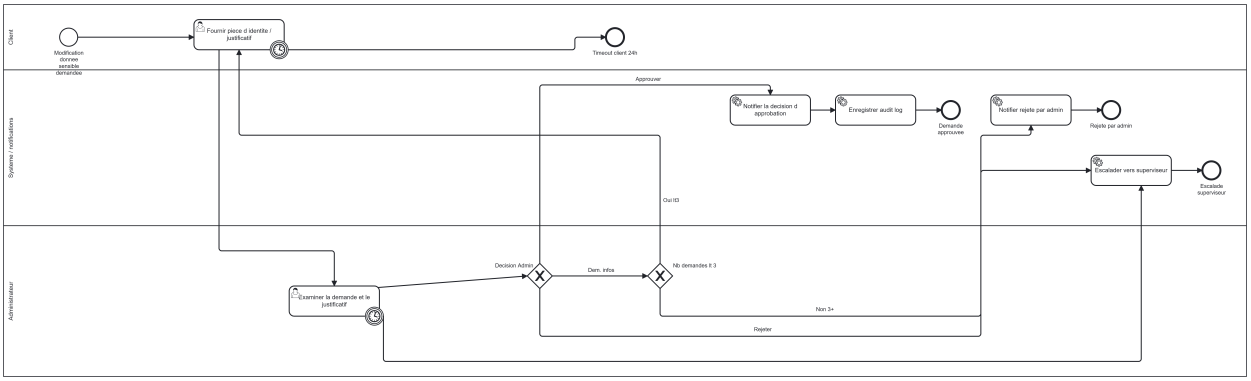


Fig 2.5 : Processus BPMN de validation d'une mise à jour des données

Ces diagrammes décrivent la conception des processus métier. Leur déclenchement et leur exécution nécessitent leur intégration au backend et leur orchestration par le moteur Flowable.

2.5 Conduite du projet avec Scrum

Cette section décrit l'organisation de l'équipe et la gestion du backlog selon la méthodologie Scrum.

2.5.1 Équipe et rôles

Scrum repose sur trois rôles clés :

— **Product Owner (PO)** : représentant des clients et des utilisateurs finaux. Dans ce projet, ce rôle est assuré par l'encadrante professionnelle, **Mme Feriel JRIDI**, qui a défini les besoins fonctionnels et validé les livrables à chaque fin de sprint.

— **Scrum Master (SM)** : garant du bon déroulement du processus Scrum. Ce rôle est assuré par l'encadrant académique, **M. Haythem SAOUIDI**, qui a veillé au respect de la méthodologie et à la levée des obstacles rencontrés.

— **Development Team** : équipe de développement. Dans le cadre de ce projet individuel, l'équipe est réduite à une seule personne, **Raef GAEED**, qui a assuré la conception, le développement et les tests de la plateforme, tout en endossant ponctuellement certaines responsabilités de PO et de SM.

2.5.2 Le Product Backlog

À partir des cas d'utilisation identifiés précédemment, les besoins fonctionnels sont traduits en *user stories* et regroupés dans le Product Backlog. Chaque user story exprime un besoin du point de vue de l'utilisateur et s'accompagne de critères d'acceptation permettant de vérifier le résultat attendu.

Les éléments du backlog sont priorisés selon la méthode **MoSCoW** [7] :

— **M – Must have** : fonctionnalité indispensable au fonctionnement principal de la plateforme.

— **S – Should have** : fonctionnalité importante, mais non bloquante pour une première version.

— **C – Could have** : fonctionnalité utile, pouvant être réalisée si le temps le permet.

— **W – Won't have for now** : fonctionnalité écartée du périmètre actuel et reportée à une version ultérieure.

Le tableau 2.4 présente un extrait du backlog produit de la plateforme FinBank. Dans la colonne « Priorité », les lettres **M**, **S** et **C** renvoient aux catégories définies ci-dessus. Cet extrait ne contient aucun élément de catégorie **W**, réservée aux fonctionnalités exclues du périmètre actuel.

Tab 2.4 : Backlog produit de la plateforme FinBank

ID	**User Story**	**Critères d'acceptation**	**Priorité**
US01	En tant que visiteur, je veux créer un compte afin d'accéder aux services proposés par la plateforme.	– Le formulaire permet de saisir les informations demandées. – Les champs obligatoires sont vérifiés avant la création. – L'adresse électronique doit être disponible pour être associée au nouveau compte. – Après la création, l'utilisateur peut s'authentifier.	M
US02	En tant qu'utilisateur, je veux m'authentifier et me déconnecter afin de contrôler l'accès à mon espace personnel.	– Des identifiants valides permettent d'accéder aux fonctionnalités autorisées selon le rôle de l'utilisateur. – Des identifiants incorrects ne permettent pas l'accès. – La déconnexion invalide le jeton de renouvellement utilisé. – Un jeton expiré ou révoqué ne permet pas de renouveler l'accès.	M
US03	En tant qu'utilisateur, je veux réinitialiser mon mot de passe lorsque je l'ai oublié afin de retrouver l'accès à mon compte.	– L'utilisateur peut soumettre une demande de réinitialisation. – Un jeton valide permet de définir un nouveau mot de passe. – Un jeton expiré ou déjà utilisé est refusé. – Le nouveau mot de passe est enregistré après validation.	S

Suite à la page suivante

Tab 2.4 – suite du backlog produit

ID	**User Story**	**Critères d'acceptation**	**Priorité**
US04	En tant qu'utilisateur, je veux activer l'authentification à deux facteurs afin de renforcer la sécurité de mon compte.	– L'utilisateur peut demander les informations nécessaires à la configuration du second facteur. – L'activation est confirmée uniquement après validation d'un code correct. – Un code incorrect ne permet pas d'activer le second facteur. – Lorsque cette option est activée, le parcours d'authentification tient compte de la vérification supplémentaire.	S
US05	En tant qu'utilisateur connecté, je veux consulter et modifier les informations de mon profil afin de maintenir mes données à jour.	– Le profil présente les informations disponibles de l'utilisateur connecté. – L'utilisateur peut modifier son nom. – Une modification d'adresse électronique nécessite le mot de passe actuel et une demande de mise à jour approuvée. – Les informations d'un autre utilisateur ne sont pas modifiables depuis cet espace.	S
US06	En tant qu'utilisateur connecté, je veux changer mon mot de passe afin de protéger mon compte.	– Le mot de passe actuel est vérifié avant le changement. – Si le mot de passe actuel est incorrect, la modification est refusée. – Le nouveau mot de passe est enregistré après validation. – Les jetons de l'utilisateur sont révoqués après le changement.	S

Suite à la page suivante

Tab 2.4 – suite du backlog produit

ID	**User Story**	**Critères d'acceptation**	**Priorité**
US07	En tant que client, je veux consulter les informations de mon compte bancaire afin de connaître mon solde et mes coordonnées de compte.	– Le client authentifié peut consulter son compte. – Les informations présentées correspondent au compte associé au client. – Le solde affiché correspond aux données disponibles dans la plateforme. – Un client ne peut pas consulter le compte d'un autre utilisateur.	M
US08	En tant que client, je veux effectuer un dépôt sur mon compte afin d'en augmenter le solde.	– Le client peut soumettre un dépôt avec un montant valide. – Le dépôt accepté est ajouté au solde du compte. – Une transaction est créée et apparaît dans l'historique. – Une notification liée au dépôt est créée lorsqu'elle est prise en charge par le système.	M
US09	En tant que client, je veux effectuer un retrait afin de débiter mon compte.	– Le client peut soumettre un retrait avec un montant valide. – Le montant du retrait ne peut pas dépasser le solde disponible. – Un retrait accepté met à jour le solde et crée une transaction. – Une notification liée au retrait est créée lorsqu'elle est prise en charge par le système.	M

Suite à la page suivante

Tab 2.4 – suite du backlog produit

ID	**User Story**	**Critères d'acceptation**	**Priorité**
US10	En tant que client, je veux effectuer un virement vers un autre compte afin de transférer des fonds.	– Le client peut indiquer un compte bénéficiaire et un montant. – Le compte bénéficiaire doit être retrouvé par la plateforme. – Un virement accepté débite le compte source et crédite le compte cible. – La transaction créée est consultable dans l'historique.	M
US11	En tant que client, je veux que les virements dépassant le seuil configuré soient soumis à validation afin qu'ils soient examinés avant leur exécution.	– Le seuil est configurable ; sa valeur par défaut dans le projet est de 5 000. – La devise de ce seuil n'est pas précisée dans la configuration consultée. – Un virement supérieur au seuil est enregistré avec un statut en attente. – Selon la décision de validation, le processus BPMN peut être déclenché. – Le client est informé lorsque le virement est en attente de traitement.	M
US12	En tant que client, je veux enregistrer et consulter mes clés de paiement afin de les utiliser pour effectuer des paiements.	– Le client peut enregistrer une clé en fournissant les informations demandées. – Le client peut consulter ses clés enregistrées. – Les types de clés pris en charge sont ceux proposés par la plateforme. – Une clé de paiement recherchée doit être connue du système pour être utilisée comme destination.	S

Suite à la page suivante

Tab 2.4 – suite du backlog produit

ID	**User Story**	**Critères d'acceptation**	**Priorité**
US13	En tant que client, je veux effectuer un paiement à l'aide d'une clé de paiement afin de transférer des fonds à un bénéficiaire.	– Le client peut saisir une clé de paiement et un montant. – La clé saisie doit correspondre à une clé connue du système. – Une opération acceptée crée une transaction. – Le résultat de l'opération est retourné au client.	S
US14	En tant que client, je veux consulter l'historique de mes transactions afin de suivre les opérations effectuées sur mon compte.	– L'historique présente les transactions associées au compte du client. – Chaque entrée présente les informations disponibles, notamment le type, le montant, le statut et la date. – Le client peut demander le reçu PDF d'une transaction qui lui appartient. – Le téléchargement du reçu d'une transaction appartenant à un autre utilisateur est refusé.	M
US15	En tant que client, je veux gérer mes cartes afin de maintenir à jour les cartes associées à mon espace.	– Le client peut consulter la liste de ses cartes. – Il peut ajouter une carte avec les informations demandées. – Il peut modifier une carte qui lui appartient. – Il peut supprimer une carte de son espace. – Les opérations sur les cartes d'un autre utilisateur sont refusées.	S

Suite à la page suivante

Tab 2.4 – suite du backlog produit

ID	**User Story**	**Critères d'acceptation**	**Priorité**
US16	En tant que client, je veux consulter et gérer mes investissements afin de suivre leur valeur et leur statut.	– Le client peut consulter ses investissements. – Il peut créer un investissement avec un type et un montant valides. – Il peut demander le rachat d'un investissement qui lui appartient. – Le statut et les informations de l'investissement sont mis à jour après le traitement.	S
US17	En tant que client, je veux consulter un tableau de bord financier afin d'obtenir une synthèse de mes données bancaires.	– Le tableau de bord présente les indicateurs et visualisations disponibles pour le client. – Les données sont calculées à partir des informations accessibles à ce client. – Les transactions affichées correspondent à ses opérations. – Une absence de données est représentée sans inventer de résultats.	S
US18	En tant qu'utilisateur, je veux dialoguer avec l'assistant bancaire afin de poser des questions ou de formuler une demande en langage naturel.	– L'utilisateur peut envoyer un message depuis l'interface de conversation. – Les messages et réponses disponibles sont affichés dans la conversation. – La demande peut être analysée afin d'identifier une intention et ses paramètres. – Une demande ambiguë ou insuffisamment comprise ne doit pas être présentée comme une opération exécutée.	M

Suite à la page suivante

Tab 2.4 – suite du backlog produit

ID	**User Story**	**Critères d'acceptation**	**Priorité**
US19	En tant qu'utilisateur, je veux créer et consulter mes sessions de conversation afin de retrouver mes échanges avec l'assistant.	– L'utilisateur peut créer une session et consulter ses sessions. – Il peut consulter les messages associés à une session. – Il peut ouvrir ou fermer une session. – Une session expirée ou fermée n'est plus considérée comme active.	S
US20	En tant qu'utilisateur, je veux consulter l'activité de l'agent bancaire associé à ma session afin de comprendre le suivi de mes demandes.	– L'utilisateur peut consulter les agents associés à ses sessions. – Les activités enregistrées sont présentées lorsqu'elles sont disponibles. – La trace peut inclure l'intention reconnue et la décision de l'agent. – Un utilisateur ne peut pas consulter les données d'agent d'un autre utilisateur.	S
US21	En tant qu'utilisateur, je veux rechercher des informations dans la base de connaissances afin d'obtenir des réponses liées aux services bancaires.	– L'utilisateur peut soumettre une question à la fonction de recherche. – Les résultats pertinents sont retournés lorsqu'ils sont disponibles. – Le contenu et la source du document sont présentés lorsque ces informations existent. – En l'absence de résultat, le système indique qu'aucune correspondance n'a été trouvée.	S

Suite à la page suivante

Tab 2.4 – suite du backlog produit

ID	**User Story**	**Critères d'acceptation**	**Priorité**
US22	En tant qu'utilisateur, je veux envoyer du texte ou un document à la fonction d'extraction afin d'obtenir les informations reconnues.	– La fonction accepte du texte ou un fichier pris en charge. – Le résultat contient les champs extraits lorsqu'ils sont détectés. – Le type de document est retourné lorsqu'il peut être déterminé. – Cette fonctionnalité est exposée par l'API et ne doit pas être présentée comme une page dédiée si aucune page correspondante n'existe dans l'interface.	C
US23	En tant que client, je veux soumettre une demande de support afin d'obtenir de l'aide en cas de difficulté.	– Le client peut créer un ticket à partir d'un sujet et d'une description. – Il peut préciser le type de demande lorsque cette information est proposée. – Il peut joindre un fichier lorsque l'option correspondante est utilisée. – Le ticket créé est associé à l'utilisateur.	M
US24	En tant que client, je veux consulter mes tickets de support afin de suivre l'avancement de mes demandes.	– Le client peut consulter la liste de ses tickets. – Il peut ouvrir le détail d'un ticket. – Le statut du ticket est visible lorsqu'il est disponible. – Les tickets d'un autre utilisateur ne sont pas présentés dans sa liste personnelle.	S

Suite à la page suivante

Tab 2.4 – suite du backlog produit

ID	**User Story**	**Critères d'acceptation**	**Priorité**
US25	En tant qu'administrateur, je veux consulter et traiter les tickets de support afin d'assurer le suivi des demandes des clients.	– L'administrateur peut consulter les tickets et leurs détails. – Il peut effectuer les actions proposées, notamment prendre en charge un ticket, demander des informations, envoyer une mise à jour ou résoudre le ticket. – Le statut du ticket reflète l'action effectuée. – Un brouillon proposé par l'assistant reste à examiner avant son envoi.	M
US26	En tant qu'administrateur, je veux consulter les tâches et processus BPMN afin de traiter les opérations nécessitant une validation.	– L'administrateur peut consulter les tâches et les processus disponibles. – Il peut consulter les détails et la trace d'un processus lorsque ces informations sont disponibles. – Il peut compléter une tâche en transmettant une décision d'approbation ou de rejet. – Le traitement respecte l'état du processus concerné.	M
US27	En tant qu'administrateur, je veux consulter la liste des utilisateurs et bloquer ou débloquer un compte afin de gérer les accès à la plateforme.	– L'administrateur peut consulter les utilisateurs avec pagination. – Il peut bloquer un compte et préciser un motif facultatif. – Il peut débloquer un compte précédemment bloqué. – Ces actions sont réservées au rôle administrateur.	S

Suite à la page suivante

Tab 2.4 – suite du backlog produit

ID	**User Story**	**Critères d'acceptation**	**Priorité**
US28	En tant qu'administrateur, je veux consulter les journaux d'audit afin de suivre les actions enregistrées par la plateforme.	– L'administrateur peut consulter les journaux d'audit. – Les entrées disponibles présentent l'action et son statut. – La liste peut être parcourue par pages. – L'accès est réservé à un administrateur.	S
US29	En tant qu'administrateur, je veux consulter et résoudre les événements de sécurité afin d'assurer le suivi des alertes enregistrées.	– L'administrateur peut consulter les événements de sécurité enregistrés. – Chaque événement présente son type et son état de résolution. – L'administrateur peut marquer un événement comme résolu. – La liste peut être parcourue par pages.	S
US30	En tant qu'administrateur, je veux gérer les documents de la base de connaissances afin de maintenir à jour les contenus utilisés par la recherche.	– L'administrateur peut consulter les documents enregistrés. – Il peut créer, modifier et supprimer un document. – Il peut définir si un document est actif. – Il peut demander le réindexage d'un document existant. – Ces opérations sont réservées à un administrateur.	S
US31	En tant qu'utilisateur, je veux choisir la langue de l'interface afin de consulter les fonctionnalités dans la langue proposée qui me convient.	– L'interface propose le français et l'anglais. – Les libellés traduits apparaissent dans la langue sélectionnée. – La réponse de l'assistant peut également tenir compte de la langue détectée lorsqu'elle est disponible.	C

Suite à la page suivante

Tab 2.4 – suite du backlog produit

ID	User Story	Critères d'acceptation	Priorité
US32	En tant qu'administrateur, je veux consulter les indicateurs du tableau de bord d'administration afin de superviser les données disponibles sur la plateforme.	– L'accès au tableau de bord est réservé à un administrateur. – Les indicateurs affichés proviennent des données retournées par la plateforme. – Les informations disponibles peuvent inclure les utilisateurs, les tickets, les tâches et les activités récentes. – Une donnée indisponible n'est pas présentée comme une valeur réelle.	S

2.5.3 Planification des sprints

Le tableau 2.5 présente la planification des travaux du projet *FinBank*. Le projet s'étend sur une durée de **six mois**, soit environ **24 semaines**, réparties en **six sprints de quatre semaines**. Chaque sprint porte sur un objectif fonctionnel ou technique et contribue progressivement à la réalisation de la plateforme.

Tab 2.5 : Planification des sprints – Projet FinBank

Sprint	Objectif du sprint	Date de début	Date de fin
Sprint 0	– Analyse des besoins et rédaction du cahier des charges. – Identification des acteurs et des cas d'utilisation. – Réalisation des diagrammes UML et des modèles BPMN. – Conception de la base de données et préparation de l'environnement.	02/02/2026	01/03/2026

Sprint	**Objectif du sprint**	**Date de début**	**Date de fin**
Sprint 1	– Mise en place de l'API Java avec Spring Boot. – Création des entités et des opérations de persistance. – Configuration de Spring Security et de l'authentification JWT. – Documentation des API REST.	02/03/2026	29/03/2026
Sprint 2	– Développement des comptes, dépôts, retraits et virements. – Gestion des cartes et des investissements. – Gestion des clés de paiement et de l'historique des transactions.	30/03/2026	26/04/2026
Sprint 3	– Intégration de la détection d'intention et de l'assistant conversationnel. – Gestion des sessions et des messages. – Mise en place de la recherche RAG et gestion des documents de connaissance.	27/04/2026	24/05/2026
Sprint 4	– Intégration de Flowable et des processus BPMN de validation des virements, de fraude et de mise à jour des données. – Développement des interfaces React et TypeScript pour les clients et les administrateurs. – Connexion des interfaces aux API REST.	25/05/2026	21/06/2026

Sprint	Objectif du sprint	Date de début	Date de fin
Sprint 5	– Exécution des tests et vérification des principaux parcours utilisateur. – Correction des anomalies et finalisation de la documentation. – Déploiement avec Docker et organisation du workflow Git. – Préparation de la livraison et de la soutenance.	22/06/2026	19/07/2026
	Durée totale	**24 semaines**	

Remarque : les modèles BPMN sont conçus avec **Camunda Modeler** et exécutés par le moteur **Flowable** intégré au backend. La recherche RAG utilise **pgvector** pour la recherche sémantique et **Ollama** pour la génération locale des embeddings avec le modèle `nomic-embed-text`. Une recherche de secours fondée sur les fichiers de connaissances est également prévue lorsque la recherche vectorielle n'est pas disponible ou ne retourne pas de résultat pertinent.

2.6 Architecture et outils

2.6.1 Architecture globale de l'application

La figure 2.6 présente l'architecture globale de *FinBank*. L'application repose sur un frontend développé avec React et TypeScript, qui communique avec une API backend réalisée avec Java et Spring Boot. Le backend regroupe la logique métier, la sécurité et l'accès aux données. Il intègre également Flowable pour l'exécution des processus BPMN.

Le tableau 2.6 résume les principaux composants de l'application et leur rôle.

Tab 2.6 : Principaux composants de l'architecture de FinBank

Composant	Rôle
Application FinBank	Plateforme bancaire permettant aux clients d'effectuer des opérations et aux administrateurs de suivre les tickets, les tâches et les événements de sécurité.

Suite du tableau 2.6

Composant	**Rôle**
Frontend (React et TypeScript)	Interface web des clients et des administrateurs : consultation du compte, opérations, investissements, conversations et fonctions d'administration.
Backend (Java et Spring Boot)	API REST qui applique la logique métier, traite les demandes du frontend et coordonne l'accès aux données et aux services intégrés.
Sécurité (Spring Security et JWT)	Authentification et contrôle de l'accès aux fonctionnalités selon le rôle de l'utilisateur. Le projet utilise Spring Security et JWT ; il ne s'appuie pas sur Keycloak.
Moteur Flowable	Moteur BPMN intégré au backend pour exécuter les processus de validation, notamment ceux associés aux virements, à la fraude et à la mise à jour des données.
PostgreSQL et pgvector	Stockage des données métier et prise en charge de la recherche sémantique dans la base documentaire au moyen de pgvector.
Redis	Cache utilisé notamment pour les informations de compte afin de limiter les lectures répétées en base de données.
Kafka	Transmission des événements liés aux transactions pour leur traitement par les composants abonnés.
Services IA et RAG	Détection d'intention et assistance conversationnelle. Les composants Spring AI peuvent utiliser les fournisseurs IA configurés. Pour le RAG, Ollama génère les embeddings utilisés par la recherche pgvector ; une recherche de secours dans les fichiers de connaissances est également prévue.

2.6.2 Architecture logicielle

FinBank adopte une architecture web en couches, organisée autour d'un backend Spring Boot et d'un frontend indépendant. Les contrôleurs du backend exposent les API REST, les services prennent en charge les règles métier et les repositories assurent l'accès aux données persistées avec Spring Data JPA. Le frontend React et TypeScript consomme ces API pour présenter les fonctionnalités destinées aux clients et aux administrateurs.

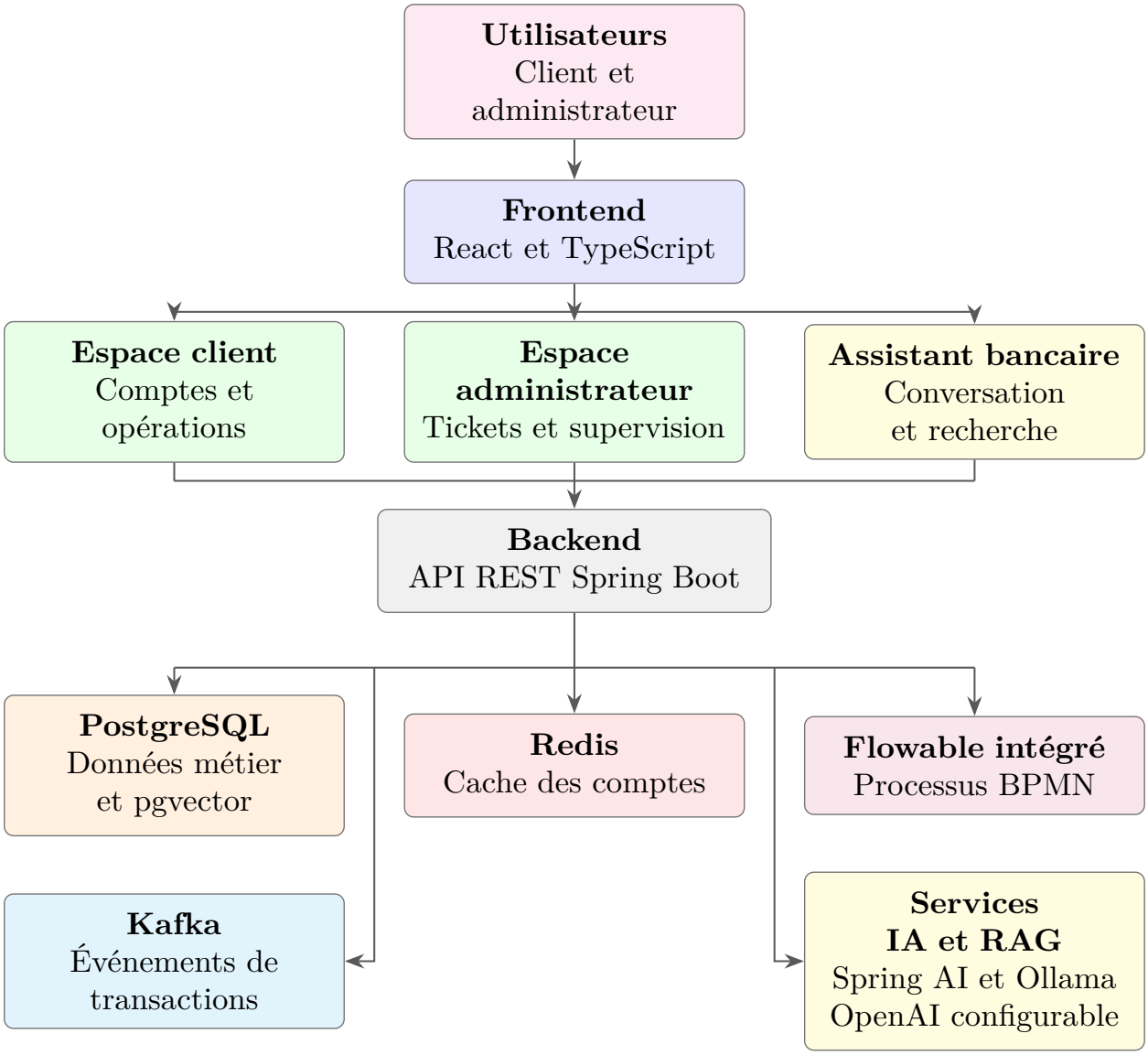


Fig 2.6 : Architecture globale de la plateforme FinBank

Flowable s'exécute au sein du backend : il ne constitue pas une application séparée. La sécurité repose sur Spring Security et des jetons JWT. Les échanges liés aux transactions peuvent également s'appuyer sur Kafka, tandis que Redis est utilisé pour le cache des comptes. Enfin, les fonctions d'assistance utilisent les composants IA configurés et le module RAG s'appuie sur PostgreSQL et pgvector pour la recherche documentaire.

La figure 2.7 illustre le principe de séparation entre le modèle, la vue et le contrôleur.

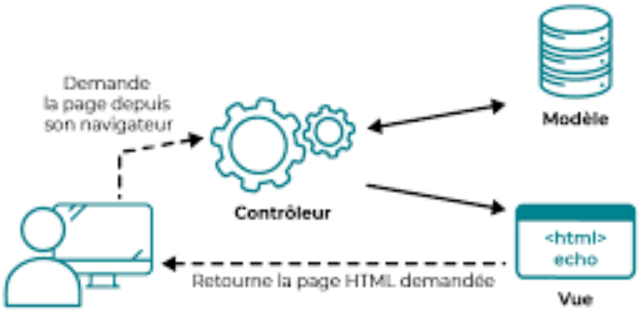


Fig 2.7 : Architecture logicielle : principe modèle-vue-contrôleur

2.7 Environnement de travail

Cette section présente l'environnement matériel utilisé pendant le développement ainsi que les principaux outils logiciels de la plateforme FinBank.

2.7.1 Environnement matériel

Pendant le processus de développement, un ordinateur portable **Lenovo IdeaPad 3** a été utilisé avec la configuration suivante :

- **Système d'exploitation :** Microsoft Windows 11 Famille Mono Langue (version 10.0.22631).
- **Fabricant du système :** Lenovo.
- **Modèle du système :** 82K1.
- **Processeur :** AMD Ryzen série 7000.
- **Graphiques :** circuit graphique intégré AMD Radeon et carte graphique NVIDIA.
- **Mémoire vive :** 16 Go de RAM.
- **Stockage :** SSD de 1 To.

2.7.2 Environnement logiciel

Les principaux outils et technologies utilisés pour le développement de *FinBank* sont présentés ci-dessous. Pour chaque technologie, son logo, son rôle et sa référence officielle sont indiqués.

Java 21 [8]

Langage de programmation orienté objet utilisé pour développer le backend de l'application.



Spring Boot 3.5.14 [9]

Framework Java utilisé pour développer le backend et exposer les API REST de manière structurée.



Spring Security [10]

Framework de sécurité utilisé pour contrôler l'accès aux ressources protégées et gérer l'authentification.

JWT (JSON Web Token) [11]



Format de jeton standardisé utilisé pour l'authentification stateless et la transmission sécurisée des informations.



Maven [12]

Outil de construction utilisé pour compiler le backend Java et gérer ses dépendances.



Swagger UI / OpenAPI [13]

Outils utilisés pour documenter et explorer les API REST de manière interactive.



React 19 [14]

Bibliothèque JavaScript utilisée pour construire les interfaces utilisateur du frontend de manière réactive.



TypeScript 5 [15]

Sur-ensemble de JavaScript ajoutant un typage statique, utilisé pour développer le frontend de façon plus robuste.



Material UI 5 [16]

Bibliothèque de composants d'interface React utilisée pour accélérer le développement frontend.

Node.js et npm [17]



Environnement d'exécution JavaScript et gestionnaire de paquets utilisés pour le frontend.



Vite [18]

Outil de build moderne utilisé pour exécuter le frontend en développement et produire sa version optimisée.



PostgreSQL 16 [19]

Système de gestion de base de données relationnelle utilisé pour stocker les données de l'application.



pgvector [20]

Extension PostgreSQL utilisée pour la recherche sémantique par similarité vectorielle dans le mécanisme RAG.



Redis 7 [21]

Base de données en mémoire utilisée comme cache pour améliorer les performances des consultations fréquentes.



Flowable 6.8.0 [22]

Moteur BPMN intégré au backend pour exécuter les processus de validation des opérations sensibles.



Camunda Modeler [23]

Outil graphique utilisé pour concevoir les processus BPMN 2.0 (déployés ensuite dans Flowable).



Ollama [24]

Outil d'exécution locale de modèles de langage, utilisé pour la détection d'intention et la génération d'embeddings.



Spring AI [25]

Bibliothèque utilisée pour intégrer les modèles d'intelligence artificielle au backend Spring Boot.



Python et FastAPI [26, 27]

Technologies utilisées pour développer le service d'agent IA optionnel, qui traite les demandes conversationnelles complexes.



Apache Kafka [28]

Plateforme de messagerie distribuée utilisée pour transmettre les événements liés aux transactions de manière asynchrone.



Docker et Docker Compose [29]

Outils de conteneurisation utilisés pour lancer et orchestrer les services de l'application dans un environnement reproductible.



IntelliJ IDEA [30]

Environnement de développement intégré utilisé pour le backend Java.



Git et GitHub [31, 32]

Outils de gestion de version utilisés pour le suivi et l'hébergement du code source.



Postman [33]

Outil utilisé pour tester et valider les API REST.



Draw.io [34]

Outil de diagramme utilisé pour concevoir les schémas UML et l'architecture du projet.

Conclusion

Ce chapitre a présenté le contexte du projet FinBank, ses acteurs, ses besoins fonctionnels et non fonctionnels, ainsi que le Product Backlog et la planification des sprints. Il a également introduit les diagrammes de conception, l'architecture globale de l'application et l'environnement matériel et logiciel utilisé.

Le chapitre suivant sera consacré à la conception détaillée de la solution. Il présentera notamment les processus BPMN, les principaux parcours métier et la réalisation des composants de l'application.

3

Sprint 1 : Mise en place du backend et sécurisation des accès

3.1 Introduction

Dans ce chapitre, nous présentons l'étude et la réalisation du **premier sprint** de la plateforme bancaire intelligente FinBank. Ce sprint représente une étape fondamentale dans le développement du projet, car il permet de mettre en place les **bases fonctionnelles et sécuritaires** nécessaires aux modules suivants.

Durant ce sprint, le travail s'est concentré sur la mise en place du **backend Spring Boot**, la création des **entités persistantes**, la configuration de **PostgreSQL et JPA**, ainsi que l'implémentation de **Spring Security** avec l'**authentification JWT**. Il s'agit principalement de la gestion des utilisateurs, des rôles, du profil et de la sécurité des accès à la plateforme.

Ce chapitre est organisé en trois parties principales : la première présente le **backlog du sprint** ; la deuxième détaille la **conception** à travers les diagrammes UML ; la troisième présente la **réalisation** des interfaces développées.

3.2 Backlog du Sprint

Le tableau 3.1 présente les tâches planifiées durant le Sprint 1, avec une estimation de leur durée de réalisation.

Tab 3.1 : Backlog du Sprint 1

ID	Tâche	Estimation
S1-T1	Mise en place de l'API Spring Boot, contexte `/api/v1` et configuration PostgreSQL / JPA	2 jours
S1-T2	Développement de l'authentification JWT : inscription, connexion, renouvellement de jeton et déconnexion	4 jours
S1-T3	Gestion des utilisateurs et des rôles applicatifs (`USER`, `ADMIN`)	3 jours
S1-T4	Configuration de Spring Security et protection des ressources selon le rôle et l'état du compte	3 jours
S1-T5	Gestion du profil utilisateur et changement de mot de passe	2 jours
S1-T6	Activation et vérification de la double authentification (2FA / TOTP)	2 jours
S1-T7	Documentation des API REST avec Swagger / OpenAPI 3	1 jour
S1-T8	Tests unitaires et d'intégration du backend	2 jours
	Total	**19 jours**

3.3 Conception

La phase de conception du Sprint 1 a pour objectif de modéliser les fonctionnalités d'authentification et de gestion du profil. Elle repose sur plusieurs diagrammes UML : cas d'utilisation, de

classes et de séquence.

3.3.1 Diagramme de cas d'utilisation

Le diagramme de la figure 3.1 représente les interactions entre les acteurs et le système pour le Sprint 1.

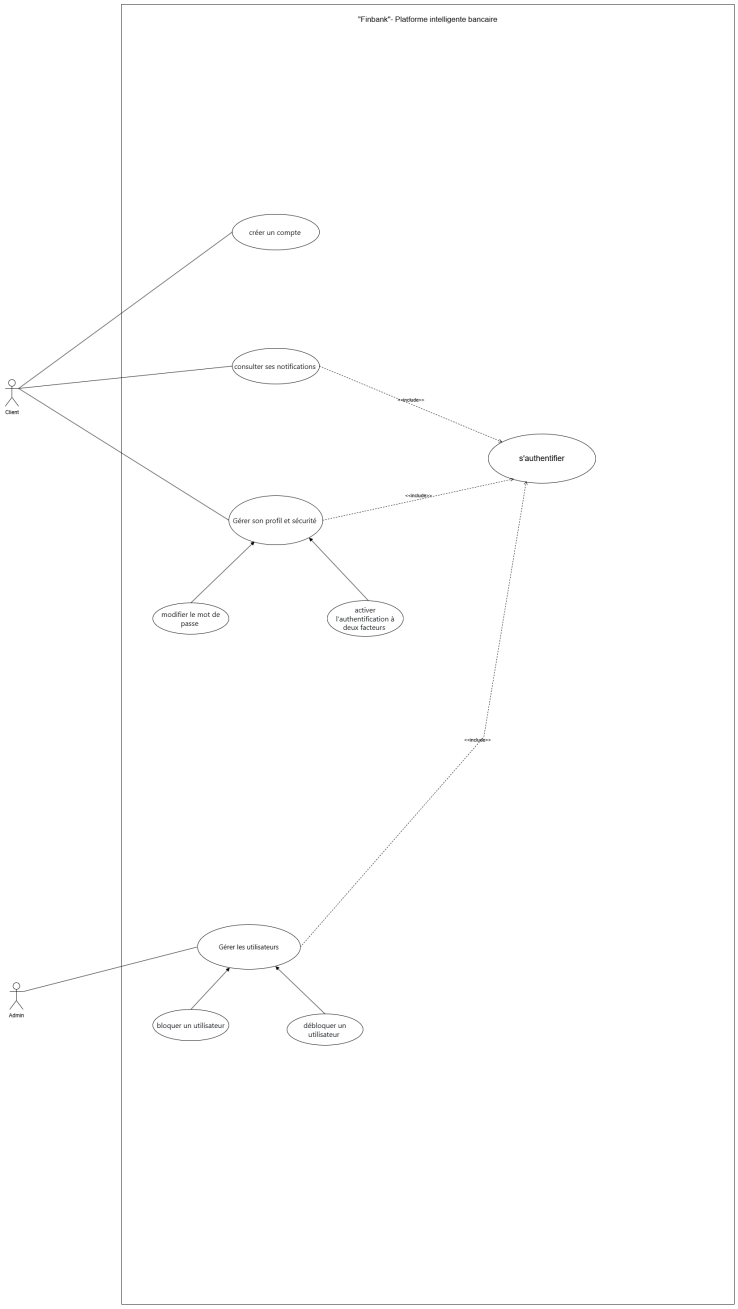


Fig 3.1 : Diagramme de cas d'utilisation du Sprint 1

Ce diagramme met en évidence deux acteurs principaux. Le **Client** (`ROLE_USER`) peut créer un

compte, s'authentifier, activer la double authentification et gérer son profil. L'**Administrateur** (`ROLE_ADMIN`) dispose en plus des fonctions de gestion des comptes utilisateurs. L'authentification constitue une précondition à tous les cas protégés.

3.3.2 Diagramme de classes

Le diagramme de la figure 3.2 présente les entités nécessaires à l'authentification, à la gestion des rôles et à la gestion du profil utilisateur.

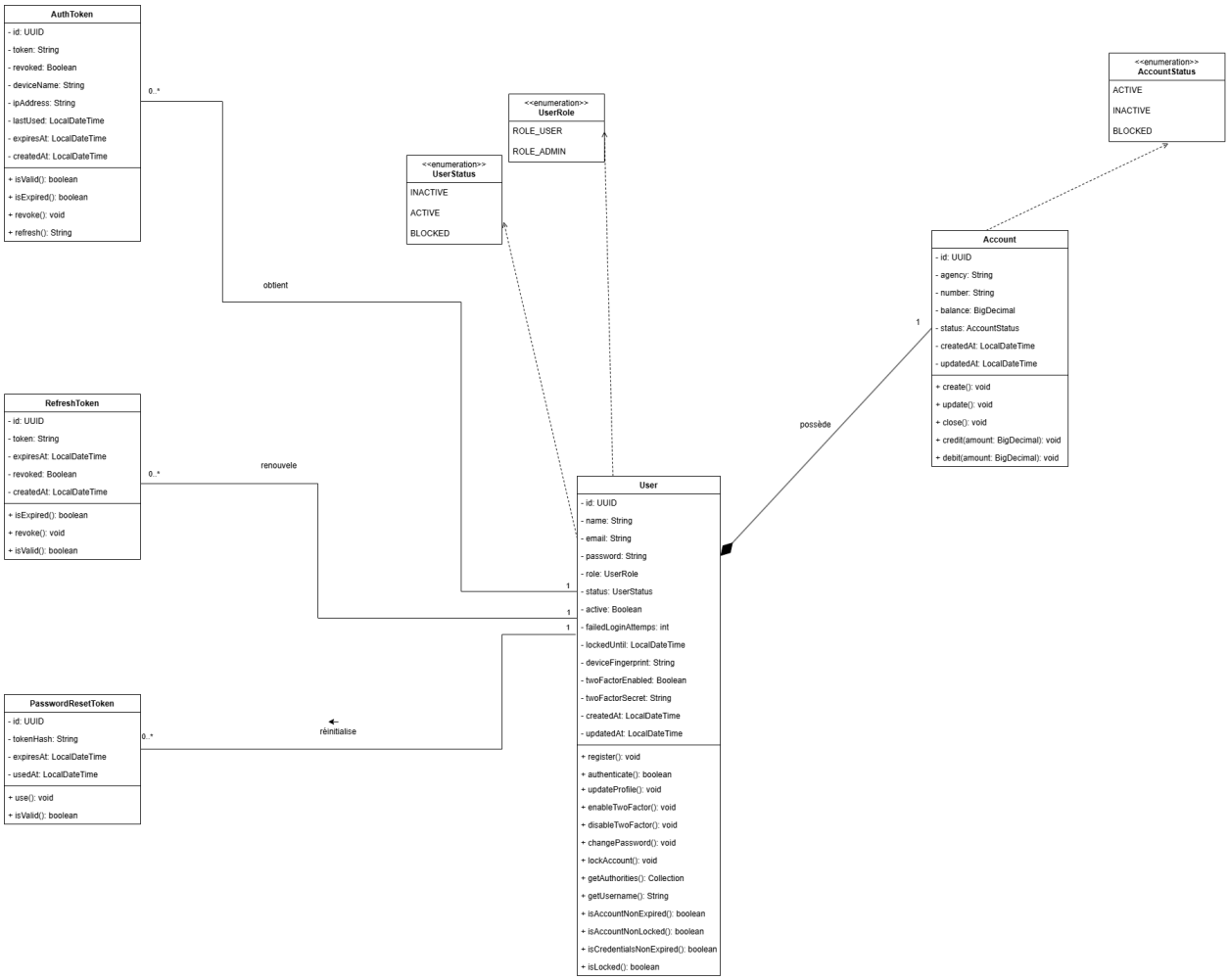


Fig 3.2 : Diagramme de classes du Sprint 1

La classe `User` occupe une place centrale : elle regroupe les informations d'identité, le rôle et les paramètres de sécurité du compte. La classe `RefreshToken` permet de renouveler une session sans ressaisir les identifiants. La classe `AuthToken` conserve les jetons d'authentification persistés. Enfin, la classe `SecurityEvent` trace les événements suspects tels que les tentatives de réutilisation de jeton ou les attaques par force brute.

3.3.3 Diagramme de séquence : authentification sans 2FA

La figure 3.3 illustre le scénario d'authentification lorsque la double authentification n'est pas activée sur le compte de l'utilisateur.

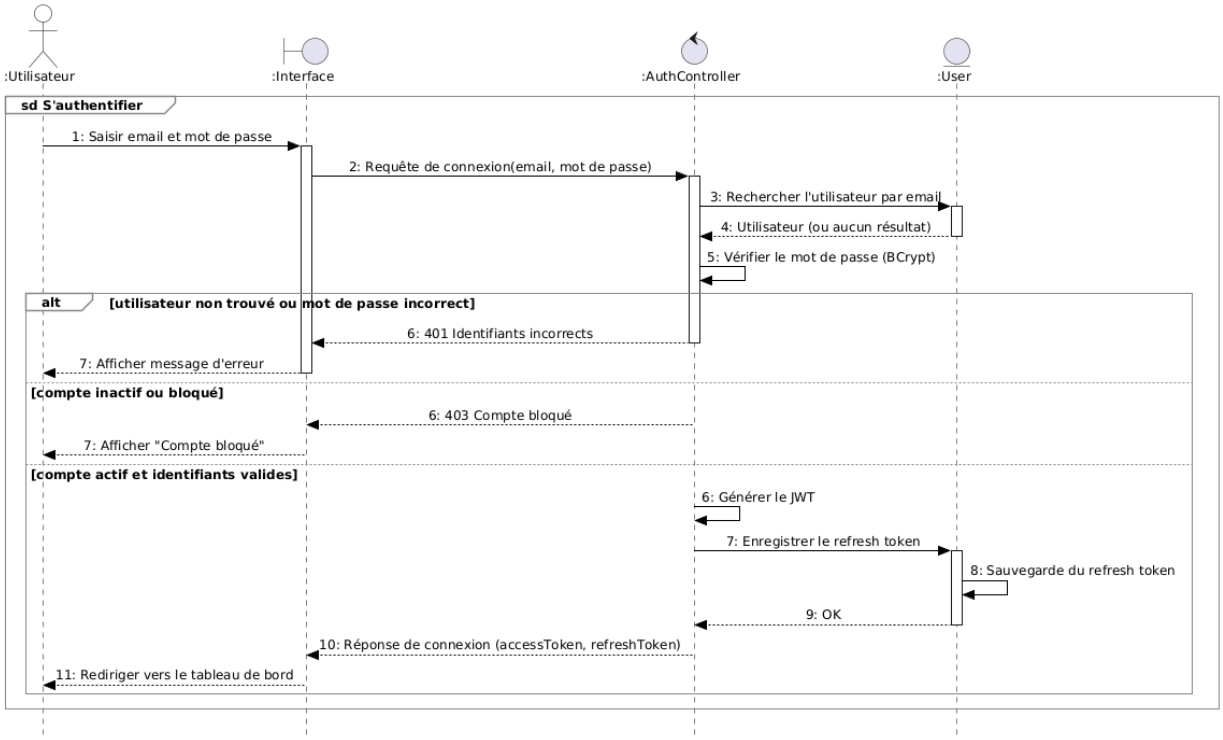


Fig 3.3 : Diagramme de séquence – Authentification sans 2FA

Le **Client** soumet ses identifiants via `POST /auth/login`. Le `AuthController` transmet la requête au `AuthService`, qui interroge l'`UserRepository` pour retrouver le compte, puis vérifie le mot de passe à l'aide de `BCrypt`. En cas de succès, un jeton d'accès valide 15 minutes et un jeton de renouvellement valide 7 jours sont générés et retournés au client. En cas d'échec, une erreur `401 Unauthorized` est renvoyée. Après cinq tentatives infructueuses, le compte est bloqué et un `SecurityEvent` est enregistré.

3.3.4 Diagramme de séquence : authentification avec 2FA

La figure 3.4 illustre le scénario d'authentification en deux phases lorsque la double authentification est activée.

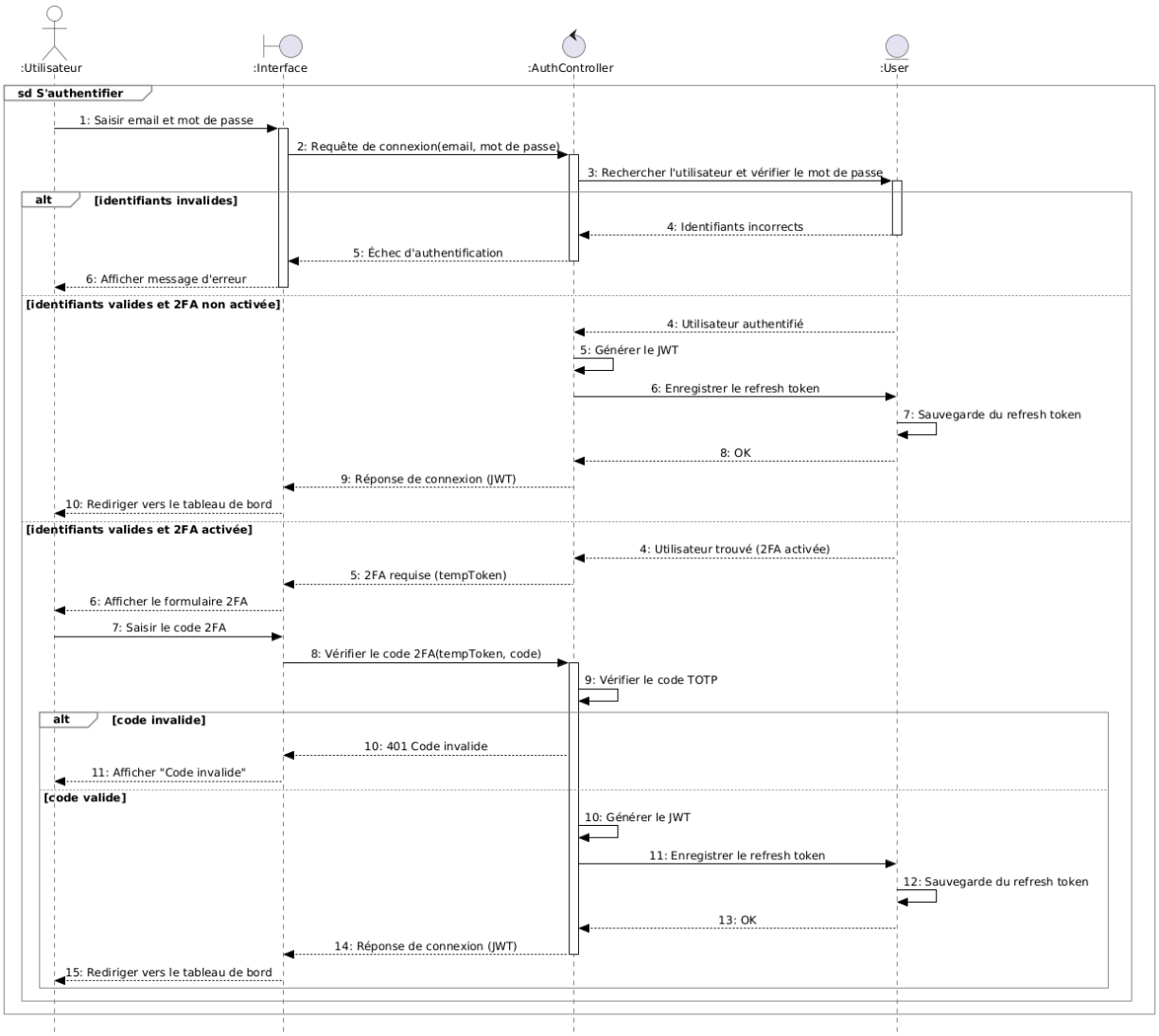


Fig 3.4 : Diagramme de séquence – Authentification avec 2FA

L'authentification se déroule en deux phases. Durant la **Phase 1**, le système valide les identifiants et détecte que la 2FA est activée : un jeton temporaire est alors émis et l'utilisateur est invité à saisir son code TOTP. Durant la **Phase 2**, le `TotpService` vérifie le code à six chiffres généré par l'application d'authentification. Si le code est valide, le jeton JWT définitif est généré et le jeton de renouvellement est persisté. En cas d'échec, l'accès est refusé et l'erreur est journalisée.

3.3.5 Description textuelle des cas d'utilisation

Cette section présente la description textuelle structurée des deux cas d'utilisation principaux du sprint.

3.3.5.1 S'authentifier

Tab 3.2 : Description textuelle – S'authentifier

Élément	**Description**
Titre	S'authentifier
Acteur	Client, Administrateur
But	Permettre à l'utilisateur d'accéder à son espace personnel selon son rôle.
Résumé	L'utilisateur saisit son adresse e-mail et son mot de passe. Le système vérifie les identifiants, identifie le rôle, puis génère un jeton JWT.
Précondition	L'utilisateur possède un compte valide dans le système.
Postcondition	L'utilisateur est authentifié et reçoit un jeton d'accès et un jeton de renouvellement.
Scénario principal	1. L'utilisateur ouvre la page de connexion. 2. Il saisit son e-mail et son mot de passe. 3. Le système vérifie les identifiants. 4. Le système identifie le rôle de l'utilisateur. 5. Le système génère un jeton JWT. 6. L'utilisateur est redirigé vers son tableau de bord.
Scénario alternatif	Si les identifiants sont incorrects, le système affiche un message d'erreur. Après cinq tentatives, le compte est temporairement bloqué.

3.3.5.2 Gérer son profil

Tab 3.3 : Description textuelle – Gérer son profil

Élément	**Description**
Titre	Gérer son profil
Acteur	Client, Administrateur
But	Permettre à l'utilisateur de consulter et modifier ses informations personnelles.
Résumé	L'utilisateur authentifié peut consulter son profil, modifier ses informations et changer son mot de passe.
Précondition	L'utilisateur doit être authentifié.
Postcondition	Les informations du profil sont mises à jour dans la base de données.
Scénario principal	1. L'utilisateur accède à la page de son profil. 2. Le système affiche ses informations personnelles. 3. L'utilisateur modifie les champs souhaités. 4. Le système valide les données saisies. 5. Le système enregistre les modifications. 6. Un message de confirmation est affiché.
Scénario alternatif	Si un champ obligatoire est vide ou contient un format invalide, le système affiche un message d'erreur précis.

3.4 Réalisation

Cette section présente les interfaces effectivement développées durant le Sprint 1. Chaque interface est accompagnée d'une description détaillant son rôle et les fonctionnalités qu'elle expose à l'utilisateur.

3.4.1 Interface d'inscription

La figure 3.5 présente le formulaire d'inscription permettant à un nouvel utilisateur de créer un compte sur la plateforme FinBank.

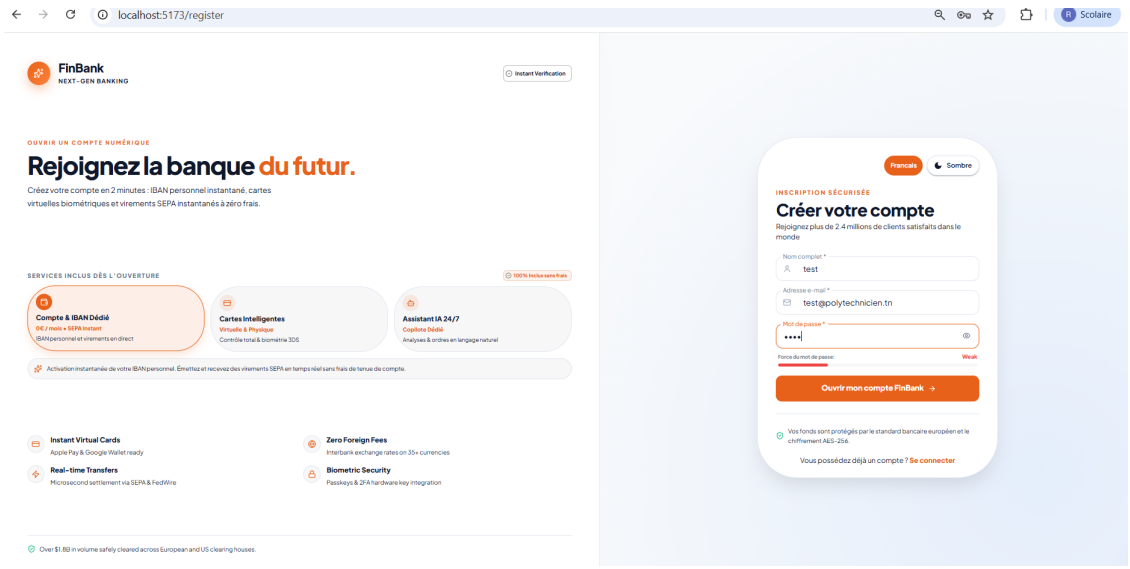


Fig 3.5 : Interface d'inscription

Ce formulaire collecte les informations nécessaires à la création du compte : nom, prénom, adresse e-mail et mot de passe. Une validation est effectuée côté client et côté serveur avant l'enregistrement. En cas de succès, un message de confirmation est affiché et l'utilisateur est invité à se connecter. Si l'adresse e-mail est déjà utilisée, un message d'erreur explicite est retourné.

3.4.2 Interface de connexion

La figure 3.6 présente la page de connexion, point d'entrée commun à tous les utilisateurs de la plateforme.

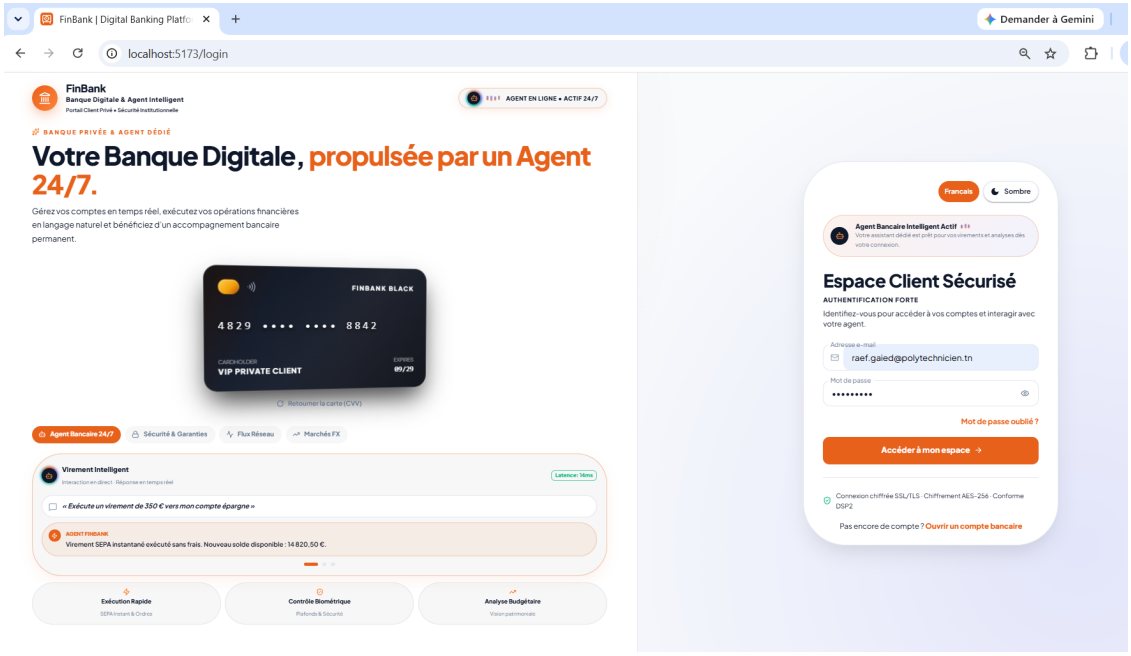


Fig 3.6 : Interface de connexion

L'utilisateur saisit son adresse e-mail et son mot de passe. Le système vérifie les identifiants et, en cas de succès, génère un jeton JWT stocké côté client. L'utilisateur est alors redirigé vers son tableau de bord en fonction de son rôle (`USER` ou `ADMIN`). Si la double authentification est activée, l'écran de vérification 2FA est affiché avant la redirection finale.

3.4.3 Interface de vérification 2FA

La figure 3.7 présente l'écran de saisie du code de double authentification, affiché après validation du mot de passe lorsque la 2FA est activée.

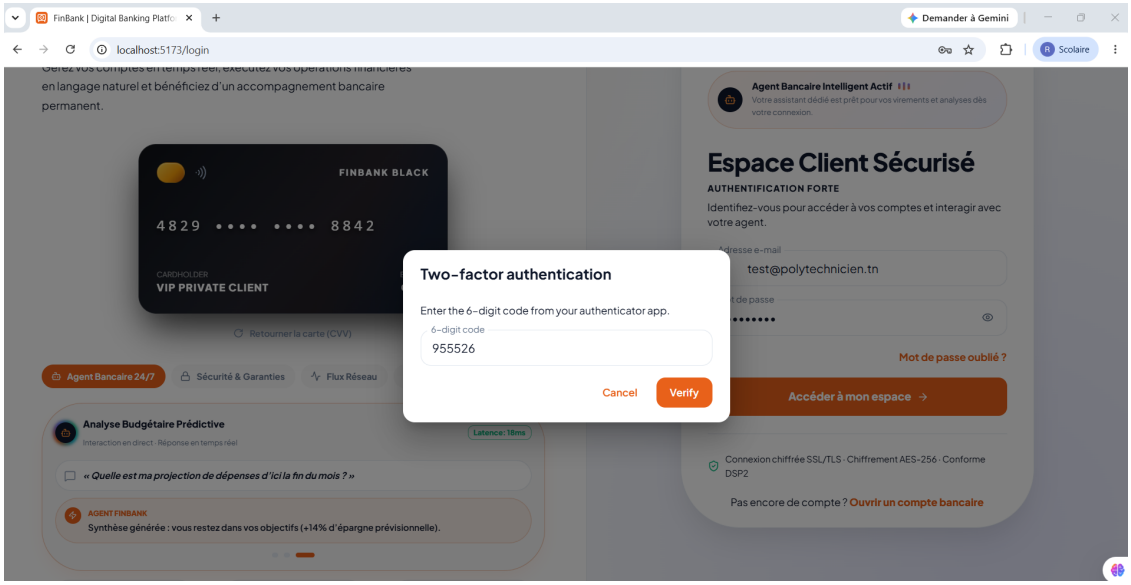


Fig 3.7 : Interface de vérification 2FA

Après validation du mot de passe, un jeton temporaire est émis et l'utilisateur doit saisir le code à six chiffres généré par son application d'authentification (Google Authenticator ou équivalent). Le `TotpService` vérifie ce code en temps réel. Le jeton JWT définitif n'est délivré qu'après vérification réussie. En cas d'échec, un message d'erreur est affiché et la tentative est journalisée.

3.4.4 Interface d'activation de la 2FA

La figure 3.8 présente l'interface permettant à l'utilisateur d'activer la double authentification depuis son profil.

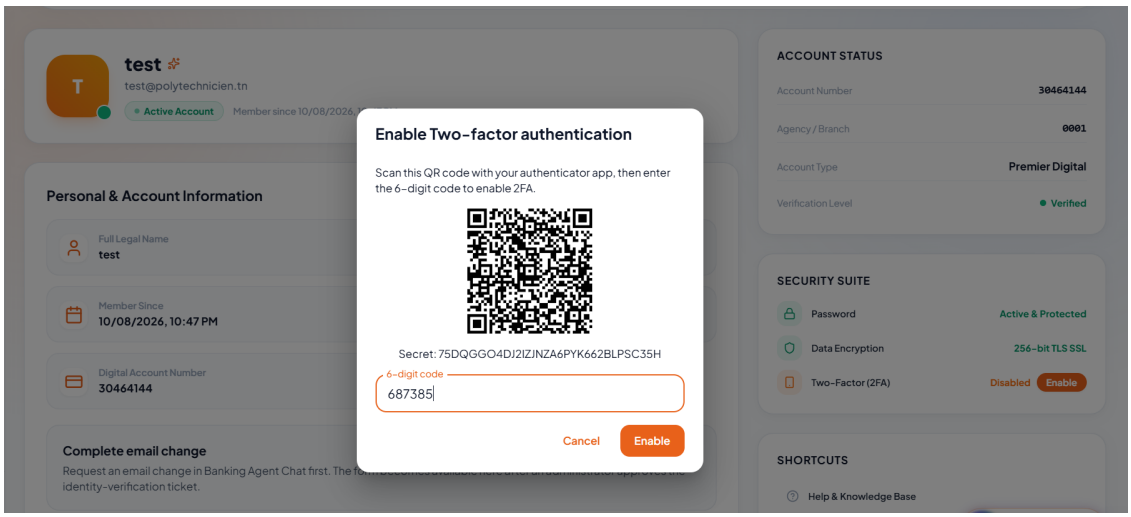


Fig 3.8 : Interface d'activation de la 2FA

Le système génère un QR code que l'utilisateur scanne avec son application d'authentification. Une fois le scan effectué, l'utilisateur saisit un code de validation pour confirmer l'activation. Après vérification, la 2FA est activée sur le compte et un ensemble de codes de secours est fourni pour la récupération du compte en cas de perte de l'appareil.



Fig 3.9 : Confirmation de l'activation de la double authentification

3.4.5 Interface de gestion du profil

La figure 3.10 présente la page de profil permettant à l'utilisateur de consulter et modifier ses informations personnelles.

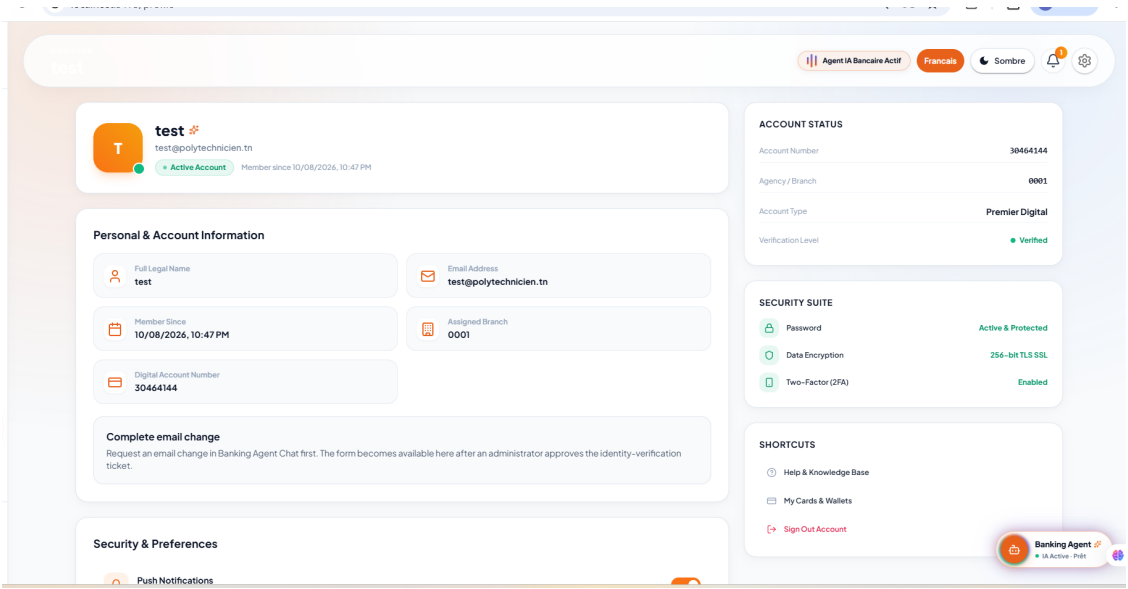


Fig 3.10 : Interface de gestion du profil

Cette interface affiche les informations du compte (nom, prénom, e-mail, statut, date d'adhésion) et les indicateurs de sécurité (mot de passe protégé, chiffrement TLS, état de la 2FA). L'utilisateur

peut modifier ses informations personnelles ou accéder à d'autres sections via la navigation. Les champs modifiables sont clairement distingués des champs en lecture seule.

3.4.6 Interface de changement de mot de passe

La figure 3.11 présente la section dédiée au changement de mot de passe, accessible depuis le profil de l'utilisateur.

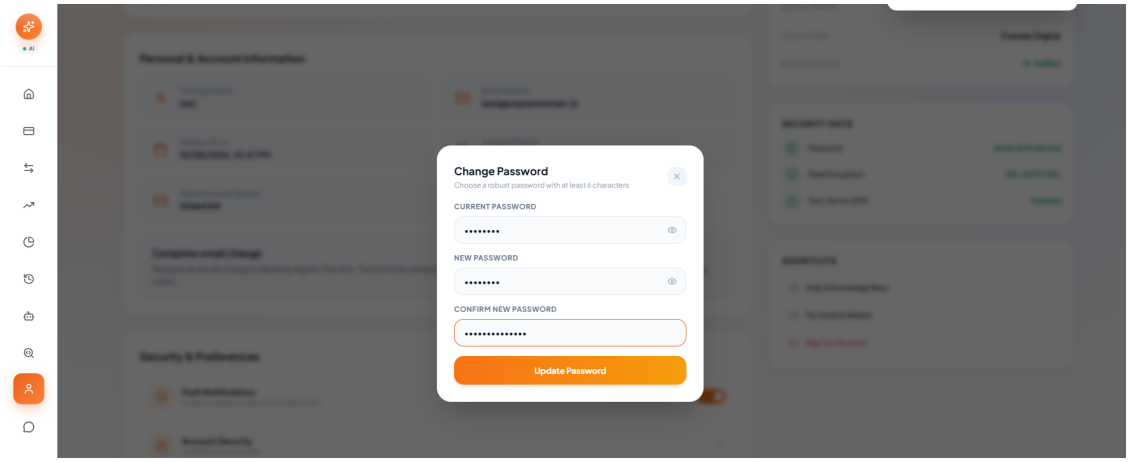


Fig 3.11 : Interface de changement de mot de passe

L'utilisateur saisit son ancien mot de passe, puis le nouveau à deux reprises pour confirmation. Le système vérifie que l'ancien mot de passe est correct avant d'enregistrer le nouveau, haché avec `BCrypt` au coût 12. Après modification, tous les jetons d'accès actifs associés au compte sont révoqués, forçant une nouvelle authentification sur les autres sessions ouvertes.

3.4.7 Interface d'administration des utilisateurs

La figure 3.12 présente l'interface d'administration réservée aux utilisateurs disposant du rôle `ROLE_ADMIN`.

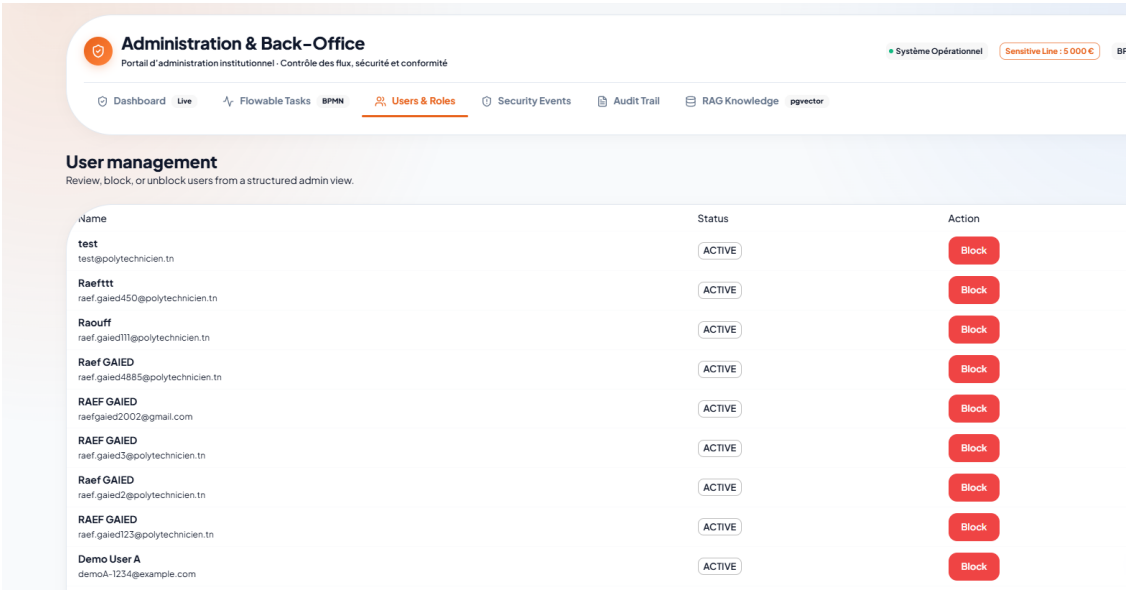


Fig 3.12 : Interface d'administration des utilisateurs

Cette interface permet à l'administrateur de consulter la liste complète des utilisateurs, de rechercher un compte par nom ou e-mail, et de bloquer ou débloquer un compte. Chaque action sensible est journalisée dans la table `audit_logs` pour assurer la traçabilité des opérations. L'accès à cette interface est strictement contrôlé par Spring Security via l'annotation `@PreAuthorize("hasRole('ADMIN')")`.

3.5 Conclusion

Ce chapitre a présenté l'étude et la réalisation du **Sprint 1** de la plateforme FinBank. Nous avons commencé par définir le **backlog du sprint**, puis nous avons modélisé les fonctionnalités à travers les diagrammes de cas d'utilisation, de classes et de séquence. La section de réalisation a présenté les sept interfaces développées, chacune accompagnée d'une description détaillée de son rôle.

Ce sprint constitue les **fondations sécuritaires** de la plateforme : authentification JWT, gestion des rôles, protection des ressources, gestion du profil et double authentification. Ces mécanismes sont transversaux à tous les sprints suivants.

Le chapitre suivant sera consacré au **Sprint 2**, qui portera sur les fonctionnalités bancaires opérationnelles : consultation des comptes, virements, historique des transactions, gestion des cartes et des investissements, ainsi que l'intégration de l'agent IA.

4

Sprint 2 : Développement des fonctionnalités bancaires

4.1 Introduction

Dans ce chapitre, nous présentons l'étude et la réalisation du **deuxième sprint** de la plateforme bancaire intelligente FinBank. Après la mise en place des fondations sécuritaires lors du Sprint 1, ce sprint est consacré aux **fonctionnalités bancaires opérationnelles** proposées à l'utilisateur authentifié.

Durant ce sprint, le travail s'est concentré sur la consultation du compte, les dépôts, les retraits et les virements, la consultation de l'historique et le téléchargement des reçus, ainsi que la gestion des cartes, des investissements et des clés de paiement.

Ce chapitre est organisé en trois parties principales : la première présente le **backlog du sprint** ; la deuxième détaille la **conception** à travers les diagrammes UML ; la troisième présente la **réalisation** des interfaces développées.

4.2 Backlog du Sprint

Le tableau 4.1 présente les tâches planifiées durant le Sprint 2, avec une estimation de leur durée de réalisation.

Tab 4.1 : Backlog du Sprint 2

ID	Tâche	Estimation
S2-T1	Consultation du compte et affichage du solde disponible	2 jours
S2-T2	Implémentation des dépôts et des retraits	3 jours
S2-T3	Implémentation des virements entre comptes avec contrôle du solde	4 jours
S2-T4	Enregistrement des transactions et consultation de l'historique	2 jours
S2-T5	Génération et téléchargement des reçus de transaction	2 jours
S2-T6	Gestion des cartes bancaires	3 jours
S2-T7	Gestion des investissements et de leur remboursement	3 jours
S2-T8	Gestion des clés de paiement et virements par clé	2 jours
S2-T9	Publication et consommation des événements de transaction avec Kafka et déduplication avec Redis	2 jours
S2-T10	Vérification des règles métier et tests du backend	2 jours
	Total	**25 jours**

4.3 Conception

Cette section présente la modélisation des fonctionnalités bancaires à travers les diagrammes de cas d'utilisation, de classes et de séquence, ainsi que la description textuelle des principaux cas d'utilisation.

4.3.1 Diagramme de cas d'utilisation

Le diagramme de la figure 4.1 représente les fonctionnalités bancaires accessibles à l'utilisateur.

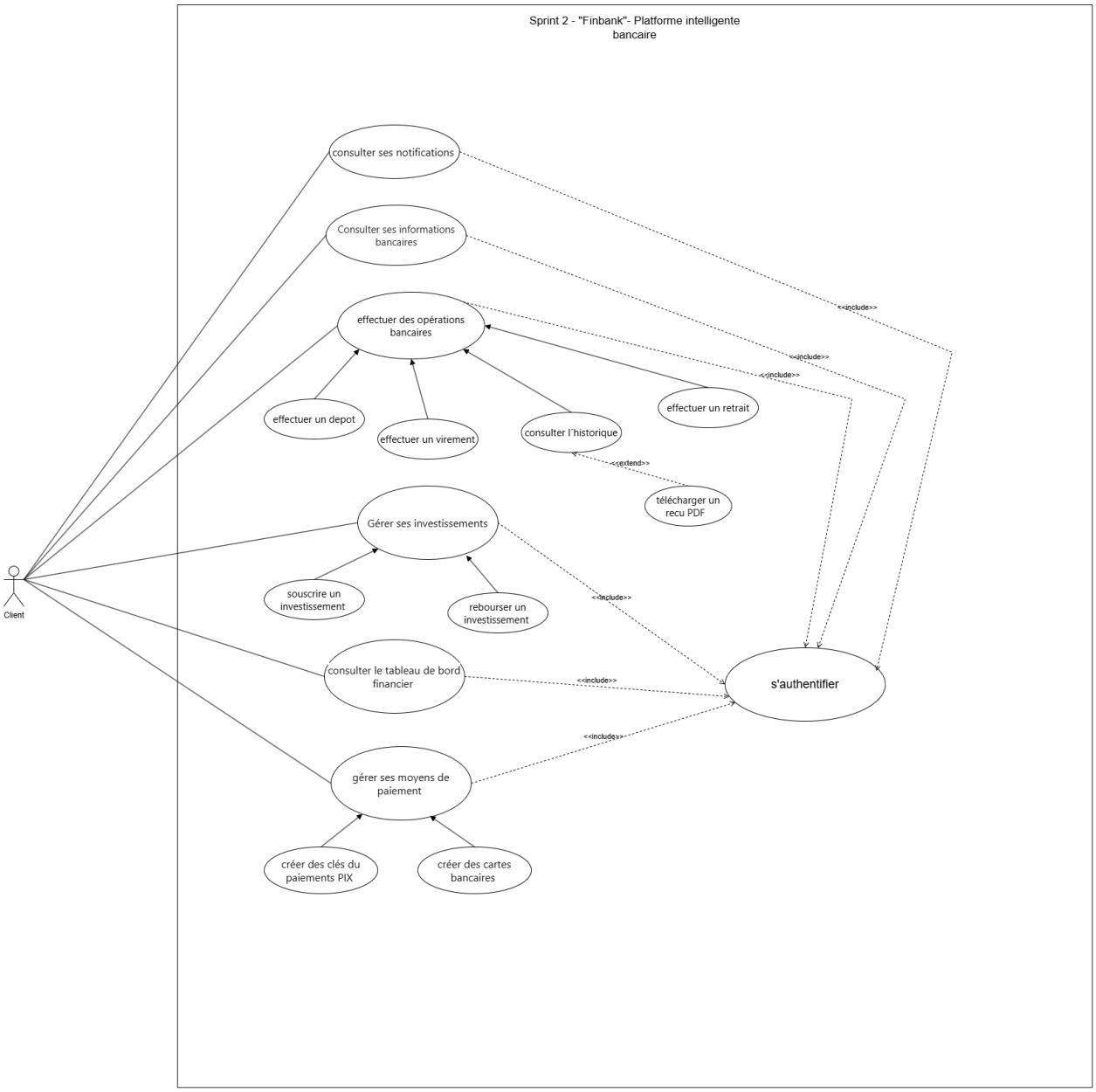


Fig 4.1 : Diagramme de cas d'utilisation du Sprint 2

L'acteur principal est l'**Utilisateur** (rôle `USER`), authentifié lors du Sprint 1. Il peut consulter ses informations bancaires et le tableau de bord financier, effectuer un dépôt, un retrait ou un virement, consulter son historique et télécharger un reçu, gérer ses cartes et ses investissements, et enregistrer ses clés de paiement. Tous ces cas d'utilisation exigent une authentification préalable ; chaque opération est rattachée à l'utilisateur identifié par son jeton, et non à un identifiant fourni

librement par le client.

4.3.2 Diagramme de classes

Le diagramme de la figure 4.2 présente les entités nécessaires aux opérations bancaires.

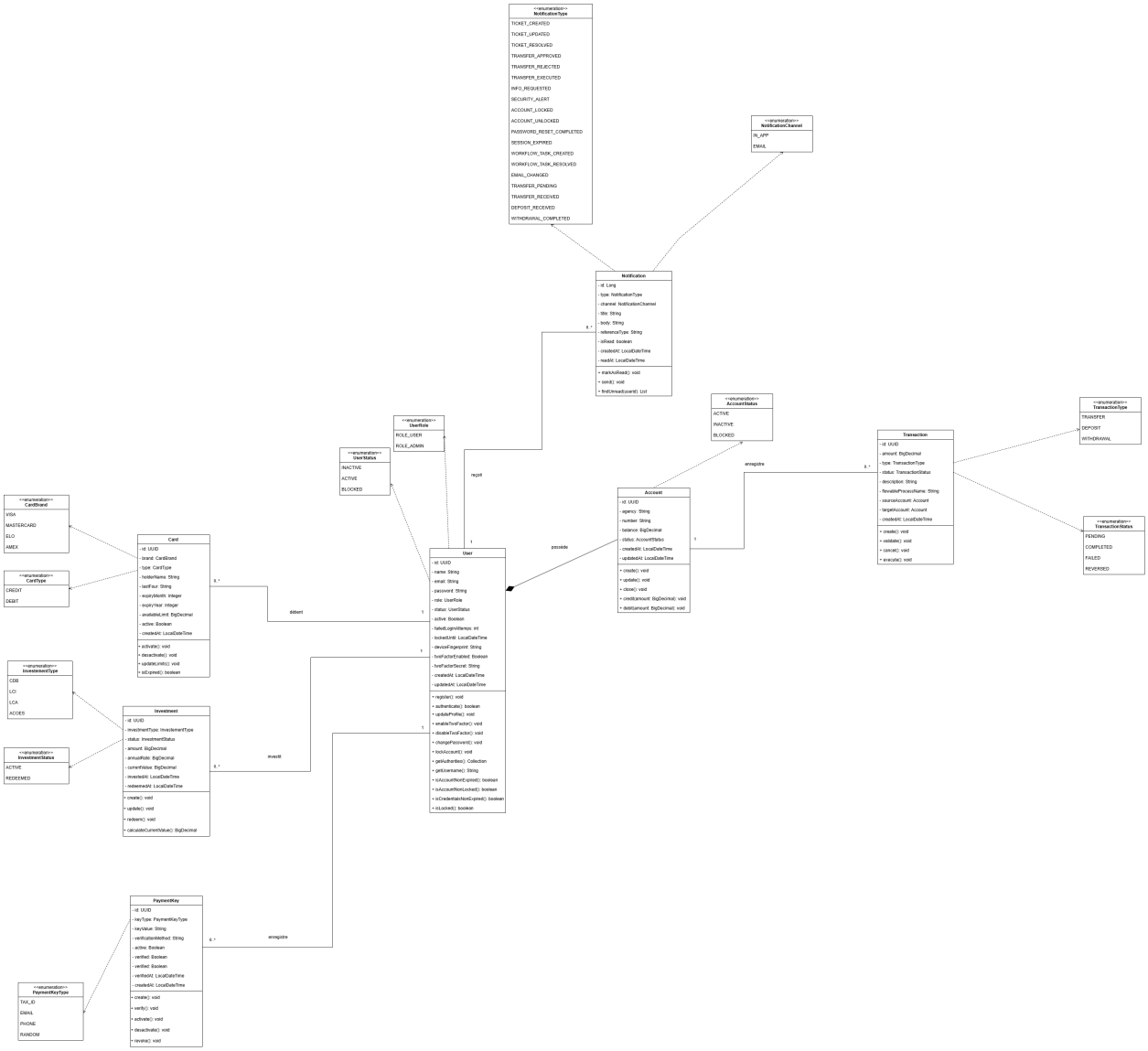


Fig 4.2 : Diagramme de classes du Sprint 2

La classe `Account` représente le compte de l'utilisateur et porte son solde ; chaque compte appartient à un `User`. La classe `Transaction` trace chaque opération avec son type (dépôt, retrait ou virement), son montant et son statut (en attente, complétée, échouée ou annulée), et référence le compte source et le compte cible. La classe `Card` décrit une carte bancaire de crédit ou de débit rattachée à l'utilisateur. La classe `Investment` enregistre un placement (type, montant, statut actif ou

remboursé). Enfin, la classe `PaymentKey` associe une clé de paiement à un compte, ce qui permet d'identifier un destinataire sans connaître son numéro de compte.

4.3.3 Diagramme de séquence : effectuer un virement

La figure 4.3 illustre le traitement d'un virement, de la requête de l'utilisateur jusqu'à l'enregistrement de la transaction.

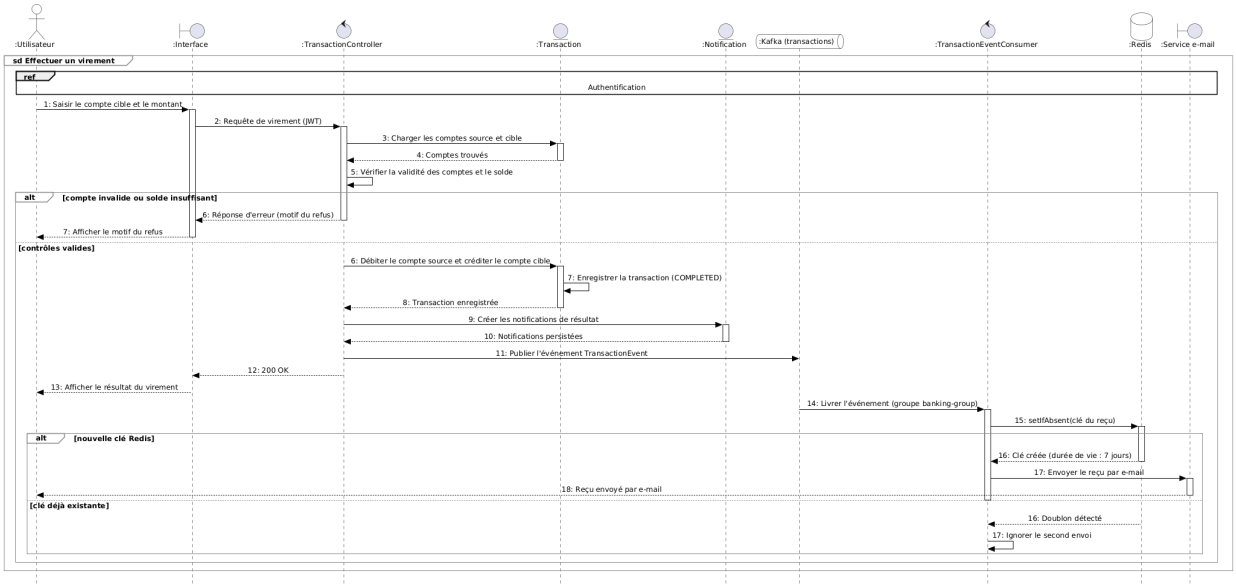


Fig 4.3 : Diagramme de séquence – Effectuer un virement

L'utilisateur saisit le compte destinataire et le montant. Le contrôleur identifie l'utilisateur à partir de son jeton et transmet la demande au service métier. Celui-ci vérifie que le montant est positif, que le compte source appartient bien à l'utilisateur, que le compte cible est valide et que le solde est suffisant. Si les contrôles réussissent, le compte source est débité, le compte cible est crédité et une transaction est enregistrée avec son statut. En cas d'échec (solde insuffisant, compte invalide), la transaction est refusée et un message explicite est retourné à l'utilisateur.

4.3.4 Diagramme de séquence : consulter le compte et le tableau de bord

La figure 4.4 décrit le chargement des informations du compte et des indicateurs du tableau de bord financier.

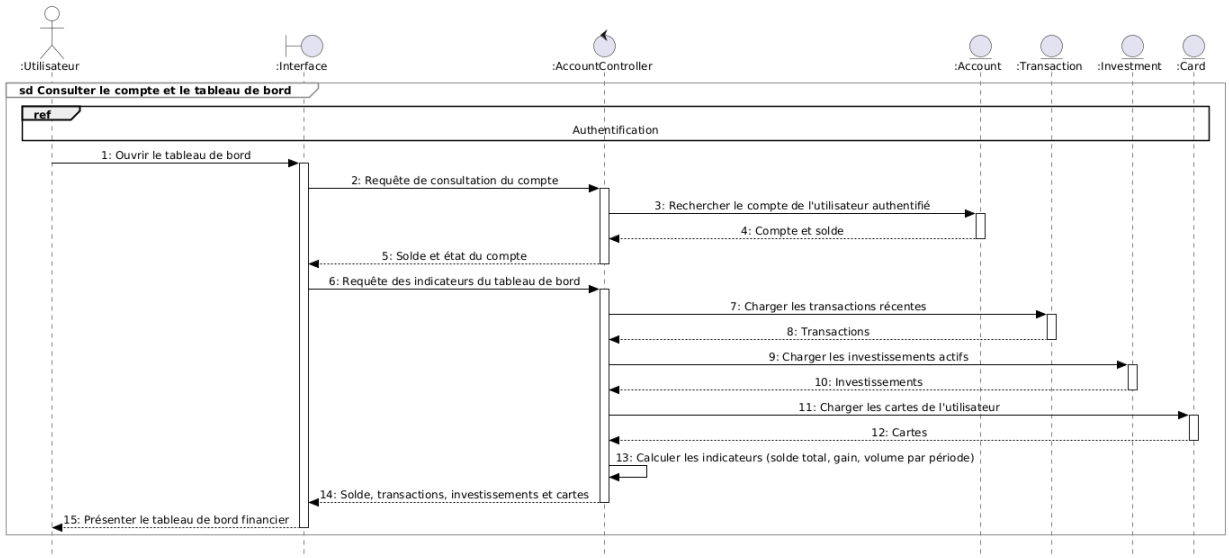


Fig 4.4 : Diagramme de séquence – Consultation du compte et du tableau de bord

L'interface demande d'abord les informations du compte de l'utilisateur authentifié. Le service récupère ensuite les transactions récentes, les investissements actifs et les cartes afin de construire la synthèse financière affichée dans le tableau de bord.

4.3.5 Diagramme de séquence : consulter ses notifications

La figure 4.5 présente la consultation des notifications persistées après les événements bancaires.

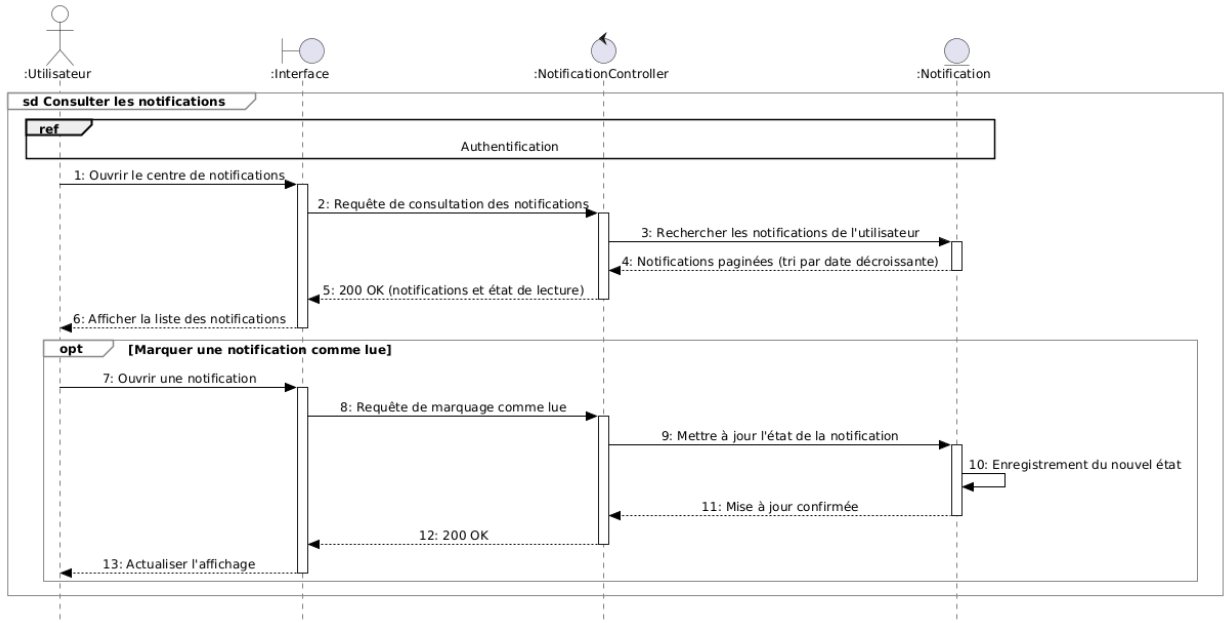


Fig 4.5 : Diagramme de séquence – Consulter ses notifications

L'utilisateur ouvre le centre de notifications et l'interface demande la liste associée à son compte. Le service interroge le dépôt de notifications, puis retourne les éléments paginés avec leur état de lecture. Une notification peut ensuite être marquée comme lue, sans modifier la transaction bancaire à l'origine de l'événement.

4.3.6 Description textuelle des cas d'utilisation

Cette section présente la description textuelle structurée des trois cas d'utilisation principaux du sprint.

4.3.6.1 Consulter ses informations bancaires

Tab 4.2 : Description textuelle – Consulter ses informations bancaires

Élément	**Description**
Titre	Consulter ses informations bancaires
Acteur	Utilisateur
But	Consulter le numéro du compte, son solde et son état.
Résumé	L'utilisateur authentifié ouvre son espace bancaire et le système affiche les informations de son compte.
Précondition	L'utilisateur est authentifié et possède un compte.
Postcondition	Les informations actualisées du compte sont affichées.
Scénario principal	1. L'utilisateur ouvre son espace bancaire. 2. Le système identifie l'utilisateur authentifié. 3. Le système recherche son compte. 4. Le système retourne le solde et les informations du compte.
Scénario alternatif	Si le compte est introuvable ou inactif, le système retourne un message explicite.

4.3.6.2 Consulter le tableau de bord financier

Tab 4.3 : Description textuelle – Consulter le tableau de bord financier

Élément	**Description**
Titre	Consulter le tableau de bord financier
Acteur	Utilisateur
But	Disposer d'une vue de synthèse de sa situation financière.
Résumé	L'utilisateur authentifié accède au tableau de bord, qui regroupe son solde total, ses investissements actifs, ses transactions récentes, ses cartes et l'évolution de ses opérations.
Précondition	L'utilisateur est authentifié.
Postcondition	Les indicateurs sont affichés à partir des données les plus récentes de l'utilisateur.
Scénario principal	1. L'utilisateur ouvre le tableau de bord. 2. Le système identifie l'utilisateur authentifié. 3. Le système récupère son compte, ses transactions, ses cartes et ses investissements. 4. Le système calcule les indicateurs (solde total, gain des investissements, volume des opérations par période). 5. Le système affiche la synthèse et les accès rapides.
Scénario alternatif	Si une source de données est indisponible, la rubrique concernée est vide et le reste du tableau de bord reste affiché.

4.3.6.3 Effectuer un virement

Tab 4.4 : Description textuelle – Effectuer un virement

Élément	**Description**
Titre	Effectuer un virement
Acteur	Utilisateur
But	Transférer un montant vers un compte destinataire.
Résumé	L'utilisateur saisit le destinataire et le montant ; le système contrôle l'opération, met à jour les soldes et enregistre la transaction.
Précondition	L'utilisateur est authentifié, le montant est positif et les comptes sont valides.
Postcondition	La transaction est enregistrée avec son statut et les soldes sont mis à jour lorsque l'opération est validée. Une notification informe l'utilisateur du résultat.
Scénario principal	1. L'utilisateur saisit le compte cible et le montant. 2. Le système vérifie les données et le solde disponible. 3. Le système débite le compte source et crédite le compte cible. 4. Le système enregistre la transaction. 5. Le système retourne le résultat de l'opération.
Scénario alternatif	En cas de solde insuffisant ou de compte invalide, la transaction est refusée et un message d'erreur est affiché.

4.4 Réalisation

Cette section présente les interfaces effectivement développées durant le Sprint 2. Chaque interface est accompagnée d'une description détaillant son rôle et les fonctionnalités qu'elle expose à

l'utilisateur.

4.4.1 Interface du tableau de bord et du compte

Les figures 4.6 et 4.7 présentent l'espace bancaire de l'utilisateur et la synthèse de ses opérations.

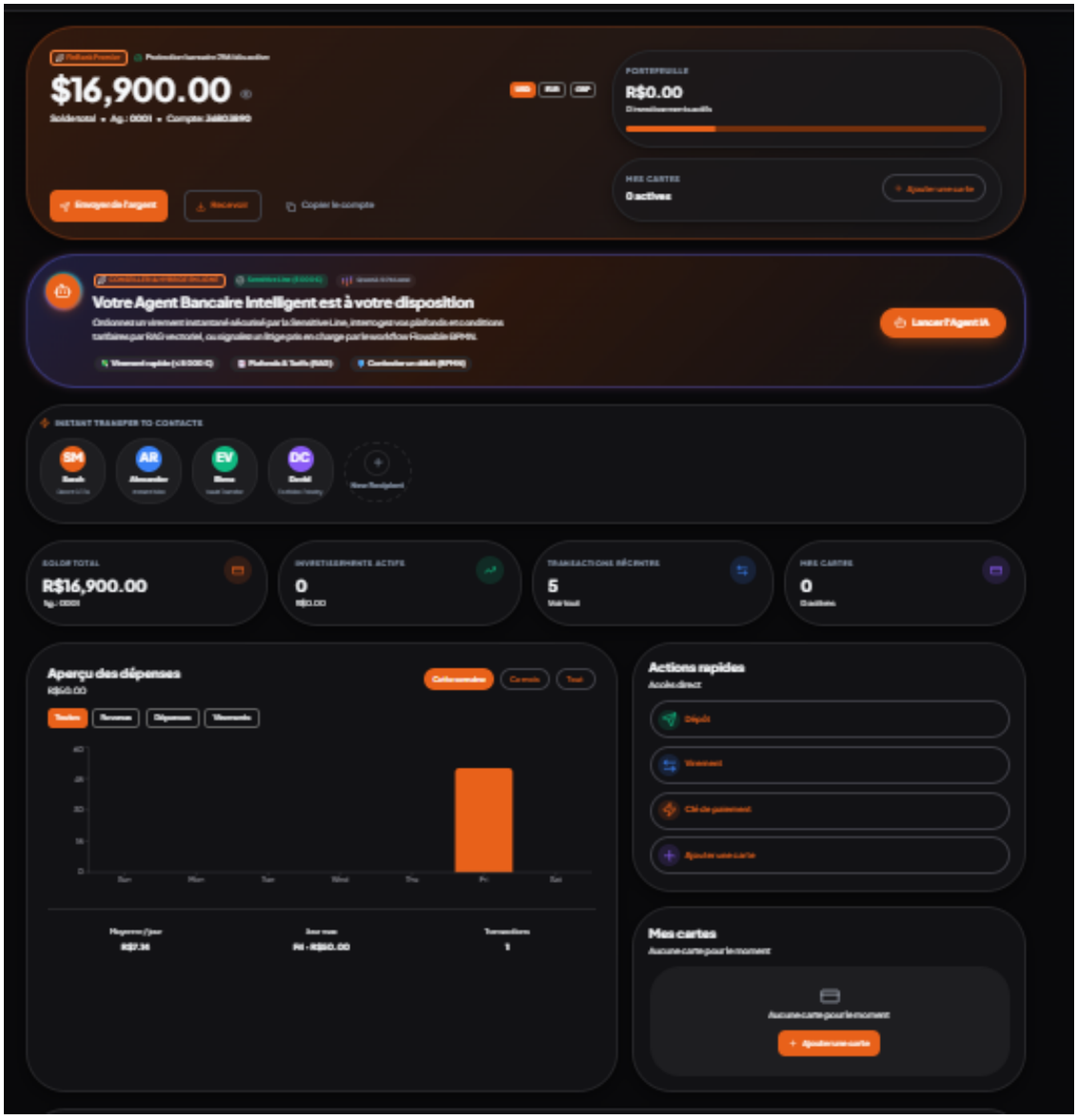


Fig 4.6 : Interface de consultation du compte

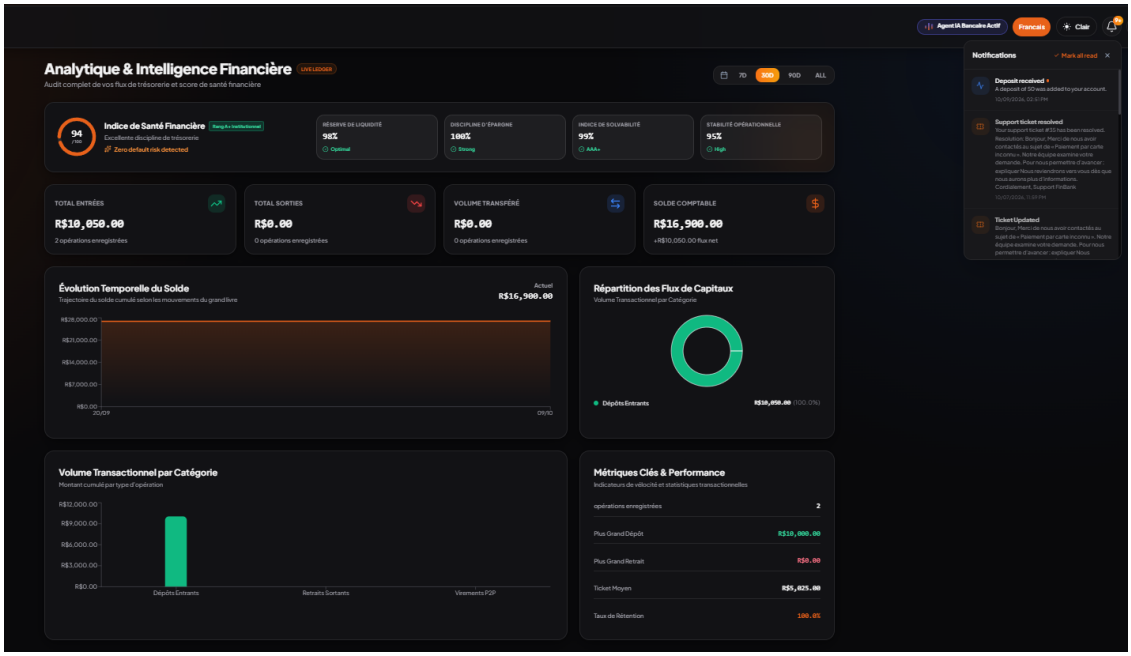


Fig 4.7 : Tableau de bord financier de l'utilisateur

Le tableau de bord financier affiche le solde total, le gain des investissements actifs, les transactions récentes et les cartes, ainsi qu'un graphique de l'évolution des opérations par période (semaine ou mois). Des accès rapides permettent de lancer un dépôt, un retrait ou un virement. Les données affichées sont celles du compte rattaché à l'utilisateur connecté : il ne peut ni consulter ni modifier le compte d'un autre client.

4.4.2 Interface des dépôts et des retraits

Les figures 4.8 et 4.9 présentent les formulaires de dépôt et de retrait.

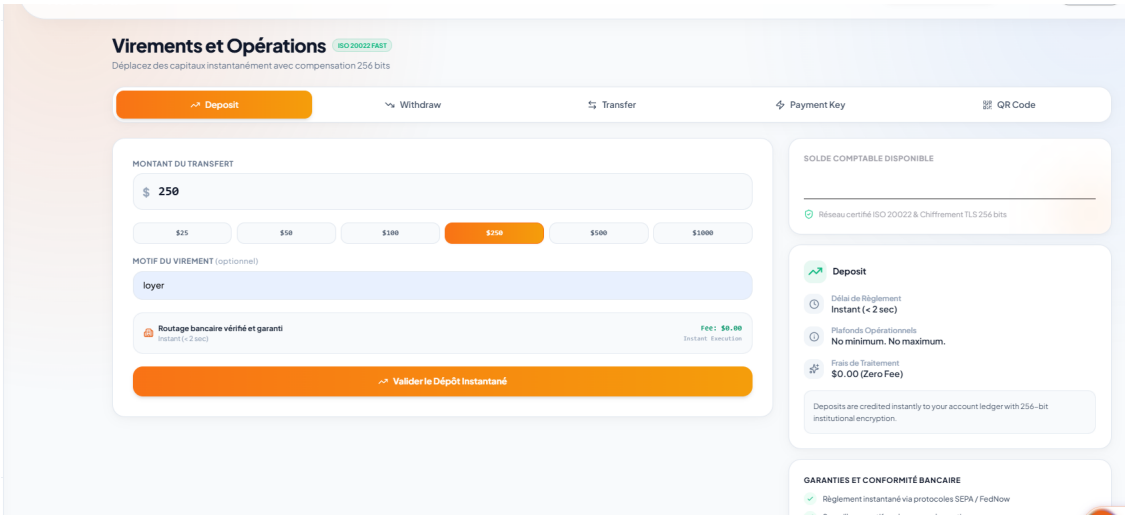


Fig 4.8 : Interface de dépôt

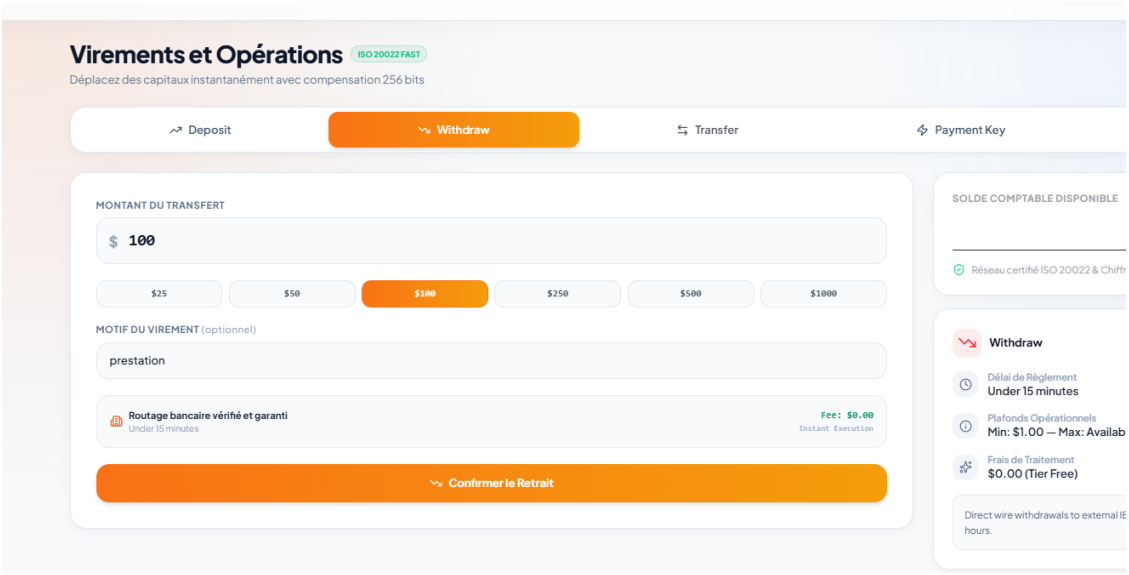


Fig 4.9 : Interface de retrait

L'utilisateur choisit le type d'opération et saisit le montant. Le système vérifie que le montant est strictement positif et, pour un retrait, que le solde est suffisant. Après validation, le solde est mis à jour et l'opération est enregistrée dans l'historique.

4.4.3 Interface des virements

La figure 4.10 présente le formulaire de virement entre comptes.

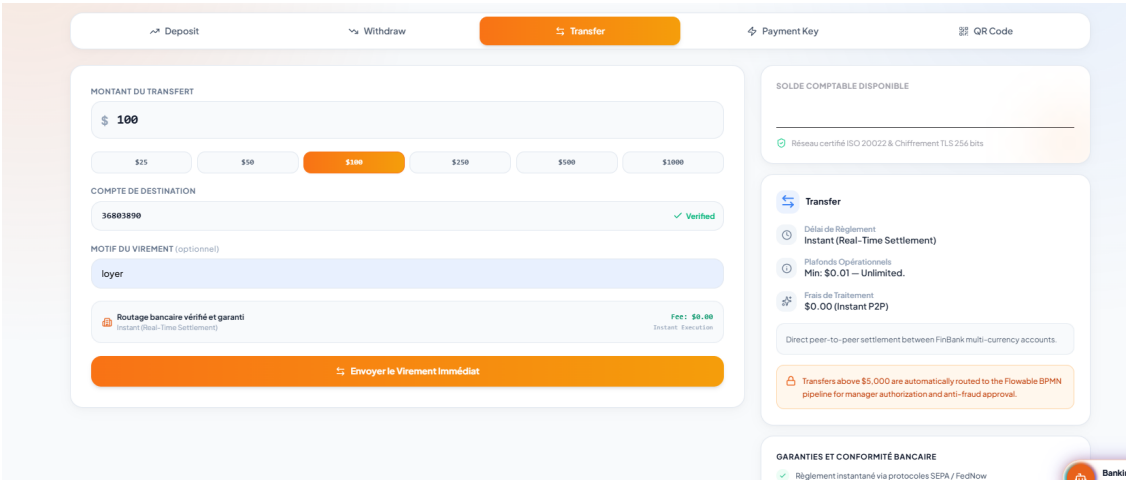


Fig 4.10 : Interface des virements

L'utilisateur renseigne le compte destinataire et le montant. Le système contrôle le solde et la validité des comptes avant de débiter le compte source et de créditer le compte cible. Le résultat de l'opération (succès ou motif de refus) est affiché à l'écran.

La figure 4.11 montre la notification reçue par l'utilisateur à l'issue du virement.

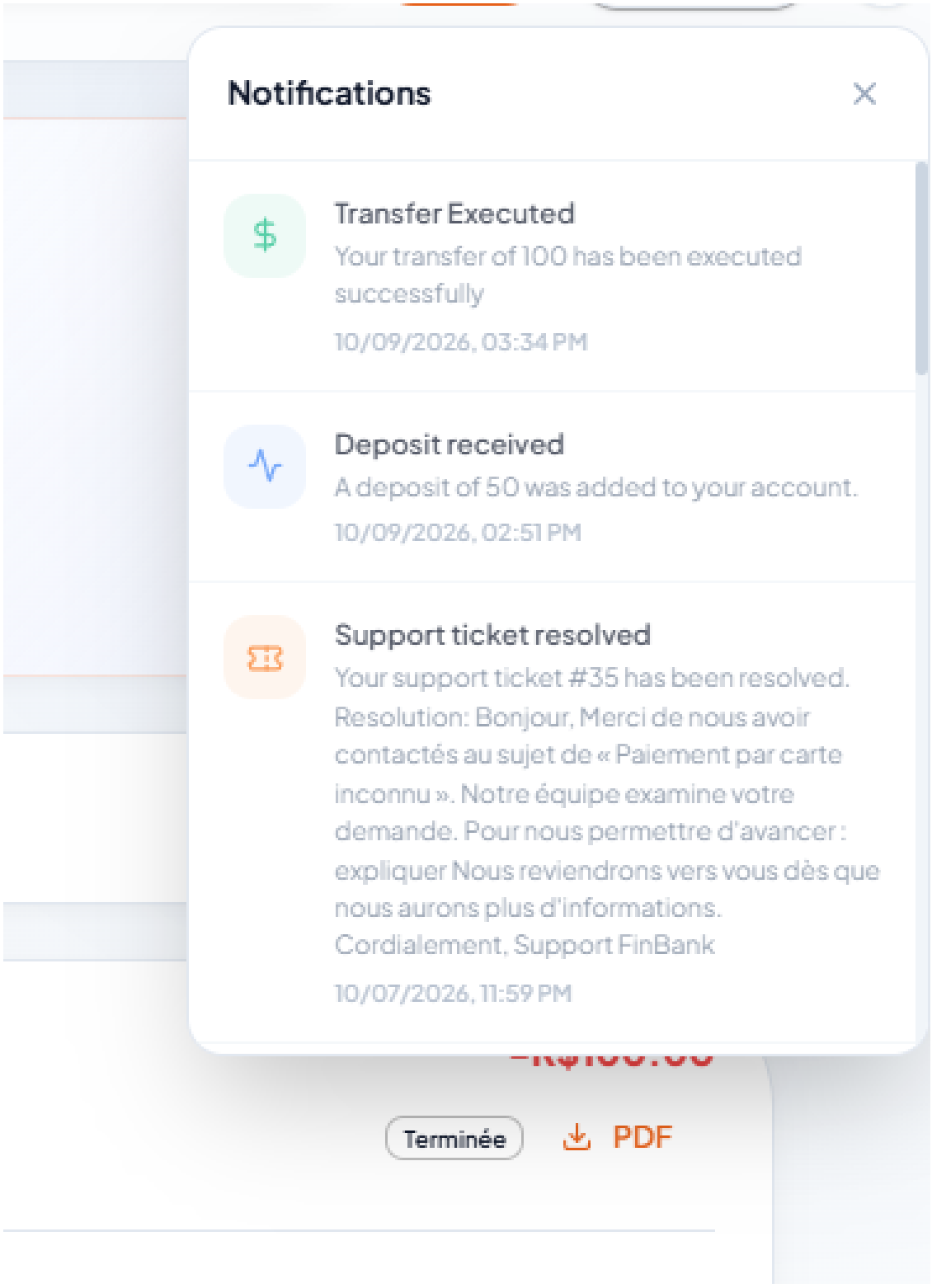


Fig 4.11 : Notification reçue après un virement

Cette notification confirme le résultat du virement sans que l'utilisateur ait à consulter son historique. Elle est générée par le système à la fin de l'opération.

4.4.3.1 Architecture événementielle avec Kafka et Redis

Après validation et enregistrement d'une opération, le producteur `TransactionEventProducer` publie au format JSON dans le topic `transactions` un événement contenant l'identifiant, le type, le montant, le statut et les comptes concernés. Le consommateur `TransactionEventConsumer` (groupe `banking-group`) traite ce message de façon asynchrone, sans retarder la réponse bancaire. Pour un virement au statut `COMPLETED`, il identifie les destinataires et déclenche l'envoi du reçu par e-mail.

Comme un événement peut être retraité, Redis déduplique les envois avec la clé `transaction-receipt:<transactionId>:<email>`, créée par `setIfAbsent` avec une durée de vie de sept jours. Un reçu est envoyé uniquement si la clé est nouvelle. Si Redis est indisponible, l'erreur est journalisée et l'envoi est maintenu sans déduplication, avec un risque de doublon. La notification affichée dans l'application reste distincte du reçu : elle est persistée par le service de notification, tandis que Kafka transporte l'événement technique.

Le schéma de la figure 4.12 résume ce flux et montre la place de Kafka et Redis dans le traitement.

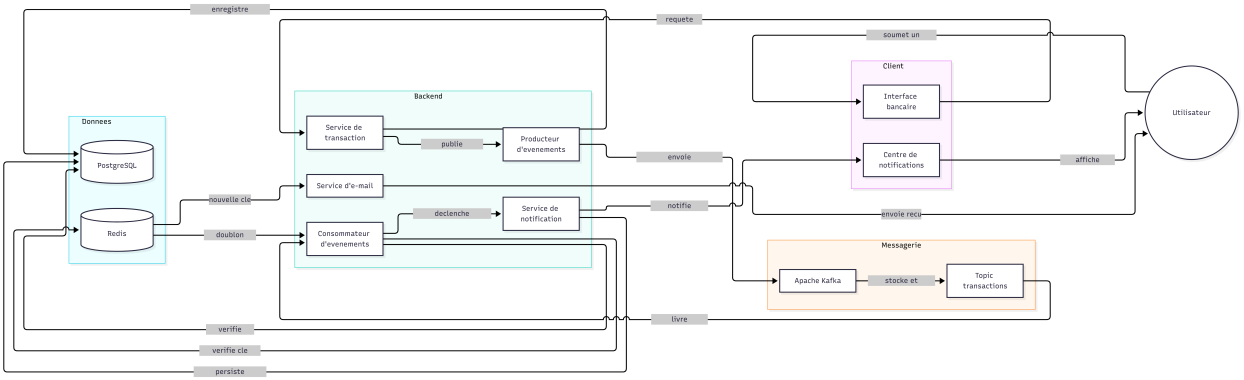


Fig 4.12 : Architecture de fonctionnement de Kafka et Redis

4.4.4 Interface de l'historique et des reçus

La figure 4.13 présente la liste des transactions et le téléchargement d'un reçu.

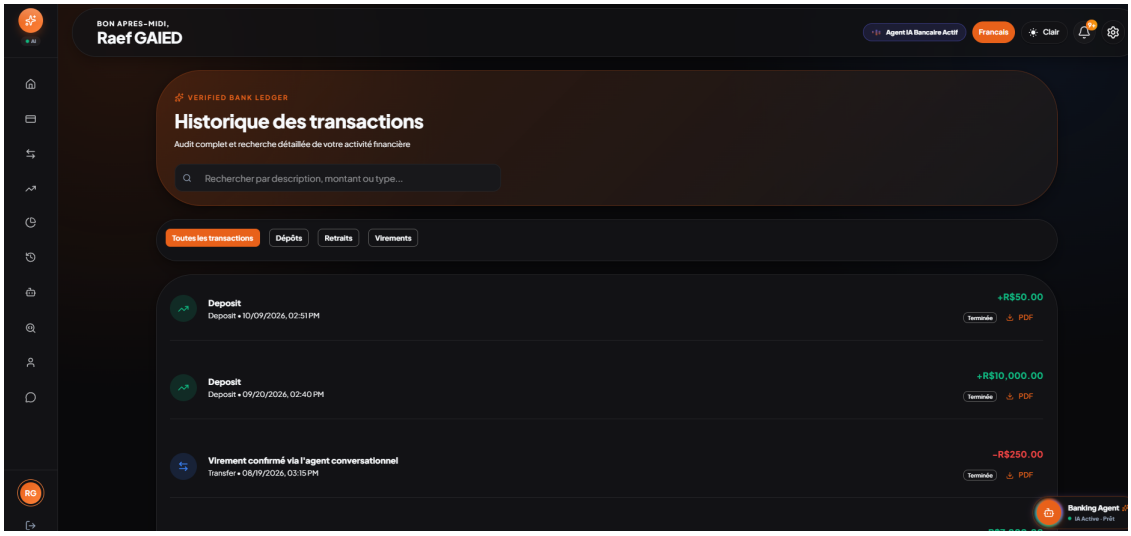


Fig 4.13 : Interface de l'historique et des reçus

L'historique liste les opérations de l'utilisateur avec leur type, leur montant, leur date et leur statut, et propose une recherche et un filtre par type. Pour chaque transaction, l'utilisateur peut télécharger un reçu au format PDF. Avant de produire le document, le système vérifie que l'utilisateur est bien propriétaire du compte source ou du compte cible de l'opération.

La figure 4.14 montre un exemple du reçu PDF généré pour une transaction.

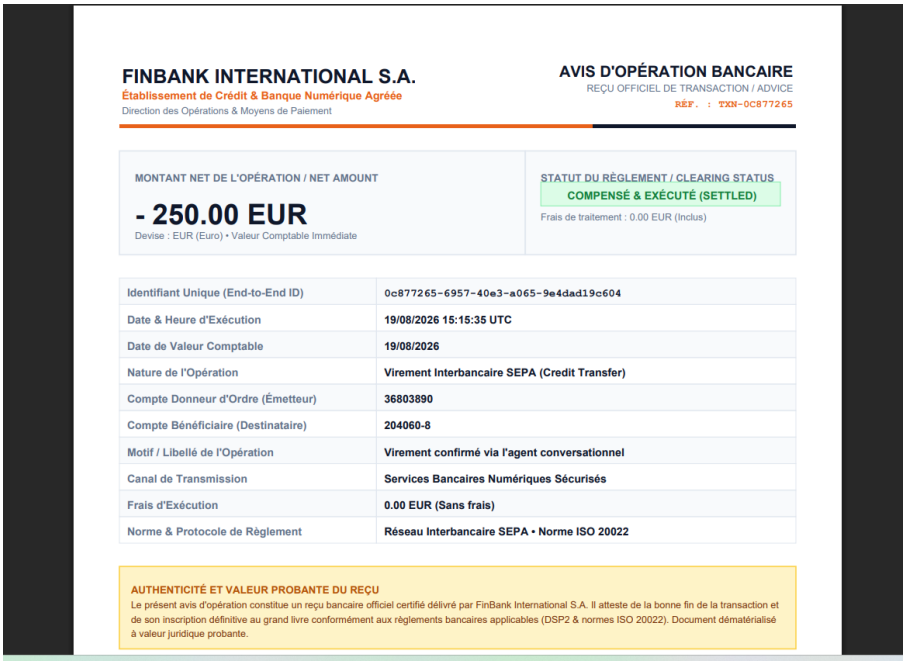


Fig 4.14 : Exemple de reçu PDF d'une transaction

4.4.5 Interface de gestion des cartes

La figure 4.15 présente la gestion des cartes bancaires.

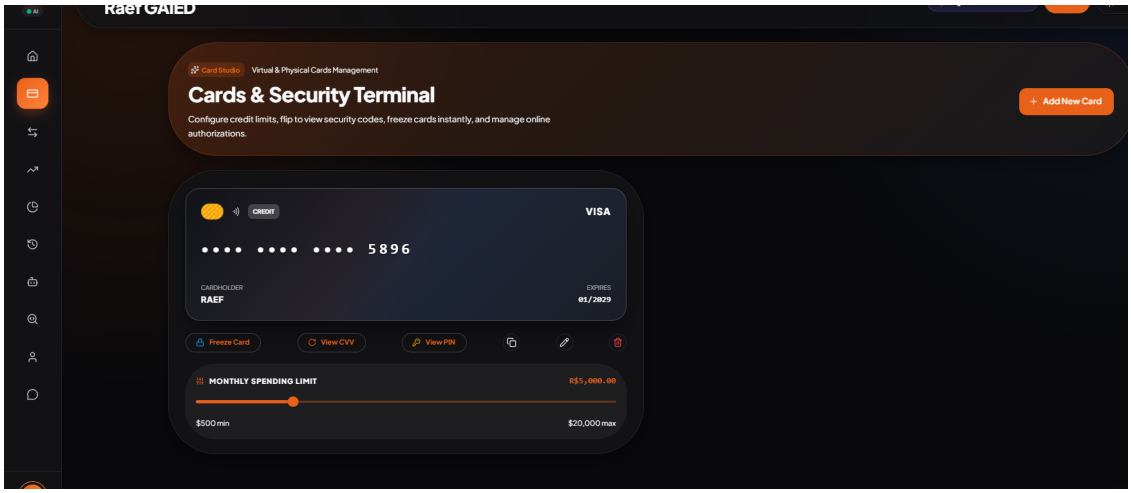


Fig 4.15 : Interface de gestion des cartes

L'utilisateur consulte ses cartes, en crée une nouvelle (crédit ou débit), modifie les informations autorisées et supprime une carte. Ces opérations ne s'appliquent qu'aux cartes de l'utilisateur connecté.

4.4.6 Interface de gestion des investissements

La figure 4.16 présente la gestion des investissements.

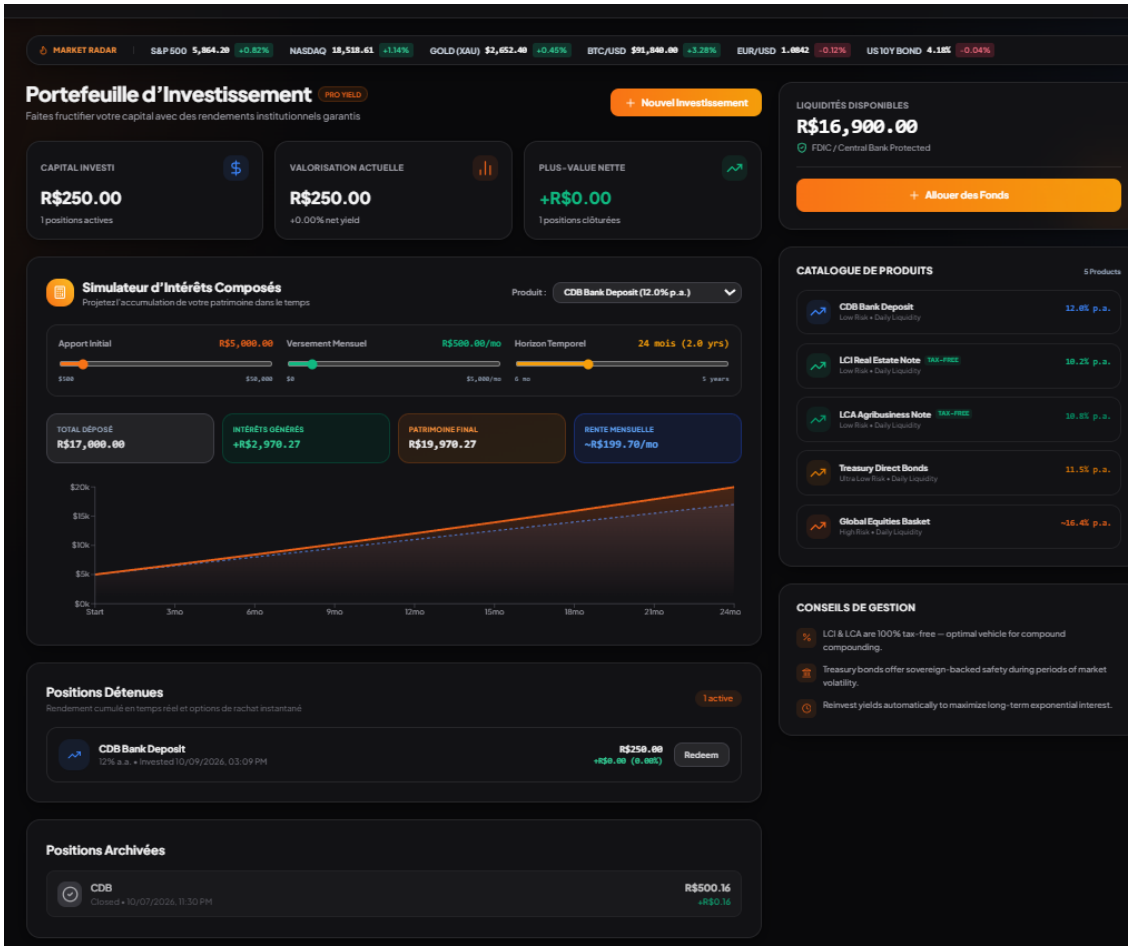


Fig 4.16 : Interface de gestion des investissements

L'utilisateur consulte ses placements, investit un montant dans un produit disponible (par exemple CDB, LCI ou LCA) et demande le remboursement d'un investissement actif. Après remboursement, le statut de l'investissement passe à « remboursé » et le compte est crédité.

4.4.7 Interface des clés de paiement

La figure 4.17 présente la gestion des clés de paiement.

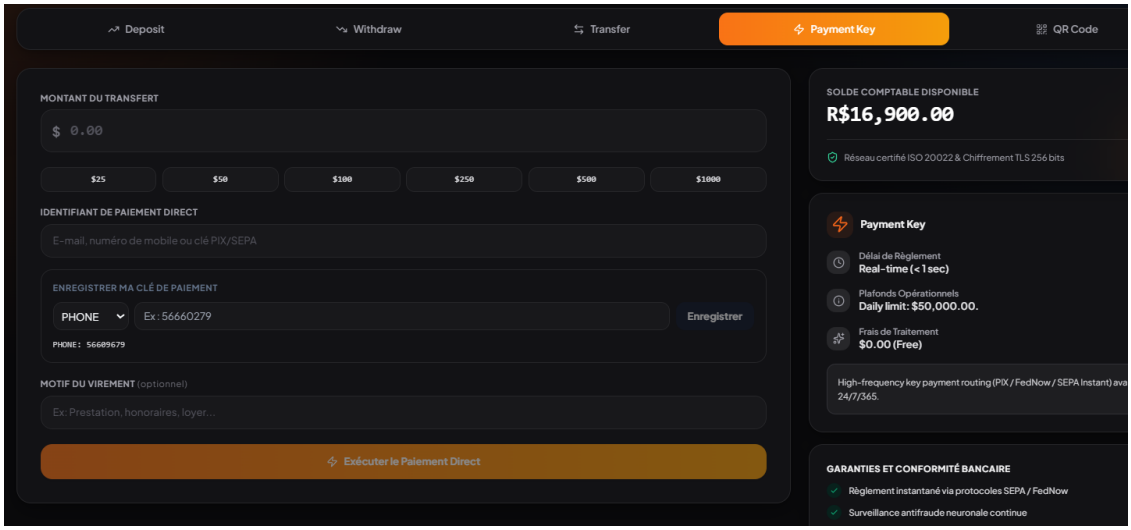


Fig 4.17 : Interface des clés de paiement

L'utilisateur enregistre et consulte les clés associées à son compte. Une clé peut aussi être recherchée afin d'identifier un destinataire avant d'exécuter un transfert par clé, sans avoir à connaître son numéro de compte.

4.5 Conclusion

Ce chapitre a présenté l'étude et la réalisation du **Sprint 2** de la plateforme FinBank. Nous avons commencé par définir le **backlog du sprint**, puis nous avons modélisé les fonctionnalités à travers les diagrammes de cas d'utilisation, de classes et de séquence. La section de réalisation a présenté les huit interfaces développées, chacune accompagnée d'une description de son rôle.

Ce sprint apporte les **fonctionnalités bancaires opérationnelles** de la plateforme : consultation du compte, dépôts, retraits, virements, historique, reçus, cartes, investissements et clés de paiement. Ces services constituent le socle fonctionnel exploité par les chapitres suivants.

5

Sprint 3 : Assistant bancaire et base de connaissances RAG

5.1 Introduction

Dans ce chapitre, nous présentons l'étude et la réalisation du **troisième sprint** de la plateforme FinBank. Ce sprint introduit l'**assistant bancaire conversationnel** et le système de recherche augmentée par récupération **(Retrieval-Augmented Generation – RAG)**.

Les travaux couvrent la détection d'intention, la gestion des sessions et des messages, la prise de décision de l'agent, les appels d'outils bancaires, l'intégration d'Ollama et la recherche documentaire. Le sprint inclut également la préparation, l'indexation et l'exploitation des documents de connaissance, ainsi que l'extraction d'informations depuis un texte ou un fichier.

Ce chapitre est organisé en trois parties : le backlog, la conception et la réalisation. L'orchestration BPMN et le traitement des tâches admin font l'objet du chapitre 6.

5.2 Backlog du Sprint

Le tableau 5.1 présente les tâches du Sprint 3.

Tab 5.1 : Backlog du Sprint 3

ID	Tâche	Estimation
S3-T1	Détection d'intention et identification de la langue	3 j
S3-T2	Gestion des sessions conversationnelles	3 j
S3-T3	Conservation et consultation des messages	3 j
S3-T4	Conception de l'agent bancaire et de ses décisions	4 j
S3-T5	Intégration du modèle Ollama et des réponses conversationnelles	3 j
S3-T6	Mise en place de la recherche RAG avec pgvector	4 j
S3-T7	Gestion, activation et réindexation des documents RAG	3 j
S3-T8	Extraction d'informations depuis un texte ou un fichier	2 j
S3-T9	Intégration et vérification de l'interface conversationnelle	2 j
	Total	**27 jours**

5.3 Conception

La conception du Sprint 3 modélise les interactions entre l'utilisateur, l'interface de conversation, le backend bancaire, le modèle de langage et la base de connaissances. L'administrateur gère les documents utilisés par le RAG.

5.3.1 Diagramme de cas d'utilisation

L'utilisateur authentifié peut ouvrir une session, envoyer des messages et interagir avec l'assistant. Le système détecte l'intention et choisit le traitement adapté : répondre, exécuter une action bancaire, demander une confirmation ou orienter vers un workflow. L'administrateur gère les documents RAG et leur indexation.

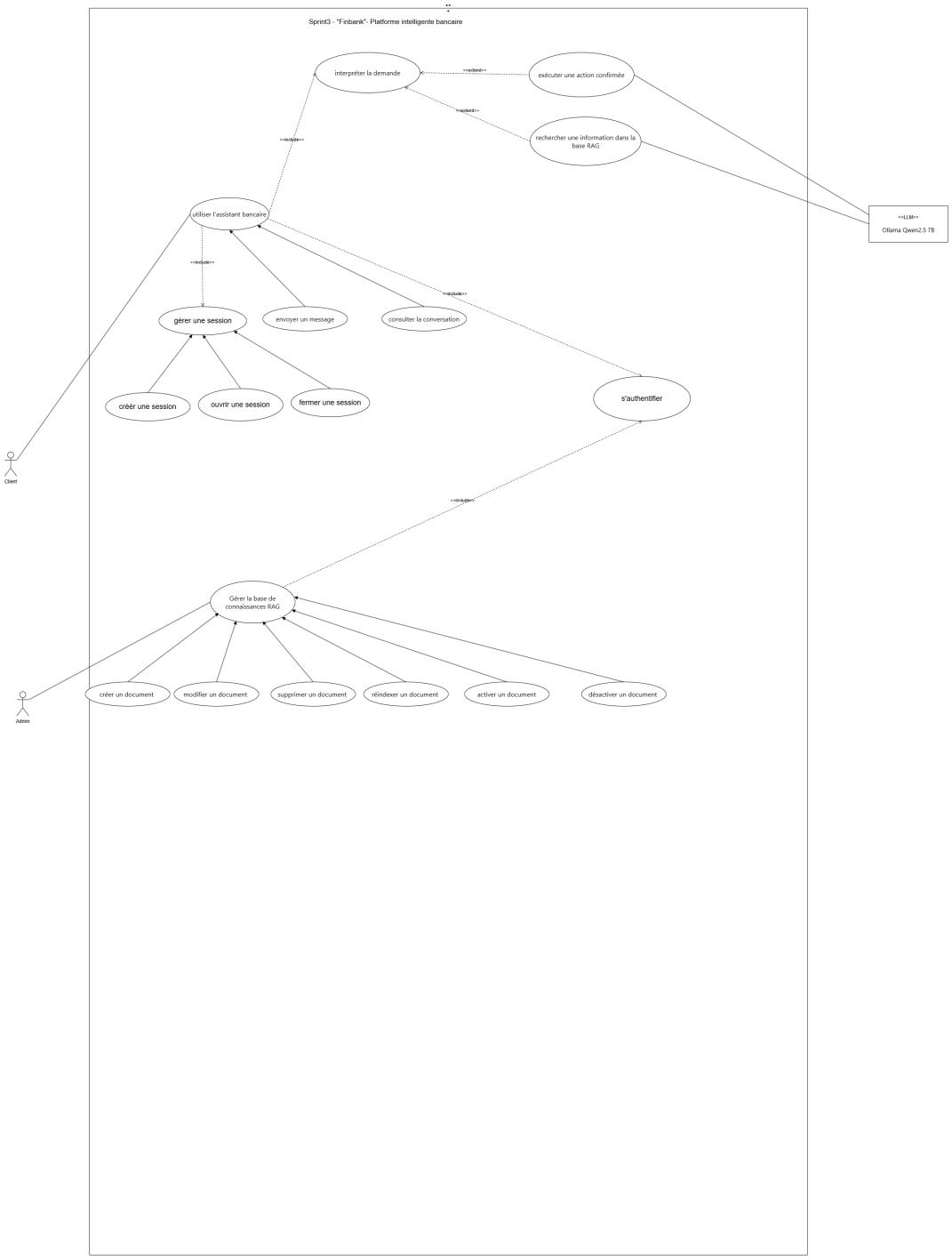


Fig 5.1 : Diagramme de cas d'utilisation du Sprint 3

Le diagramme distingue les fonctionnalités conversationnelles de l'utilisateur et les fonctions d'administration réservées au rôle `ADMIN`. L'authentification constitue une précondition héritée du Sprint 1 et n'est pas représentée comme une fonction propre à ce sprint. Les actions sensibles restent soumises aux contrôles de la plateforme, indépendamment des réponses de l'assistant.

5.3.2 Diagramme de classes

Le modèle repose sur quatre entités principales. `Session` identifie une conversation et conserve son état, ses dates d'activité et l'agent associé. `Message` stocke le contenu d'un échange, la langue détectée, l'intention et les indicateurs RAG. `BankingAgent` mémorise la décision courante de l'agent, sa politique par défaut et le contexte JSON. `RagDocument` représente un document de connaissance avec son titre, son contenu, sa source et son état d'activation.

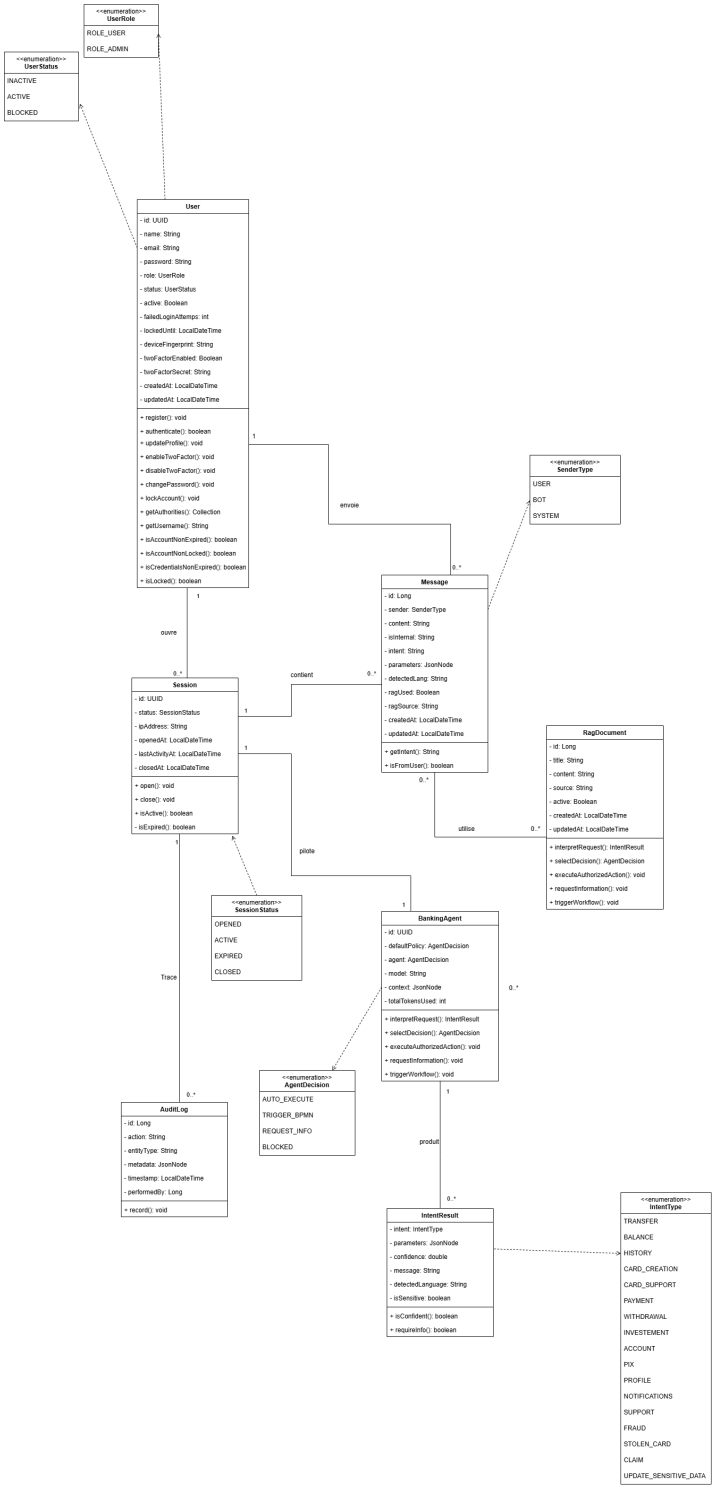


Fig 5.2 : Diagramme de classes du Sprint 3

Le champ `ragUsed` trace si une réponse a exploité la recherche RAG, tandis que `ragSource` conserve la source associée. Les documents RAG sont persistés séparément des messages afin de pouvoir être administrés et réindexés indépendamment.

5.3.3 Description textuelle des cas d'utilisation

5.3.3.1 Interagir avec l'assistant bancaire

Tab 5.2 : Description textuelle – Interagir avec l'assistant bancaire

Élément	**Description**
Titre	Interagir avec l'assistant bancaire
Acteur	Client (`ROLE_USER`)
But	Obtenir une réponse ou réaliser une action bancaire en langage naturel.
Résumé	L'utilisateur envoie un message dans une session. L'agent détecte l'intention et choisit le traitement adapté.
Précondition	L'utilisateur est authentifié et la session est active.
Postcondition	Le message et la réponse sont conservés dans la conversation.
Scénario principal	1. L'utilisateur ouvre ou crée une session. 2. Il saisit une question ou une demande. 3. Le système vérifie la session et enregistre le message. 4. L'agent détecte l'intention et détermine une décision. 5. Le système produit la réponse et la conserve.
Scénario alternatif	Si l'IA est indisponible ou si une information manque, le système retourne une erreur ou demande une précision.

5.3.3.2 Gérer les documents de connaissance

Tab 5.3 : Description textuelle – Gérer les documents de connaissance

Élément	**Description**
Titre	Gérer les documents de connaissance
Acteur	Administrateur (`ROLE_ADMIN`)
But	Maintenir les contenus utilisés par la recherche RAG.
Résumé	L'administrateur crée, modifie, active, désactive, supprime ou réindexe un document.
Précondition	L'administrateur est authentifié. Le titre et le contenu sont obligatoires.
Postcondition	Le document est persisté et son vecteur recalculé si le fournisseur d'embeddings est disponible.
Scénario principal	1. L'administrateur ouvre la gestion des documents. 2. Il saisit le titre, le contenu et la source. 3. Le système valide les champs obligatoires. 4. Le document est enregistré avec son état. 5. Le contenu est vectorisé et stocké dans pgvector.
Scénario alternatif	Si l'utilisateur n'est pas admin ou si les données sont invalides, l'opération est refusée.

5.3.4 Diagrammes de séquence

5.3.4.1 Question conversationnelle et recherche RAG

La figure 5.3 décrit le flux d'une question d'information : enregistrement du message, détection d'intention, recherche de contexte, génération de la réponse et conservation du résultat.

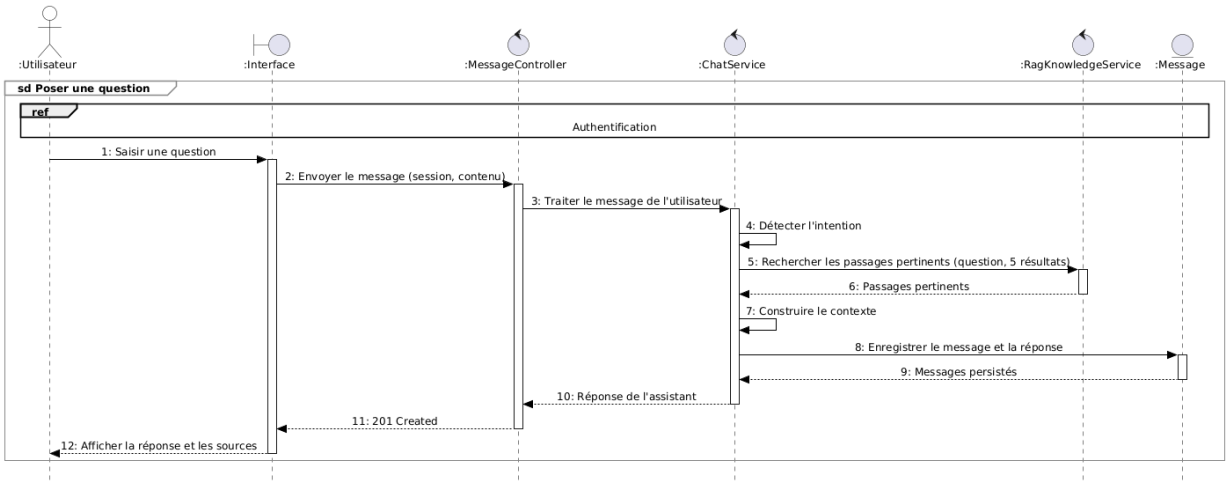


Fig 5.3 : Diagramme de séquence – Question et recherche RAG

5.3.4.2 Gestion et réindexation d'un document RAG

La figure 5.4 présente le cycle d'un document depuis sa validation jusqu'au calcul de son vecteur d'embedding.

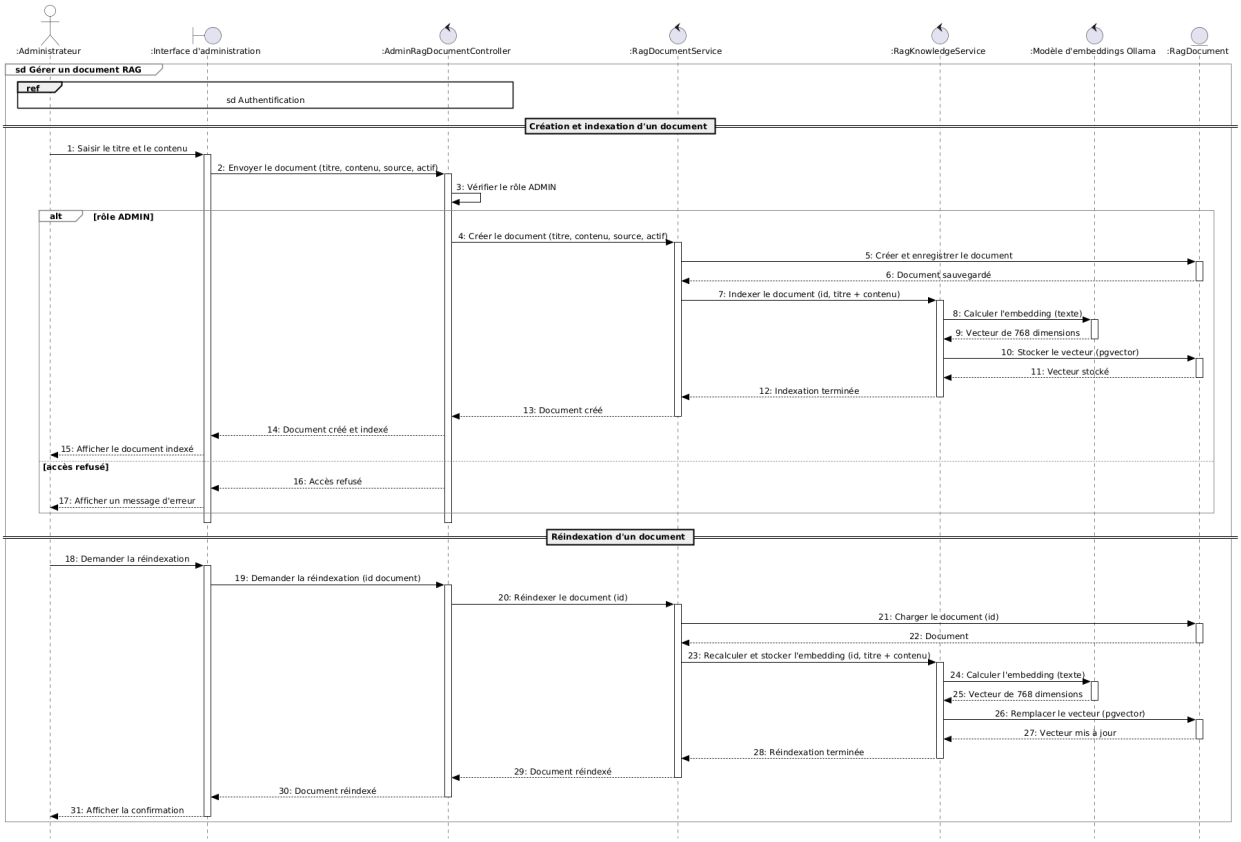


Fig 5.4 : Diagramme de séquence – Création et réindexation d'un document RAG

Ces séquences mettent en évidence la séparation entre la conversation, la recherche documentaire et l'administration de la base de connaissances. Le contexte récupéré est fourni au modèle comme contenu vérifié, sans remplacer les contrôles de sécurité.

5.4 Réalisation

Cette section présente la réalisation de l'assistant bancaire, de son architecture de décision et du système RAG.

5.4.1 Architecture générale de l'agent

Le traitement conversationnel est orchestré par le `ChatService` du backend Spring Boot. À la réception d'un message, le service vérifie la session, l'utilisateur et limite le contexte transmis à l'agent. Il crée le `BankingAgent`, détecte l'intention et détermine une décision parmi quatre choix : répondre, demander une information, appeler un outil bancaire, ou orienter vers un workflow.

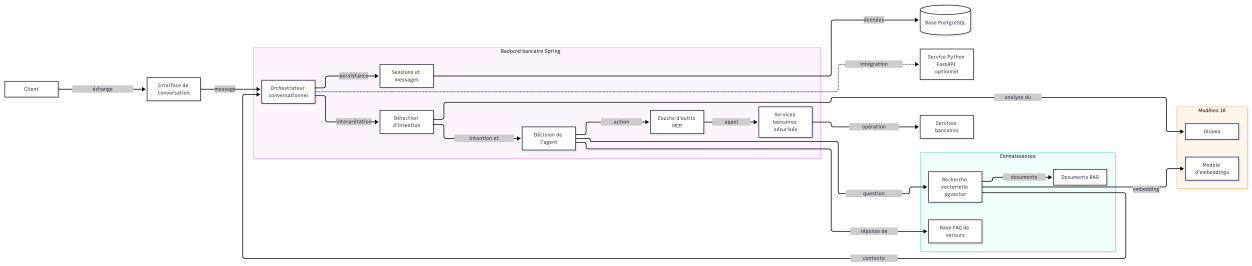


Fig 5.5 : Architecture de l'agent bancaire conversationnel

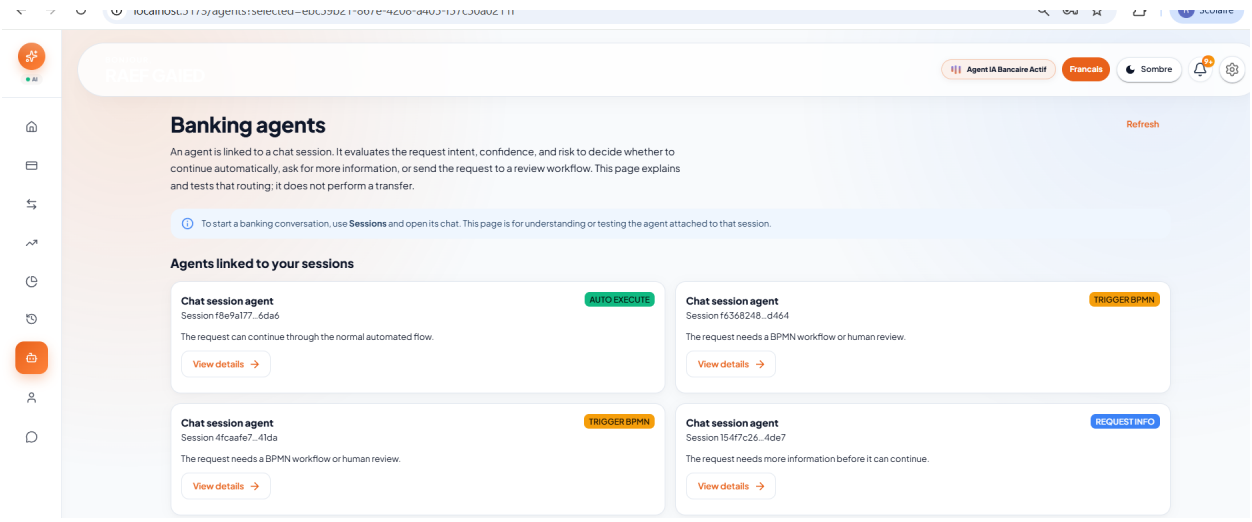


Fig 5.6 : Agents bancaires associés aux sessions et décisions affichées

Chaque agent est associé à une session et à une décision de traitement. Les actions financières passent toujours par les services et contrôles métier ; elles ne sont jamais exécutées sur la seule base d'une réponse textuelle.

5.4.2 Intégration du protocole MCP

Le projet intègre le protocole **MCP** (*Model Context Protocol*) comme couche standardisée d'accès aux outils de l'agent. MCP décrit les capacités disponibles, valide les paramètres, appelle les services du backend et normalise le résultat retourné à l'agent. Il ne contient pas les règles bancaires et ne remplace pas le backend Spring Boot.

Les outils MCP implémentés permettent de consulter le compte et l'historique, de rechercher un document RAG, de préparer un virement, de gérer la session et de conserver la décision de l'agent. Les outils d'administration sont protégés par le rôle `ADMIN`.

Le fonctionnement suit quatre étapes : l'agent sélectionne un outil ; MCP valide les données et appelle le service métier ; le backend applique les contrôles de sécurité et de confirmation ; le résultat est retourné sous forme structurée pour la réponse conversationnelle. Les appels MCP sont tracés dans l'audit sans enregistrer les données sensibles.

5.4.3 Détection d'intention et choix du modèle

La détection d'intention est assurée en priorité par **Ollama**. Un prompt structuré impose au modèle un objet JSON contenant l'intention, les paramètres, le niveau de confiance, la langue et l'indicateur de sensibilité. Les intentions couvrent le solde, l'historique, le virement, le retrait, la carte, l'investissement, le profil, le support et la fraude.

Si Ollama est indisponible ou retourne une réponse invalide, le `RuleBasedIntentDetectionService` prend le relais grâce à des règles déterministes. Le système supporte le français et l'anglais.

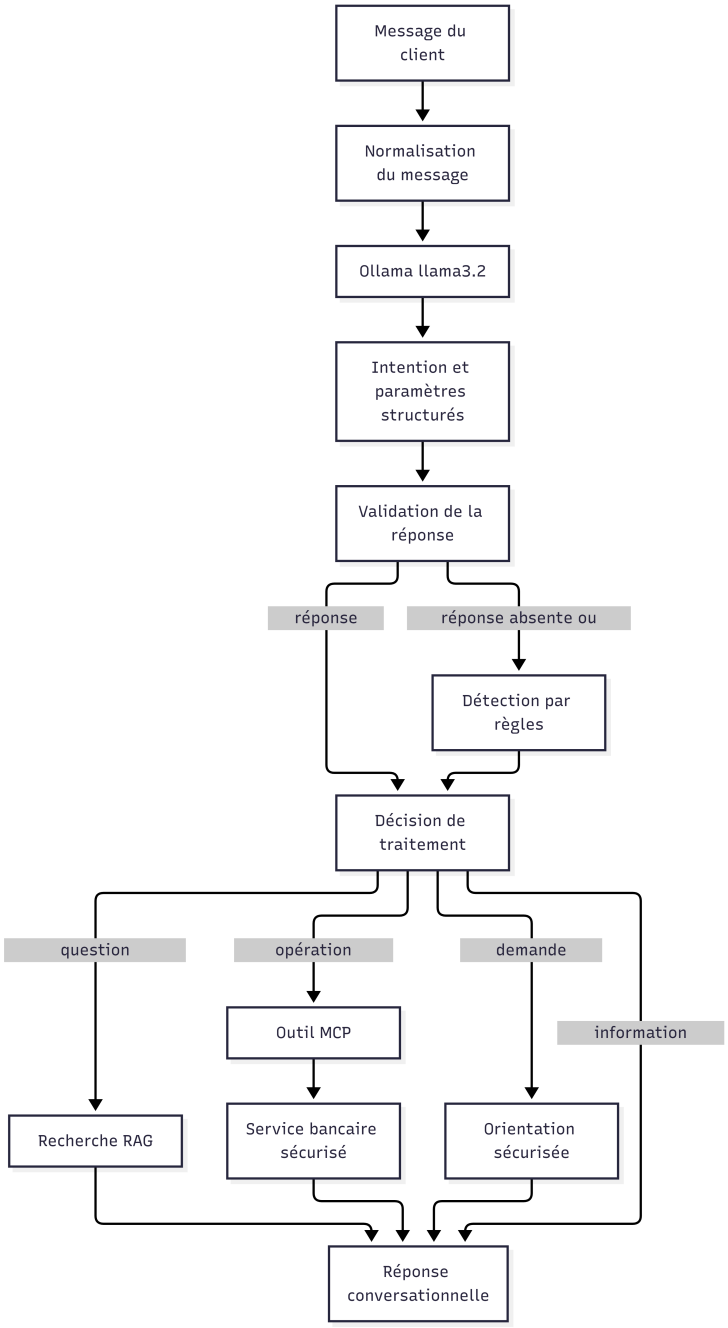
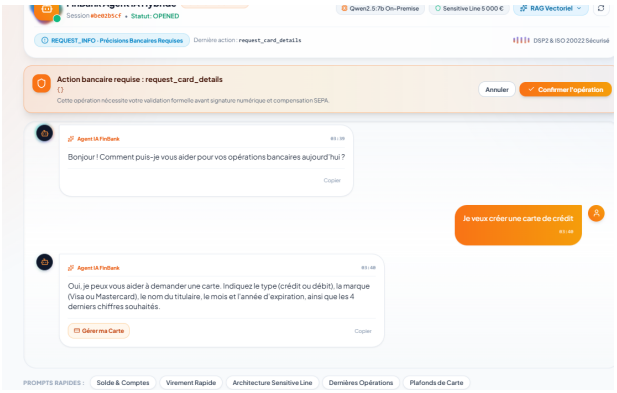


Fig 5.7 : Détection d'intention et mécanisme de repli

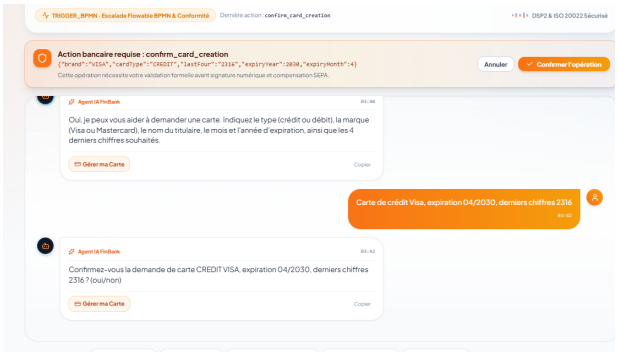
Des contrôles déterministes complètent la détection pour les cas critiques : modification d'e-mail, demandes liées au compte. Les informations financières ne sont jamais reprises depuis l'historique sans confirmation explicite de l'utilisateur.

5.4.4 Traitement conversationnel et outils bancaires

Le `BankingAgentService` détermine la décision après la détection. L'agent peut consulter le solde et l'historique, traiter un virement avec collecte et confirmation, accompagner la création d'une carte et orienter certaines demandes de support. Les demandes sensibles (fraude, réclamation, modification de données) sont toujours transmises aux workflows sécurisés du chapitre 6.

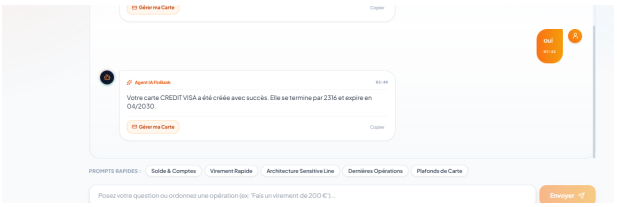


(a) Collecte des informations

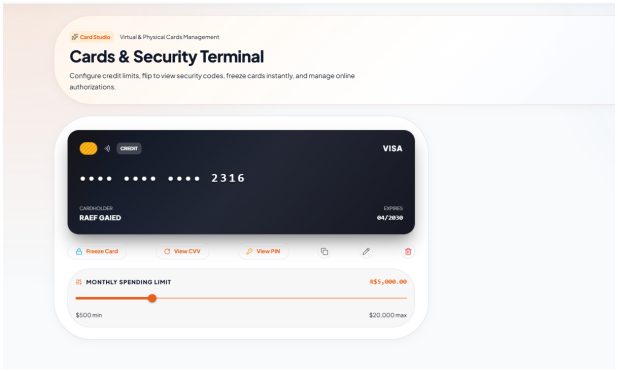


(b) Récapitulatif et confirmation demandée

Fig 5.8 : Collecte des informations et confirmation – création de carte



(a) Confirmation dans la conversation



(b) Carte affichée dans l'interface

Fig 5.9 : Confirmation de la création et vérification de la carte enregistrée

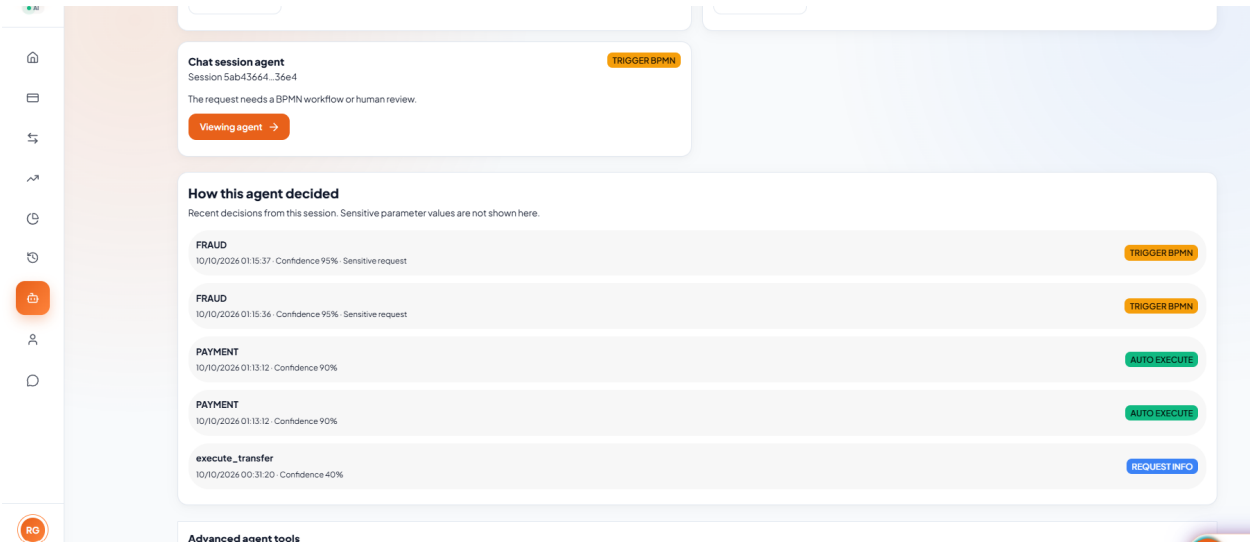


Fig 5.10 : Historique des décisions associées à un agent

La figure 5.10 montre l'intention, le niveau de confiance et la décision retenue pour chaque message, avec les valeurs sensibles masquées.

5.4.5 Service Python FastAPI

Le service Python (`ai-agent`) est une intégration **optionnelle** contrôlée par la propriété `ai.agent.enabled`. Il expose `GET /health` et `POST /chat`, et communique avec le backend via `PythonAgentGateway`. Si le service est indisponible, le traitement Java prend le relais.

Le cycle d'une requête est le suivant : le service valide le corps JSON, vérifie le JWT, applique une limite de vingt requêtes par utilisateur sur dix secondes, exécute l'agent et retourne la réponse avec les actions suggérées et le temps de traitement.

Le module `agent.py` reconnaît les demandes de solde, d'historique, de virement et de réclamation. Pour un virement, il collecte le montant et le compte destinataire, puis exige une confirmation explicite avant d'appeler le service métier. Le module `llm.py` choisit entre Ollama `qwen2.5:7b`, OpenAI et Groq selon leur disponibilité. Les outils de `tool_executor.py` appellent directement les API du backend ; le virement inclut une protection contre les doublons par clé d'idempotence locale.

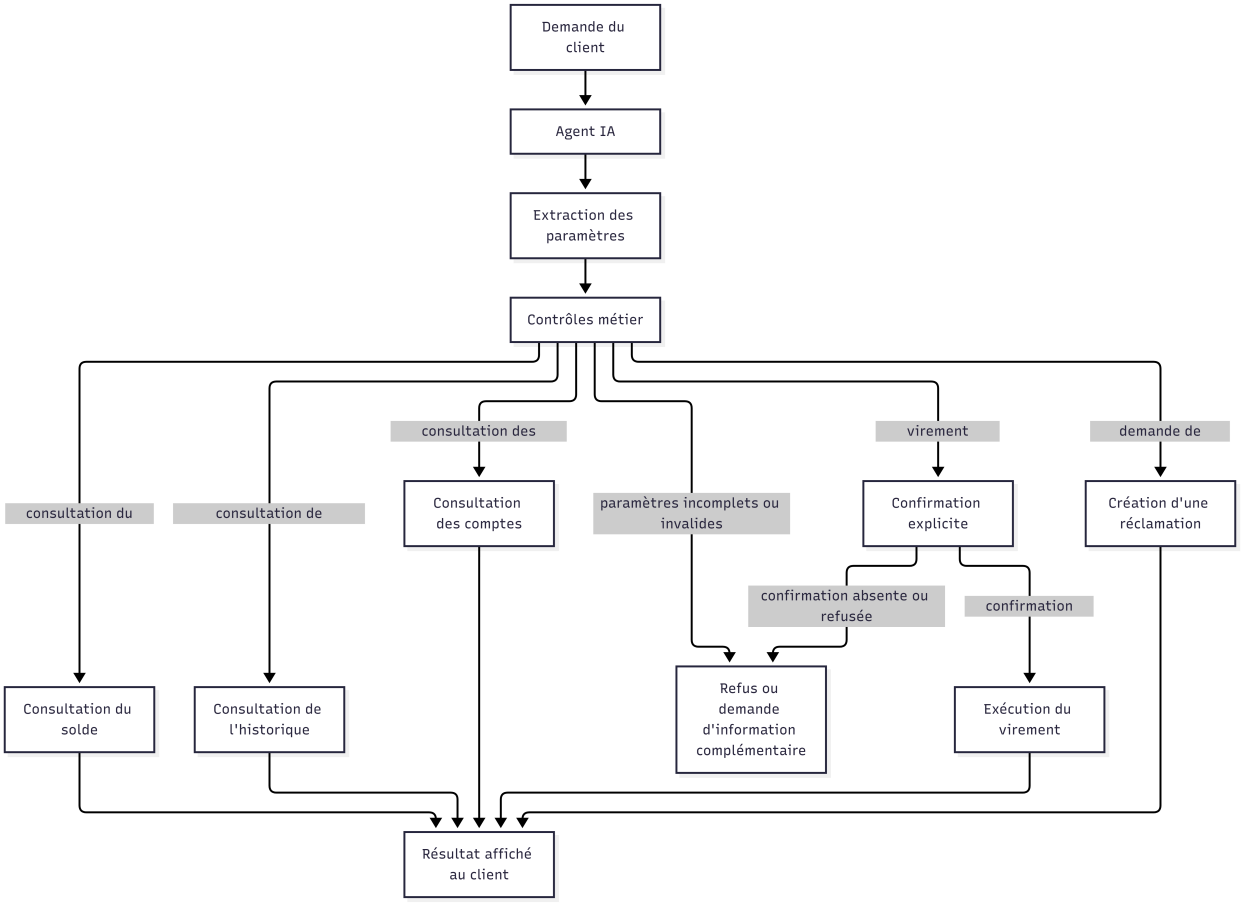


Fig 5.11 : Traitement d'une demande par l'agent et ses outils

5.4.6 Gestion des sessions et des messages

Les sessions sont persistées avec un état, un utilisateur et une date de dernière activité. Une session fermée ou expirée ne peut plus recevoir de message. Chaque message conserve l'intention détectée, les paramètres, la langue, l'indicateur `ragUsed` et la source RAG le cas échéant.

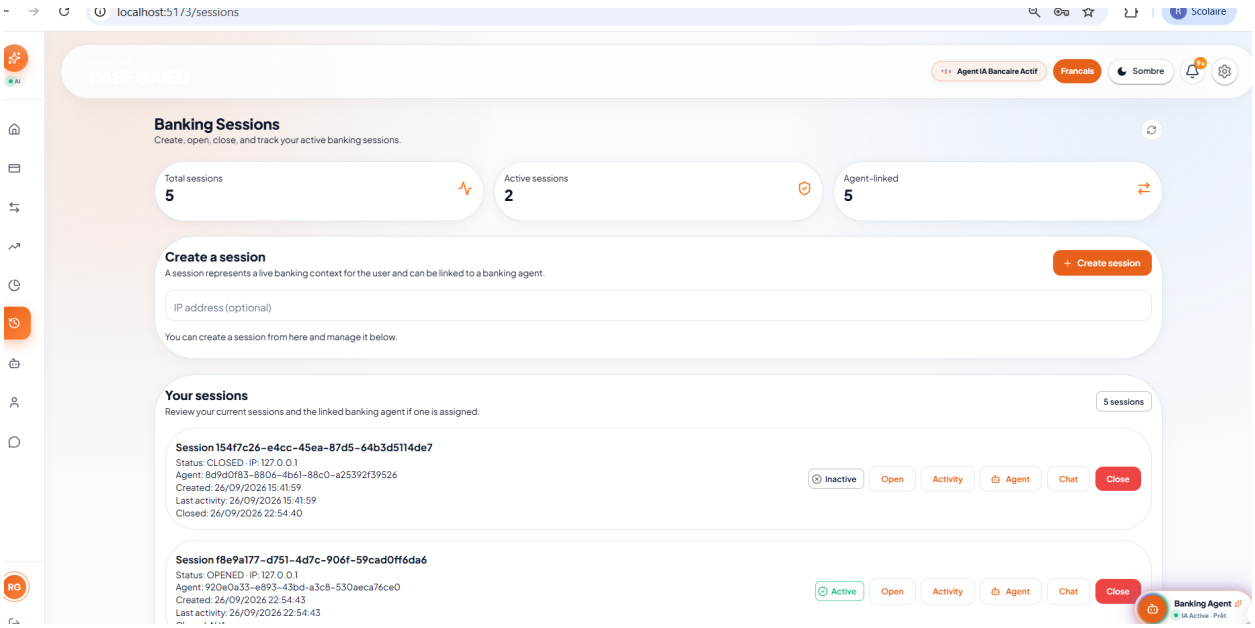


Fig 5.12 : Interface de gestion des sessions conversationnelles

Cette interface permet de visualiser l'état des sessions, leur activité récente et l'agent associé.

5.4.7 Préparation et indexation des données RAG

Lors de la création ou de la modification d'un document, le titre et le contenu sont concaténés puis transmis au modèle Ollama `nomic-embed-text`, qui produit un vecteur de 768 dimensions. Ce vecteur est enregistré dans la colonne vectorielle de la table `rag_documents` via pgvector. Si le fournisseur d'embeddings est indisponible, le document reste persisté et peut être réindexé ultérieurement.

5.4.8 Récupération du contexte et génération de la réponse

Pour une demande d'information, le `RagKnowledgeService` vectorise la question et interroge PostgreSQL avec l'opérateur de distance cosinus `<=>` de pgvector. Seuls les documents actifs avec un embedding sont considérés. Les résultats sont filtrés par un seuil de similarité de `0.72` et limités à trois. Ils sont assemblés dans un contexte transmis au modèle avec la question et l'historique récent. En cas d'échec, le système bascule sur une recherche lexicale.

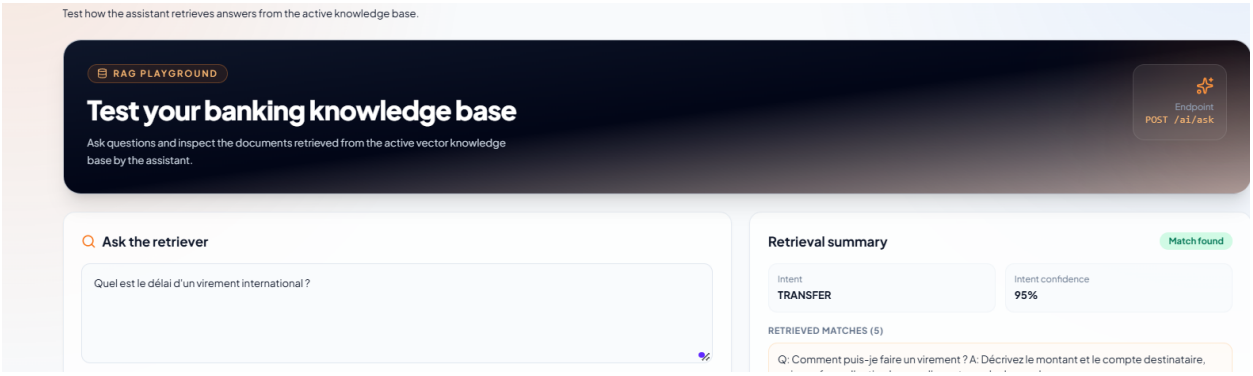


Fig 5.13 : Test du moteur RAG – correspondances documentaires récupérées

La capture montre une question de test, l'intention détectée avec un niveau de confiance de 95% et les correspondances documentaires remontées. Elle atteste la récupération de passages pertinents, sans constituer une preuve d'exécution d'une opération bancaire.

5.4.9 Deux niveaux de recherche augmentée

Le projet dispose de deux mécanismes RAG distincts qu'il convient de différencier.

RAG Java (principal) : les documents sont administrés dans la table `rag_documents` et vectorisés par Ollama `nomic-embed-text` (768 dimensions, pgvector). La recherche utilise la distance cosinus avec un seuil de `0.72` et retourne au maximum trois résultats. La base est administrable, réindexable et intégrée au backend.

RAG Python (local) : le service utilise un fichier statique `ai-agent/rag/data.txt`, découpé en sections et vectorisé par TF-IDF (scikit-learn). La similarité cosinus sélectionne le meilleur résultat si son score atteint `0.2`. Cette base est locale et simplifiée ; elle ne remplace pas le RAG pgvector du backend Java.

5.4.10 Administration des documents et extraction

Les endpoints d'administration permettent de lister, créer, modifier, supprimer et réindexer les documents RAG. Chaque opération exige le rôle `ADMIN`. L'activation d'un document contrôle son inclusion dans la recherche.

Le service d'extraction accepte un texte brut ou un fichier multipart et en extrait : type de document, nom, e-mail, téléphone, adresse, montant, numéro de compte et produit. Il utilise des paires clé-valeur, des expressions régulières et une normalisation des types.

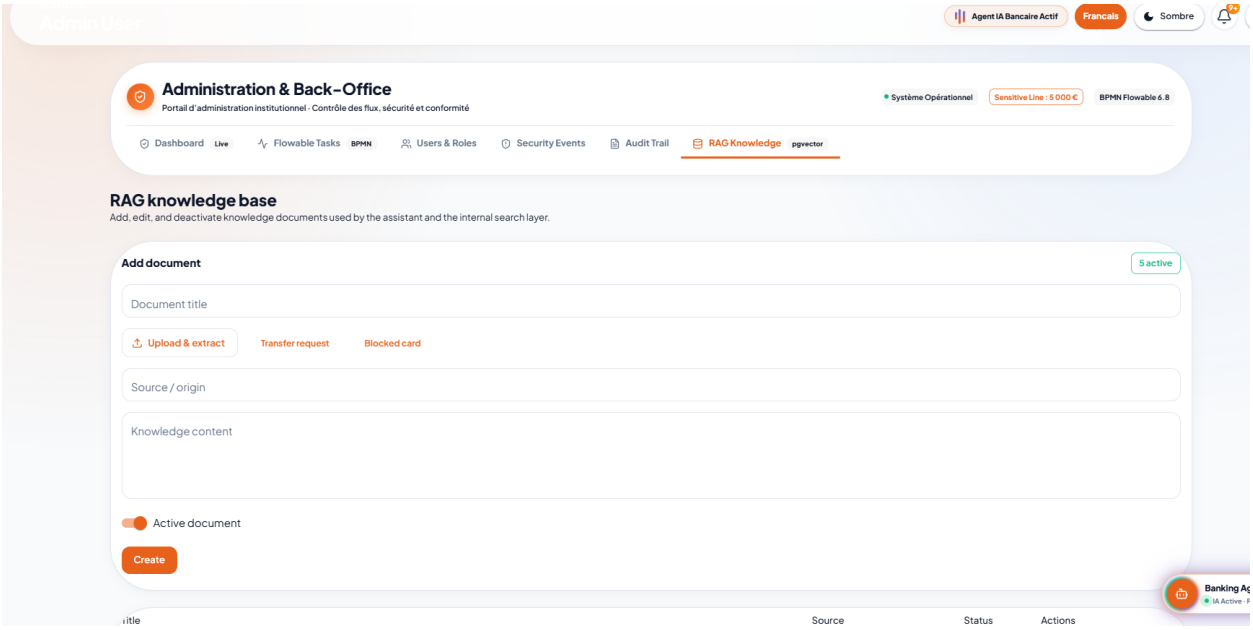
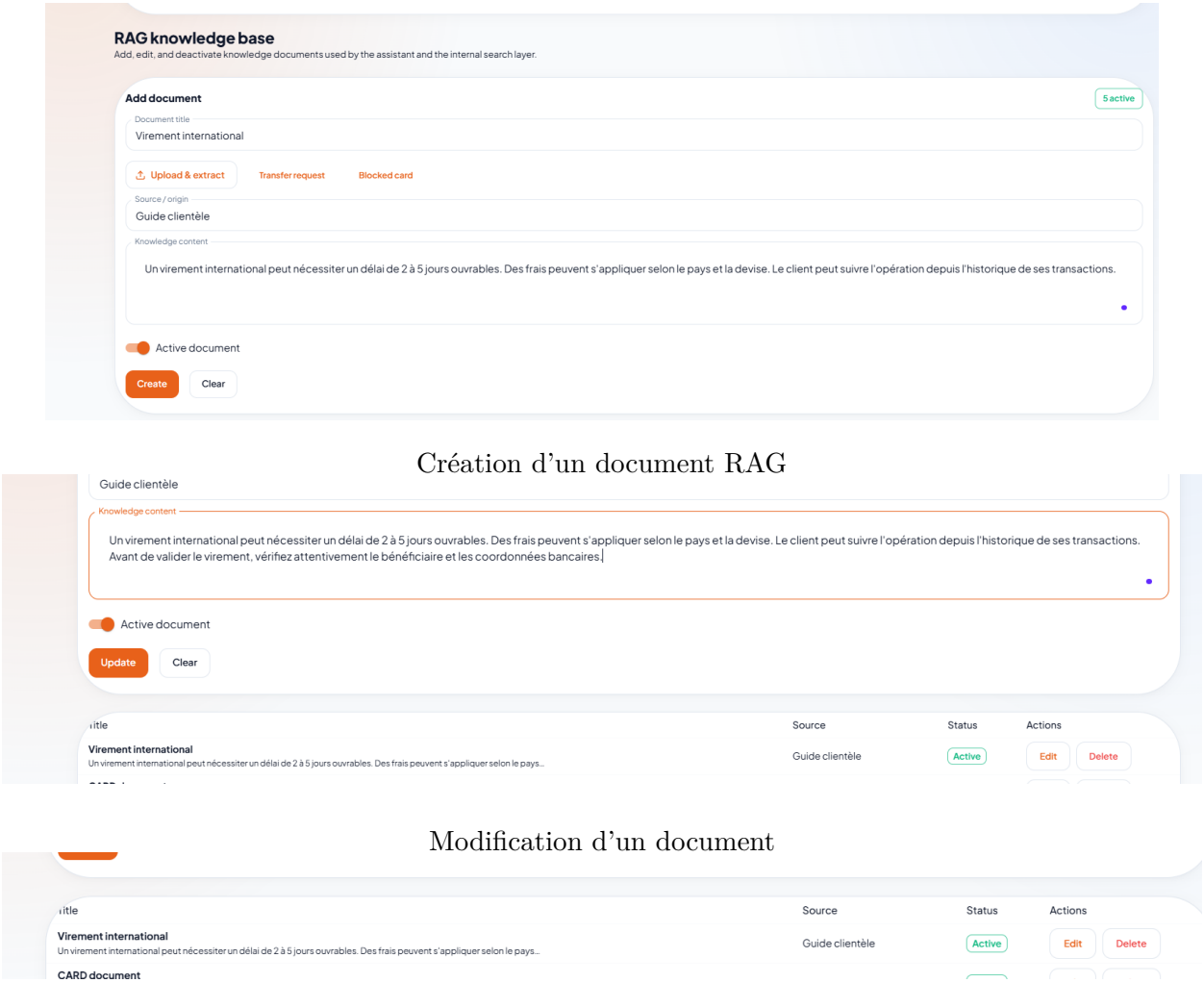


Fig 5.14 : Administration des documents de connaissance



Création d'un document RAG

Modification d'un document

Activation d'un document

Fig 5.15 : Étapes de gestion des documents de connaissance

La gestion documentaire maintient une base contrôlée par l'administrateur, tandis que l'extraction exploite des contenus transmis par l'utilisateur. Ces deux fonctions sont distinctes : l'extraction produit des champs structurés depuis un document, alors que le RAG indexe des connaissances pour répondre aux questions.

5.5 Conclusion

Ce chapitre a présenté l'étude et la réalisation du **Sprint 3** de FinBank. La conception a modélisé l'assistant conversationnel, la gestion des sessions, la détection d'intention et l'administration des documents RAG.

La réalisation a mis en place le modèle local Ollama, l'intégration optionnelle du service Python, la

vectorisation des documents avec pgvector et la génération de réponses ancrées sur un contexte vérifié. L'assistant constitue ainsi une couche conversationnelle contrôlée, capable d'informer l'utilisateur et d'orienter les actions sensibles vers les workflows sécurisés présentés au chapitre 6.

6

Sprint 4 : Workflows BPMN et validation des opérations sensibles

6.1 Introduction

Ce chapitre présente l'étude et la réalisation du **quatrième sprint** de la plateforme bancaire intelligente FinBank. Les sprints précédents ont mis en place les fondations sécuritaires, les opérations bancaires et l'assistant conversationnel. Ce sprint traite les situations qui ne peuvent pas être réglées automatiquement : celles qui exigent une validation métier, une intervention humaine ou une traçabilité renforcée.

Les travaux portent sur l'intégration du moteur de processus **Flowable**, la modélisation de processus BPMN exécutables et la gestion des tâches humaines. Trois familles de processus sont prises en charge : la validation des virements importants, le traitement des alertes de fraude et la validation des modifications de données sensibles. Le sprint comprend également la réalisation des interfaces React et TypeScript destinées au client et à l'administrateur.

Le principe directeur est de ne jamais laisser le modèle de langage décider seul d'une opération sensible. L'agent identifie la demande et peut déclencher un processus ; Flowable exécute ensuite les étapes prévues, les contrôles métier vérifient les conditions nécessaires et l'administrateur intervient lorsque le processus contient une tâche humaine. Les décisions, notifications et résultats sont

conservés afin de permettre le suivi de chaque demande.

Ce chapitre est organisé en trois parties : la première présente le **backlog du sprint** ; la deuxième détaille la **conception**, des diagrammes UML jusqu'aux modèles BPMN ; la troisième présente la **réalisation**, depuis l'intégration du moteur jusqu'aux interfaces qui rendent les processus visibles.

6.2 Backlog du Sprint

Le tableau 6.1 présente les tâches réalisées pendant le Sprint 4, avec une estimation de leur durée. Elles suivent l'ordre du chapitre : les tâches S4-T1 à S4-T5 concernent le moteur, les trois processus et la gestion des tâches humaines ; les tâches S4-T6 à S4-T8 concernent les interfaces.

Tab 6.1 : Backlog du Sprint 4

ID	Tâche	Estimation
S4-T1	Configurer Flowable et déployer les définitions BPMN exécutables	3 jours
S4-T2	Réaliser le processus de validation des virements importants	4 jours
S4-T3	Réaliser le processus de détection et de traitement de la fraude	4 jours
S4-T4	Réaliser le processus de validation des modifications de données sensibles	3 jours
S4-T5	Mettre en place les tâches humaines, les décisions et les délais de traitement	3 jours
S4-T6	Développer les interfaces client en React et TypeScript	4 jours
S4-T7	Développer l'espace administrateur et la supervision des processus	3 jours
S4-T8	Connecter les interfaces au backend et gérer les états de succès, d'erreur et de chargement	3 jours
	Total	**27 jours**

Les processus et les tâches humaines (S4-T2 à S4-T5) sont d'abord modélisés dans la section 6.3. La section 6.4 décrit ensuite l'intégration du moteur (S4-T1) puis l'usage de ces processus dans les interfaces (S4-T6 à S4-T8).

6.3 Conception

La conception du Sprint 4 distingue trois acteurs et niveaux de responsabilité. Le **client**, c'est-à-dire l'utilisateur de rôle `USER` des sprints précédents, déclenche une demande ou fournit les informations demandées. Le **système** exécute les contrôles automatiques, enregistre les événements et envoie les notifications. L'**administrateur** (rôle `ADMIN`) examine les tâches qui lui sont attribuées et choisit une décision autorisée.

Trois termes sont employés de façon constante dans la suite. Une **demande** est l'opération initiée par le client (virement, signalement de fraude, modification d'une donnée). Une **instance** est l'exécution du processus BPMN associé à cette demande. Une **tâche** est l'étape humaine que cette instance confie à l'administrateur.

La conception progresse du général au particulier : le diagramme de cas d'utilisation situe les acteurs, le diagramme de classes décrit les données de suivi, les descriptions textuelles et le diagramme de séquence précisent le scénario de traitement, puis les modèles BPMN détaillent chacun des trois processus.

6.3.1 Diagramme de cas d'utilisation

Le diagramme de la figure 6.1 détaille les parcours liés aux processus BPMN. Le client utilise l'assistant, effectue ses opérations bancaires, gère ses notifications et soumet ou suit une demande de support. L'administrateur traite les tâches sensibles — validation d'un virement important, d'une modification de données ou d'une alerte de fraude — et supervise les processus et les tâches. La consultation des journaux d'audit et des événements de sécurité fait aussi partie de ses fonctions, mais elle relève de l'administration et non des processus BPMN. Le modèle Ollama soutient l'interaction avec l'assistant, tandis que Flowable orchestre les validations BPMN. L'authentification est une condition préalable aux fonctionnalités protégées.

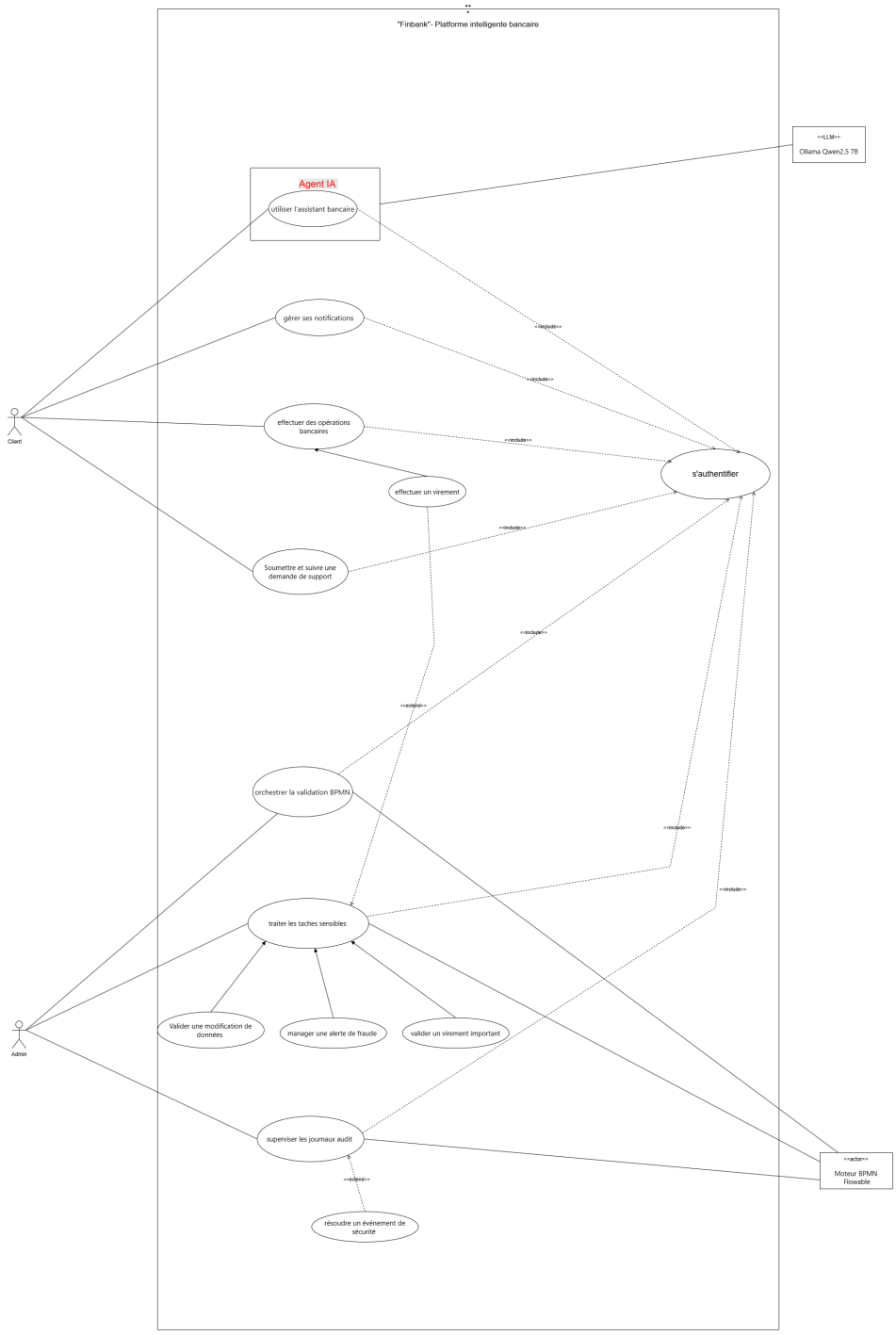


Fig 6.1 : Diagramme de cas d'utilisation du Sprint 4

Cette vue fonctionnelle indique ce que chaque acteur peut faire. Le modèle de données qui permet de suivre ces demandes est présenté dans la section suivante.

6.3.2 Diagramme de classes

Le suivi d'une demande repose sur quatre éléments : la demande à l'origine du processus (par exemple une transaction ou un ticket), l'instance de processus qui la traite, les tâches de workflow qui en découlent et les notifications qui informent le client ou les administrateurs de l'avancement et du résultat. La transaction conserve notamment son état et la référence du processus qui la traite. Une tâche de workflow est liée à une instance, possède un type et un état. La décision de l'administrateur est transmise au moteur comme variable du processus ; elle n'est pas un attribut de l'entité persistée `WorkflowTask`.

Les entités métier transversales sont présentées dans le diagramme de classes global de la figure 2.2. Le diagramme de la figure 6.2 se concentre sur leur articulation avec les instances et les tâches Flowable : la référence du processus relie la demande à son suivi, tandis que les variables transmises au moteur portent la décision administrative.

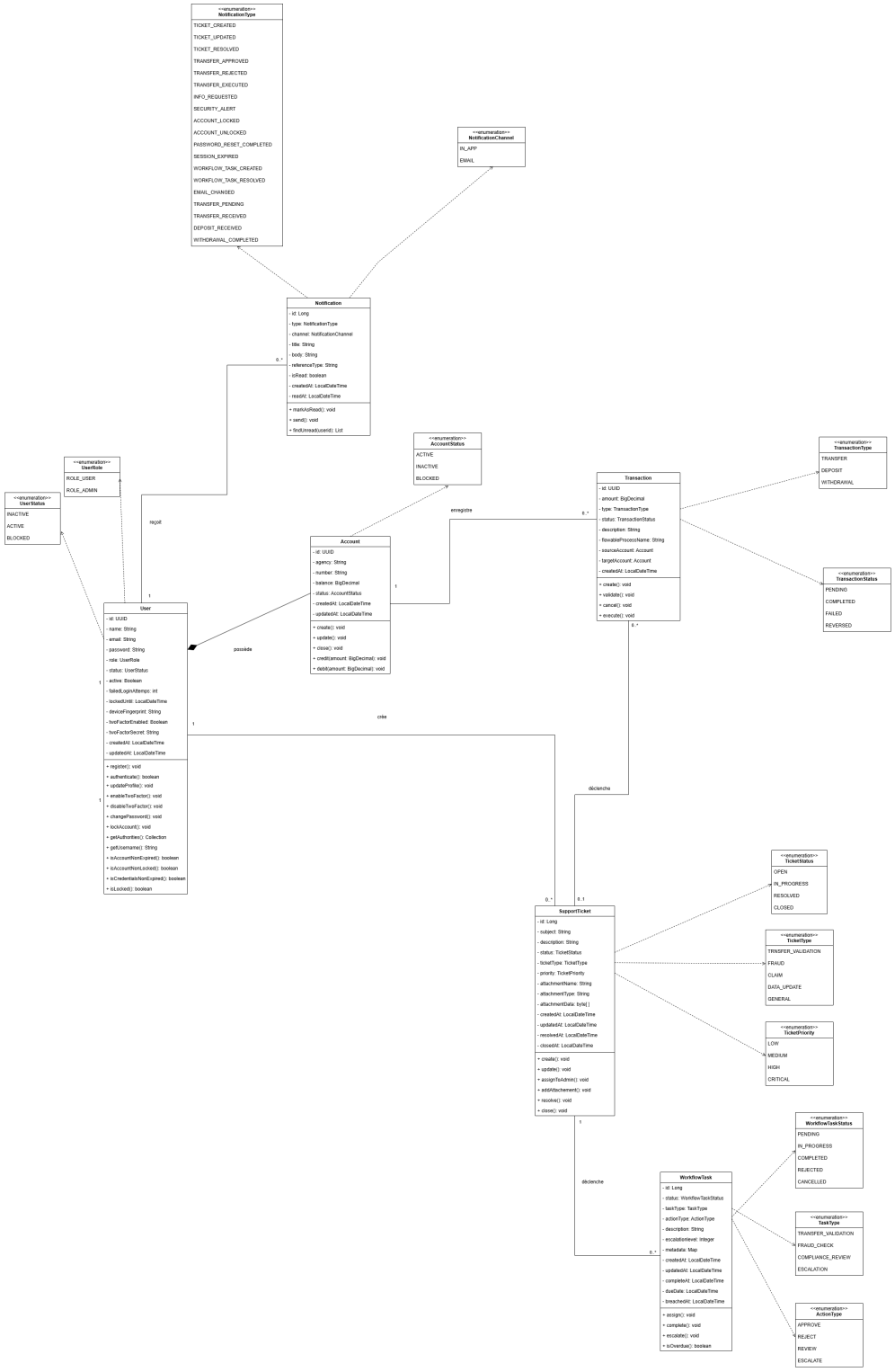


Fig 6.2 : Diagramme de classes des demandes et workflows du Sprint 4

Le diagramme montre les liens entre le compte et ses transactions, l'utilisateur et ses tickets ou notifications, ainsi que les tâches de workflow associées aux demandes. Les énumérations précisent les états des comptes, transactions, tickets et tâches, de même que les types de demande et d'action.

Cette représentation distingue ainsi les données métier du suivi d'exécution assuré par Flowable. Ces entités sont mises en jeu dans les scénarios décrits ci-après.

6.3.3 Description textuelle des cas d'utilisation

Les diagrammes précédents donnent une vue synthétique des interactions. Les tableaux 6.2 à 6.5 précisent quatre scénarios selon un même modèle. Le premier décrit le travail commun de l'administrateur face à une tâche ; les trois suivants correspondent aux trois familles de processus. Les chemins alternatifs et les délais de chaque processus sont détaillés dans les modèles BPMN (section 6.3.5).

6.3.3.1 Traiter une tâche de workflow

Ce scénario décrit le travail de l'administrateur lorsqu'une tâche humaine est créée par un processus BPMN. Il constitue le déroulement commun aux validations présentées dans les scénarios suivants.

Tab 6.2 : Description textuelle – Traiter une tâche de workflow

Élément	**Description**
Titre	Traiter une tâche de workflow
Acteur	Administrateur (`ADMIN`)
But	Examiner une demande sensible et décider.
Résumé	L'administrateur examine le contexte d'une tâche active et choisit une décision autorisée.
Précondition	Une instance est active et une tâche est disponible pour le groupe administrateur.
Postcondition	Flowable reçoit la décision, met à jour l'instance et les acteurs concernés sont notifiés.
Scénario principal	1. L'administrateur ouvre la liste et sélectionne une tâche. 2. Il examine le contexte et choisit une décision autorisée. 3. Le système vérifie que la tâche est encore active. 4. Flowable complète la tâche et actualise l'instance. 5. L'interface affiche le nouvel état et notifie les acteurs concernés.
Scénarios alternatifs	Tâche absente, déjà terminée, expirée ou décision invalide : l'opération est refusée et l'état reste inchangé.

6.3.3.2 Valider un virement important

Ce scénario concerne uniquement les virements qui dépassent le seuil de validation prévu dans le processus. Avant la décision humaine, le système vérifie le compte, le solde disponible et le bénéficiaire.

Tab 6.3 : Description textuelle – Valider un virement important

Élément	**Description**
Titre	Valider un virement important
Acteurs	Client, système, administrateur (`ADMIN`)
But	Faire valider tout virement dépassant le seuil de contrôle.
Résumé	Le système contrôle le compte, le solde et le bénéficiaire. L'administrateur approuve ou rejette ; seul un virement approuvé est exécuté.
Précondition	Le client est authentifié et le montant dépasse le seuil prévu.
Postcondition	Le résultat est tracé et communiqué au client : virement exécuté, demande rejetée, erreur, expiration ou escalade.
Scénario principal	1. Le client soumet un virement au-dessus du seuil. 2. Le système vérifie le compte, le solde et le bénéficiaire. 3. Flowable crée une tâche pour l'administrateur. 4. L'administrateur examine la demande et l'approuve. 5. Le système exécute le virement, notifie le client et journalise l'événement.
Scénarios alternatifs	Compte bloqué, solde insuffisant ou bénéficiaire invalide : demande rejetée. Rejet administratif : aucun virement n'est exécuté. Erreur, expiration et escalade suivent le modèle BPMN.

6.3.3.3 Traiter une alerte de fraude

Ce scénario présente le traitement d'une alerte nécessitant une analyse administrative. Selon les éléments disponibles, l'administrateur choisit une action et le processus informe ensuite le client du résultat ou de la suite attendue.

Tab 6.4 : Description textuelle – Traiter une alerte de fraude

Élément	**Description**
Titre	Traiter une alerte de fraude
Acteurs	Client, système, administrateur (`ADMIN`)
But	Analyser une alerte et décider de l'action appropriée.
Résumé	Après un contrôle du compte, l'administrateur peut bloquer la carte, demander des justificatifs ou classer l'alerte comme faux positif.
Précondition	Une fraude est signalée ou une activité suspecte est détectée.
Postcondition	L'action retenue est tracée et le client est informé.
Scénario principal	1. Le client signale une fraude ; le système ouvre une demande. 2. Le système vérifie le compte et crée une tâche si nécessaire. 3. L'administrateur examine le dossier et choisit une action. 4. Le système applique le blocage demandé, trace le résultat et notifie le client.
Scénarios alternatifs	Compte non actif : notification dédiée. Faux positif : alerte clôturée. Justificatifs, échec, expiration et escalade suivent les branches prévues dans le modèle BPMN.

6.3.3.4 Valider une modification de données sensibles

Ce scénario porte sur l'examen d'une demande de modification sensible et des justificatifs fournis par le client. La décision est enregistrée dans le workflow ; le processus décrit ici n'applique pas directement la modification au profil.

Tab 6.5 : Description textuelle – Valider une modification de données sensibles

Élément	**Description**
Titre	Valider une modification de données sensibles
Acteurs	Client, administrateur (`ADMIN`), système
But	Examiner une modification sensible avant son approbation.
Résumé	Le client transmet un justificatif. L'administrateur approuve, rejette ou demande des précisions ; la décision est tracée.
Précondition	Le client authentifié demande la modification d'une donnée sensible et transmet un justificatif.
Postcondition	La décision est tracée et communiquée. Le processus ne modifie pas directement le profil.
Scénario principal	1. Le client soumet sa demande et le justificatif. 2. Le système crée un ticket et une tâche de vérification. 3. L'administrateur examine le dossier et approuve la demande. 4. Le système enregistre la décision et informe le client.
Scénarios alternatifs	Rejet : le profil reste inchangé. Informations complémentaires : retour au client dans la limite prévue. Délai dépassé : expiration ou escalade selon le modèle BPMN.

6.3.4 Diagramme de séquence

Le diagramme de la figure 6.3 illustre le scénario du tableau 6.2, commun aux trois processus. L'administrateur ouvre l'espace de supervision et sélectionne une tâche. Les contrôles sont effectués avant la complétion : le système vérifie que la tâche est encore active, puis Flowable la complète et actualise l'instance. Le résultat est enfin notifié aux acteurs concernés.

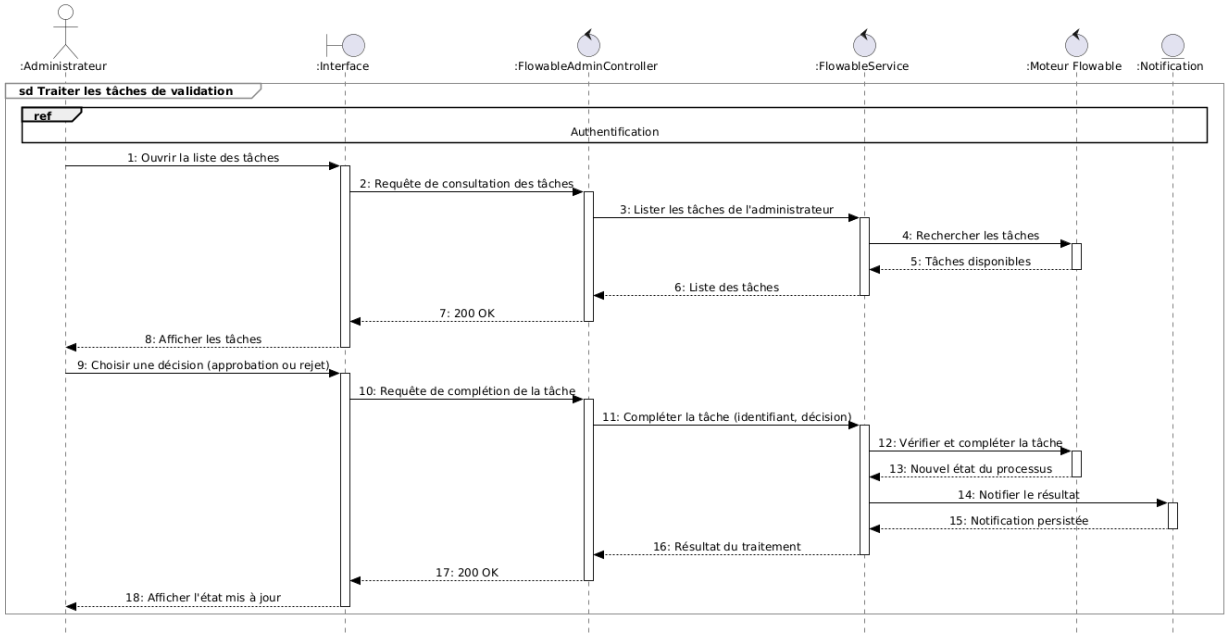


Fig 6.3 : Supervision et complétion d'une tâche Flowable

Ce scénario commun étant précisé, il reste à décrire ce qui distingue chaque processus : ses contrôles, ses décisions, ses délais et ses issues. C'est l'objet des modèles BPMN.

6.3.5 Modèles BPMN des processus

Les trois processus sont modélisés dans des diagrammes BPMN distincts afin de conserver une lecture claire des lanes, des tâches, des passerelles, des événements de délai et des chemins de sortie. Ils partagent la même structure : des contrôles automatiques, une tâche humaine attribuée au groupe administrateur, puis une issue notifiée et tracée. Ils partagent aussi la gestion des délais : l'administrateur dispose de quarante-huit heures pour examiner une tâche, avec escalade vers un superviseur à l'expiration, et le client de vingt-quatre heures pour répondre lorsqu'une information complémentaire lui est demandée.

6.3.5.1 Validation d'un virement important

Le processus de la figure 6.4 est déclenché pour une opération dépassant le seuil de contrôle défini par la plateforme. Le compte est d'abord vérifié, puis le solde et le bénéficiaire sont contrôlés. Un compte bloqué, un solde insuffisant ou un bénéficiaire invalide conduit à un rejet accompagné d'une notification.

Lorsque les contrôles sont positifs, une tâche de validation est attribuée au groupe administrateur,

qui peut approuver ou rejeter la demande. En cas d'approbation, le virement est exécuté, le client est notifié et l'événement est ajouté à l'audit. Une erreur d'exécution peut déclencher une nouvelle tentative ; si celle-ci échoue, le processus se termine par une erreur technique. Une expiration du délai d'examen peut conduire à une escalade vers un superviseur.

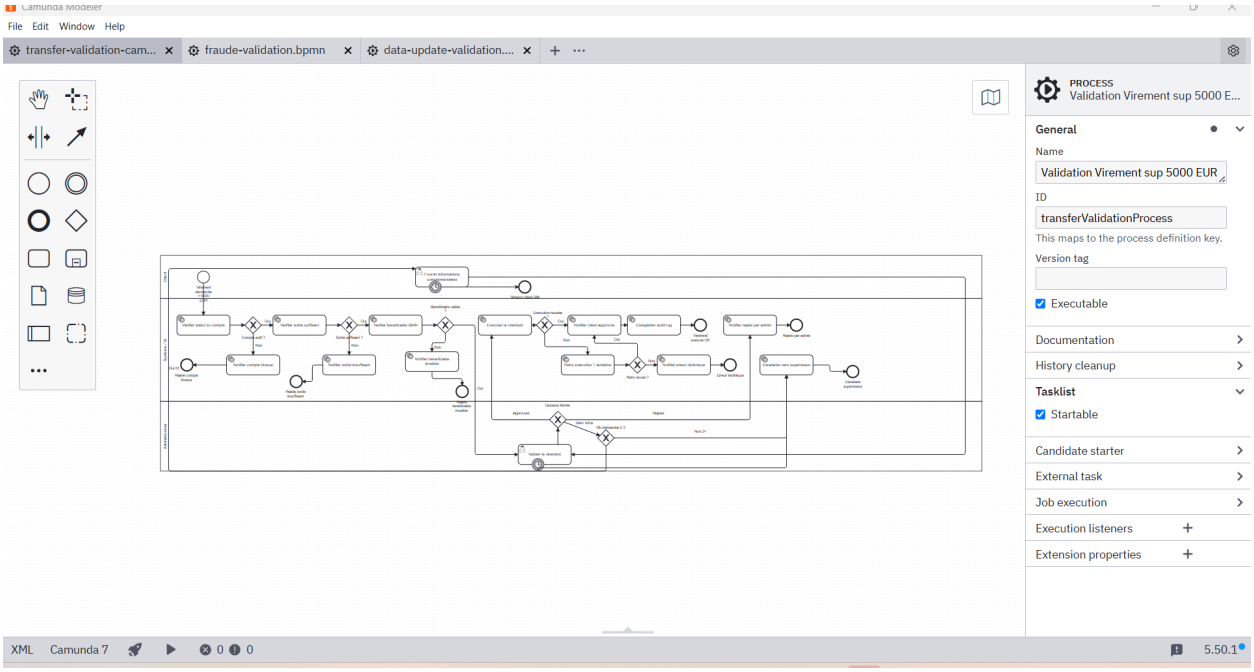


Fig 6.4 : Processus BPMN de validation d'un virement important

6.3.5.2 Traitement d'une alerte de fraude

Lorsqu'une fraude est signalée, le processus de la figure 6.5 vérifie d'abord le statut du compte. Un compte non actif est traité par une notification dédiée. Pour un compte actif, une tâche d'analyse est proposée au groupe administrateur.

L'administrateur peut choisir de bloquer la carte, de contacter le client pour demander des justificatifs ou de considérer l'alerte comme un faux positif. Le blocage est vérifié avant la notification de réussite ; en cas d'échec, une notification d'erreur est produite. Si le client doit fournir des justificatifs, le processus vérifie le nombre d'informations reçues ; au-delà de la limite prévue, la demande est escaladée. Les délais de vingt-quatre heures pour le client et de quarante-huit heures pour l'analyse administrative sont gérés par des événements de temporisation.

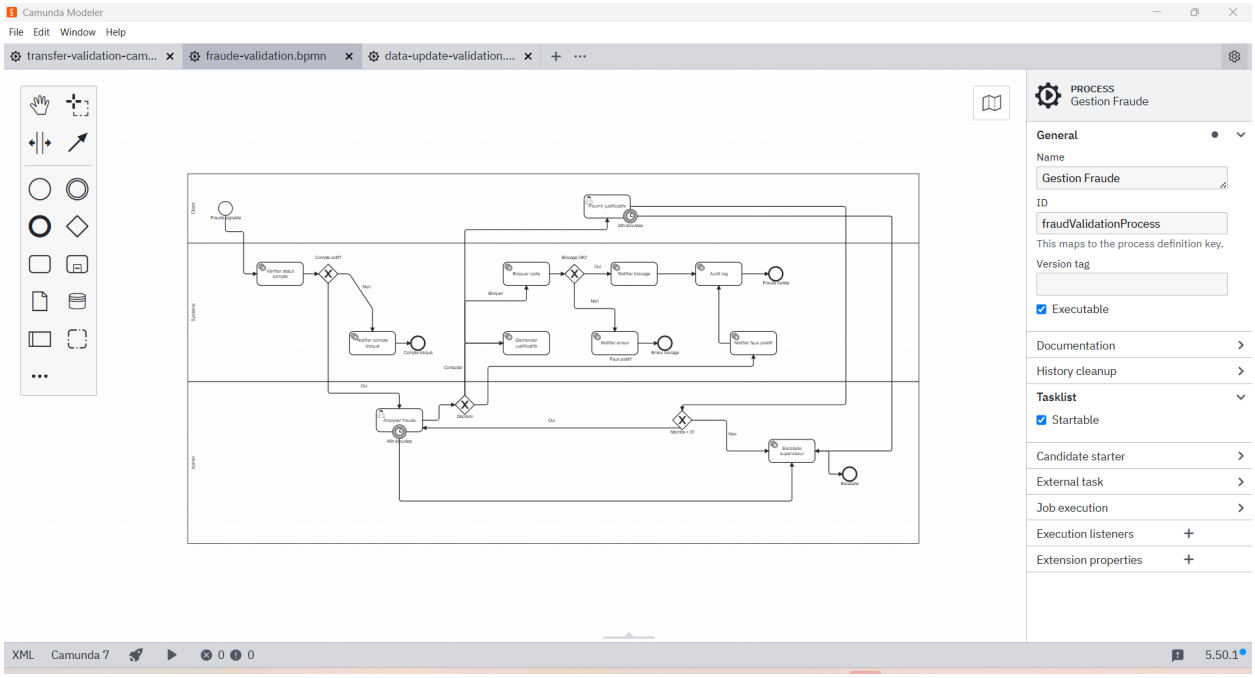


Fig 6.5 : Processus BPMN de traitement d'une alerte de fraude

6.3.5.3 Validation d'une modification de données sensibles

Le processus de la figure 6.6 protège les modifications qui ne doivent pas être appliquées immédiatement à partir d'une simple conversation. Le client fournit une pièce ou un justificatif, puis l'administrateur examine la demande et les éléments transmis.

Trois décisions sont possibles : approuver la demande, la rejeter ou demander des informations complémentaires. Une approbation déclenche la notification et l'enregistrement dans l'audit ; le processus BPMN ne modifie pas lui-même la donnée du profil. Un rejet informe le client sans modifier la donnée. Une demande d'informations complémentaires renvoie vers le client tant que le nombre maximal de demandes n'est pas atteint. Les délais d'attente et l'escalade empêchent qu'une demande sensible reste indéfiniment ouverte.

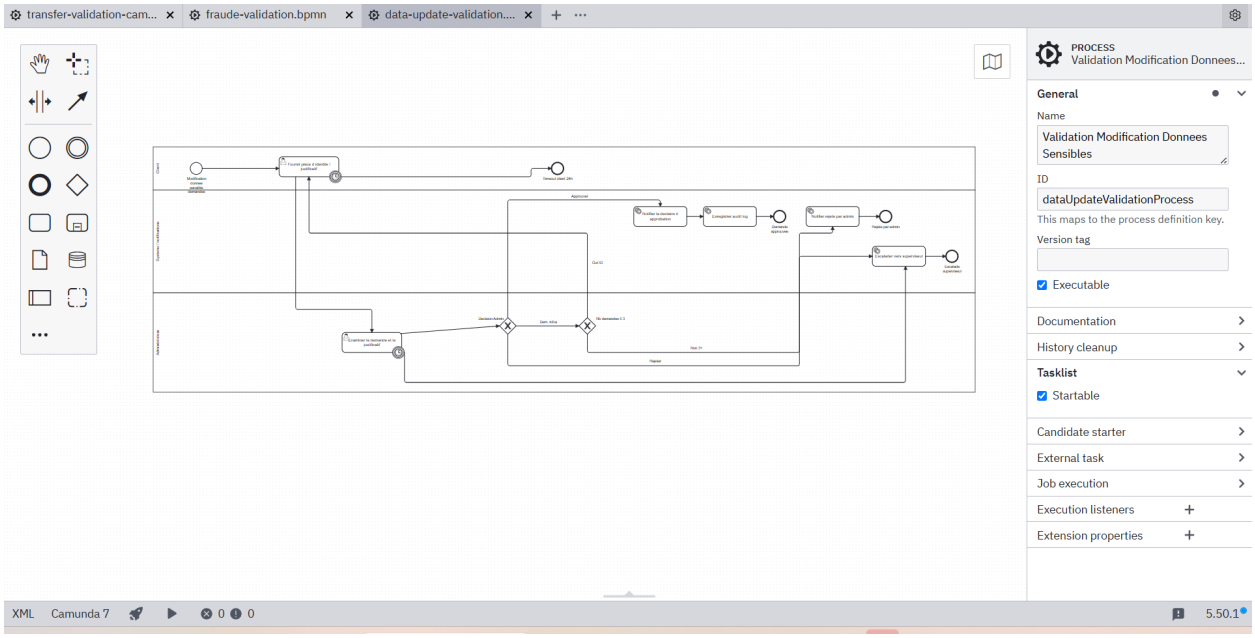


Fig 6.6 : Processus BPMN de validation d'une modification sensible

Les modèles étant définis, la section suivante présente leur mise en œuvre dans la plateforme.

6.4 Réalisation

Cette section présente la mise en œuvre des modèles de la section précédente. Elle part du moteur lui-même (intégration de Flowable), puis suit les trois parcours de bout en bout — côté client puis côté administrateur — avant de décrire l'espace de supervision et la gestion des états. Les interfaces React et TypeScript rendent visibles les résultats des processus sans exposer leur implémentation interne ; chaque série de captures est commentée en renvoyant au modèle BPMN correspondant.

6.4.1 Intégration de Flowable

Les définitions BPMN modélisées à la section précédente sont placées dans les ressources du backend, puis déployées comme processus exécutables. Flowable fournit les services nécessaires au démarrage d'une instance, à la recherche des tâches et à la complétion d'une tâche. Il permet également de consulter les définitions actives, les variables du processus, les instances historiques et la trace des activités.

Le service de workflow vérifie la disponibilité du moteur avant d'utiliser ses services (figure 6.7).

Lorsqu'un processus est démarré, son identifiant est associé à la demande concernée et les administrateurs sont informés de l'apparition du workflow. Si l'instance n'est plus active, les variables et la trace peuvent être récupérées à partir de l'historique : une demande terminée ne disparaît donc pas de l'espace de supervision.

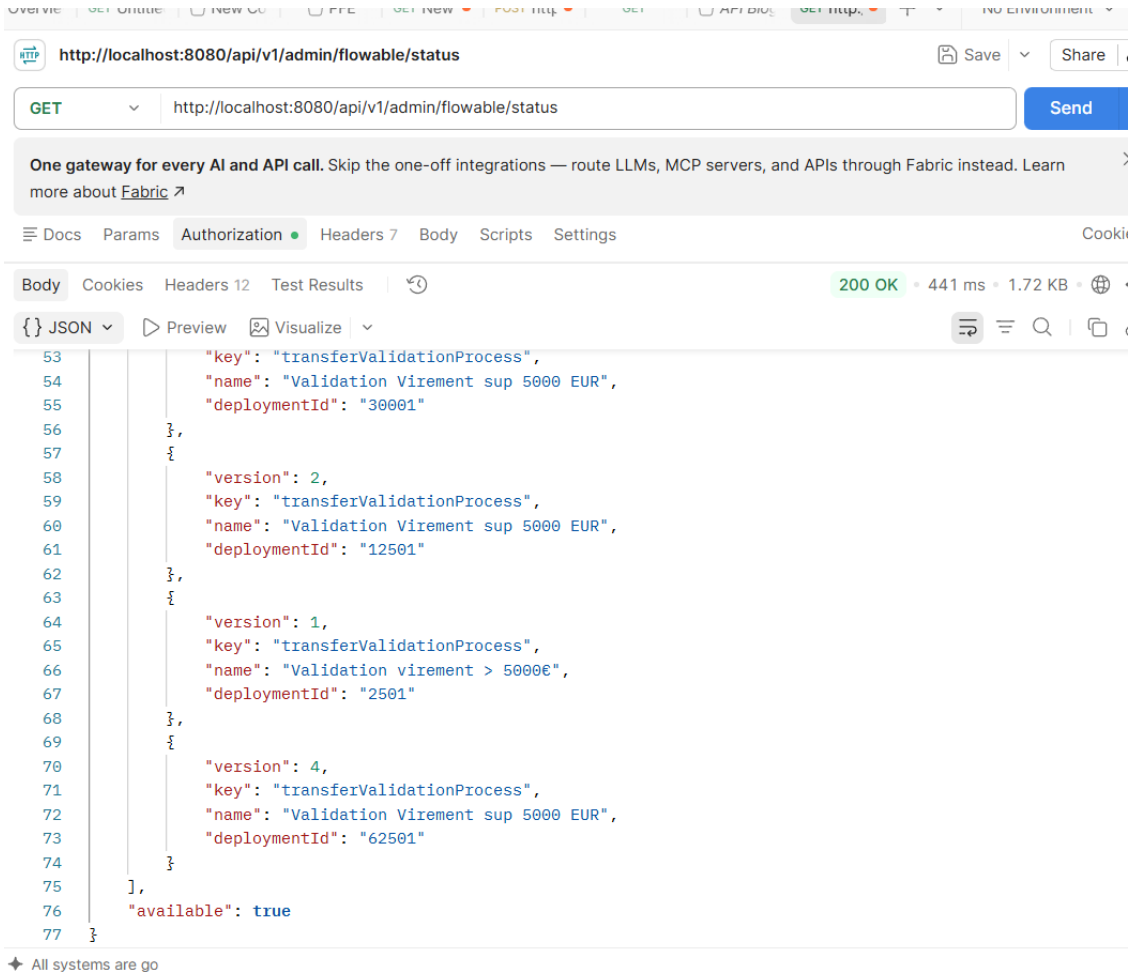
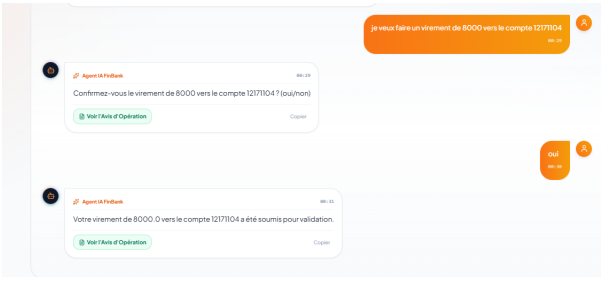


Fig 6.7 : État de disponibilité du moteur Flowable intégré

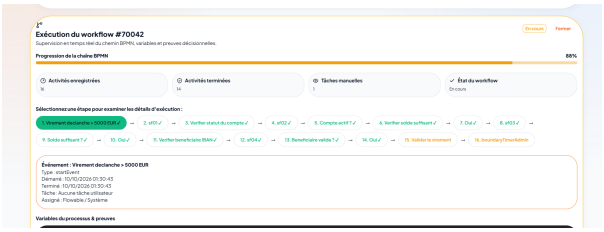
Le moteur étant opérationnel, les trois parcours peuvent être déroulés, en commençant par la validation d'un virement important.

6.4.2 Parcours : validation d'un virement important

Ce parcours met en œuvre le processus de la figure 6.4. Le client soumet d'abord sa demande ; côté administrateur, l'instance apparaît alors en attente, avec les actions proposées (figure 6.8).



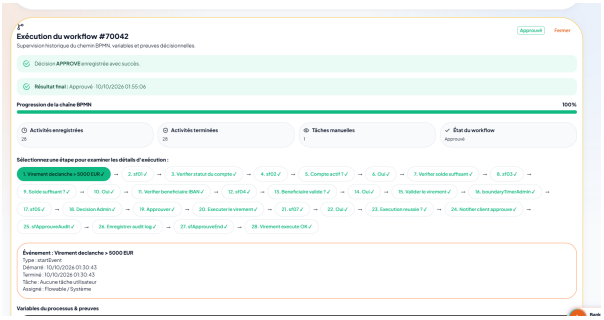
Demande de virement côté client



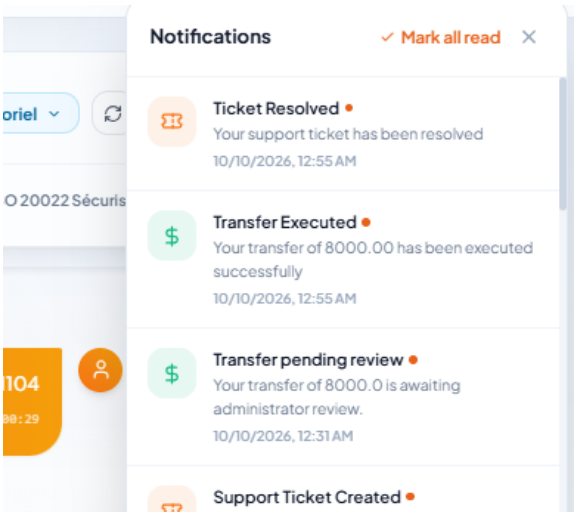
Instance en attente de décision

Fig 6.8 : Soumission et mise en attente d'un virement important

Après approbation, l'instance passe à l'état terminé et la plateforme notifie le client. La comparaison des deux vues de la figure 6.9 permet de vérifier le changement d'état et le retour d'information envoyé à l'utilisateur.



Instance après approbation



Notification reçue par le client

Fig 6.9 : Résolution et notification du virement validé

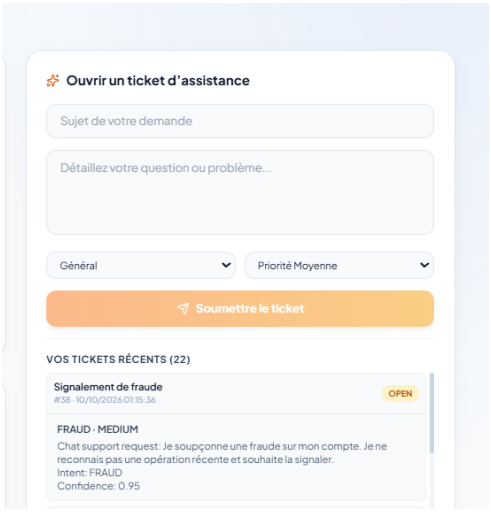
Le deuxième parcours suit la même logique, mais la décision de l'administrateur y est plus variée.

6.4.3 Parcours : traitement d'une alerte de fraude

Ce parcours met en œuvre le processus de la figure 6.5. Le client signale la fraude, ce qui crée une demande de traitement (figure 6.10).



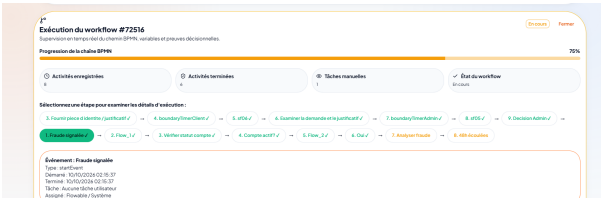
Signalement de fraude



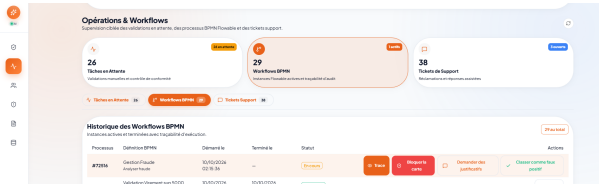
Création de la demande de traitement

Fig 6.10 : Signalement d'une fraude et création de la demande

L'instance attend ensuite l'analyse de l'administrateur, qui choisit une décision parmi celles prévues par le processus (figure 6.11).



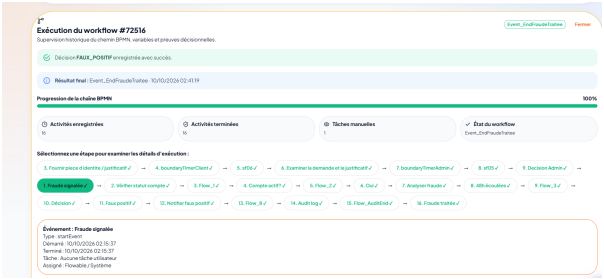
Instance avant décision



Décision de l'administrateur

Fig 6.11 : Analyse et décision sur une alerte de fraude

La figure 6.12 montre l'instance une fois la décision enregistrée. Le contexte de sécurité de l'alerte est consultable dans l'espace d'administration (section 6.4.5).



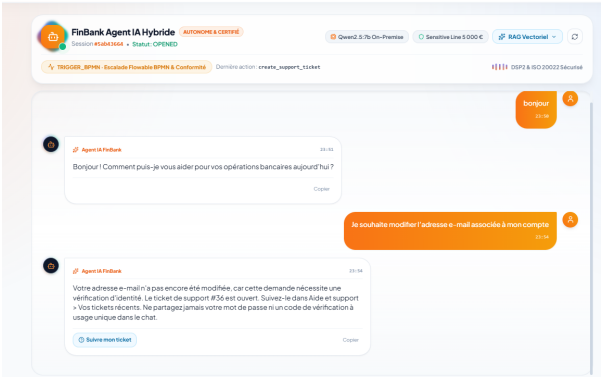
Instance après décision

Fig 6.12 : Résultat du traitement d'une alerte de fraude

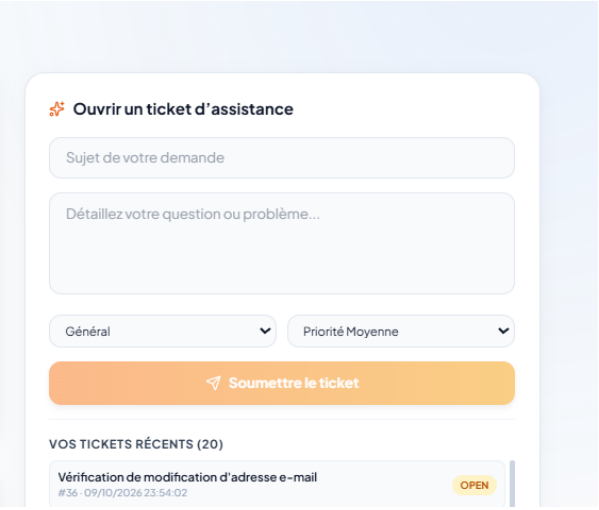
Le troisième parcours est le plus riche en échanges avec le client, puisqu'il suppose la transmission d'un justificatif.

6.4.4 Parcours : modification de données sensibles

Ce parcours met en œuvre le processus de la figure 6.6. Le client ouvre une demande de modification, ce qui crée un ticket de traitement (figure 6.13).



Demande de modification côté client



Ticket de traitement de la demande

Fig 6.13 : Ouverture d'une demande de modification sensible

Il transmet ensuite son justificatif (figure 6.14).



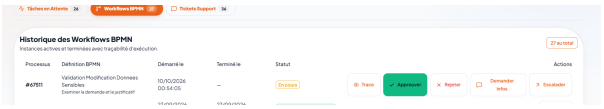
Téléversement du justificatif



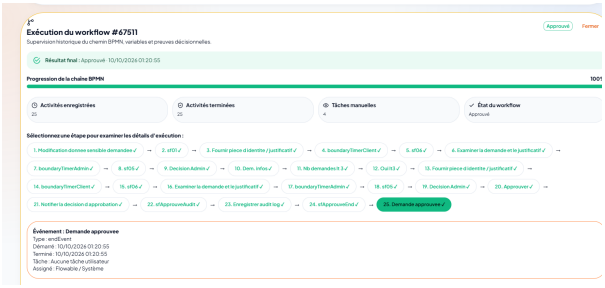
Confirmation de transmission

Fig 6.14 : Transmission du justificatif par le client

L'administrateur reçoit le justificatif, l'examine, puis approuve la demande dans Flowable (figure 6.15). Conformément au modèle, cette approbation enregistre la décision et la trace, mais n'applique pas elle-même la modification au profil.



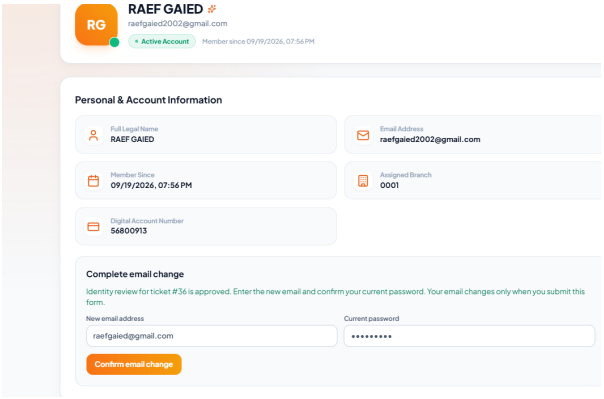
Justificatif reçu par l'administrateur



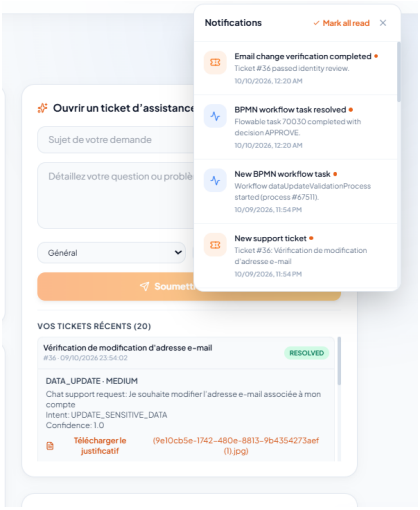
Demande approuvée par Flowable

Fig 6.15 : Examen et approbation de la demande sensible

La figure 6.16 présente enfin le parcours après résolution et la notification reçue par le client.



Parcours après résolution



Notification de résolution

Fig 6.16 : Résolution de la demande de modification et retour au client

Les trois parcours étant illustrés, la section suivante adopte le point de vue transversal de l'administrateur, qui supervise l'ensemble des processus.

6.4.5 Espace administrateur : supervision des processus

L'espace administrateur regroupe les tâches humaines, les instances de processus et leur état. L'administrateur consulte les tâches qui lui sont attribuées (figure 6.17), ouvre le détail d'une instance, consulte les variables utiles à la décision et visualise la trace des activités, comme dans les parcours précédents. Après son choix, le système vérifie que la tâche existe encore et complète le processus avec les variables correspondantes.

La supervision distingue les tâches actives des processus terminés. Elle permet ainsi de suivre une demande approuvée, rejetée, expirée, escaladée ou arrêtée à la suite d'une erreur technique. Les événements importants sont associés aux notifications et aux journaux d'audit.

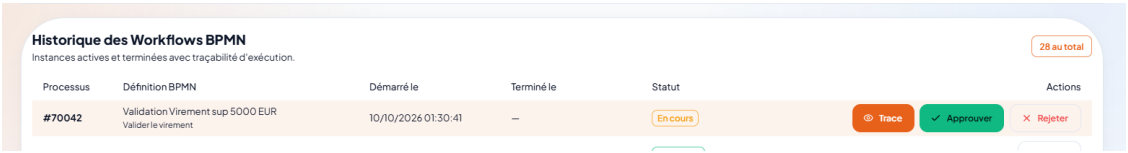


Fig 6.17 : Liste des tâches de validation destinées à l'administrateur

Le tableau de bord donne une vue synthétique de cette activité. Il regroupe des indicateurs concernant les tickets ouverts, les sessions IA actives, les tâches Flowable et les utilisateurs connectés. La capture 6.18 illustre cette vue transversale ; les valeurs affichées correspondent à l'instant de la capture et ne doivent pas être interprétées comme des statistiques calculées sur une période.

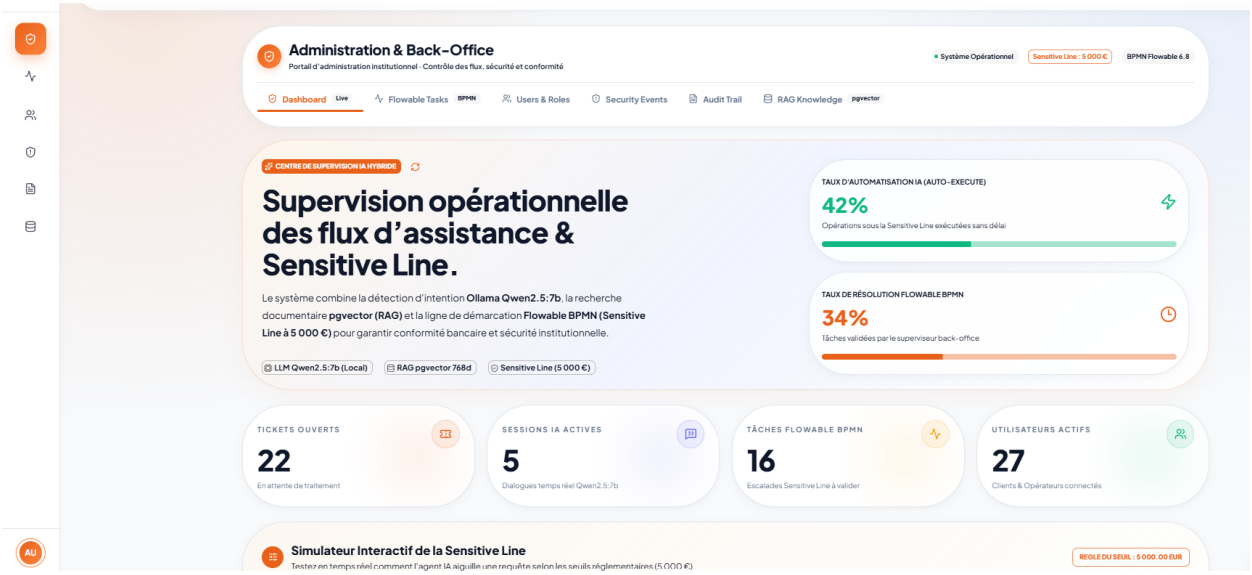


Fig 6.18 : Tableau de bord administratif et indicateurs de supervision

Le panneau de notifications complète ce tableau de bord en regroupant les événements qui peuvent nécessiter un suivi : nouveaux tickets de support, tickets résolus, démarrage de processus et tâches BPMN terminées. Comme le montre la capture 6.19, les éléments à traiter sont signalés et certaines tâches proposent une action directe depuis l'interface. Cette vue facilite le repérage des événements, tandis que l'examen du contexte et la décision restent effectués dans l'espace de traitement des tâches.

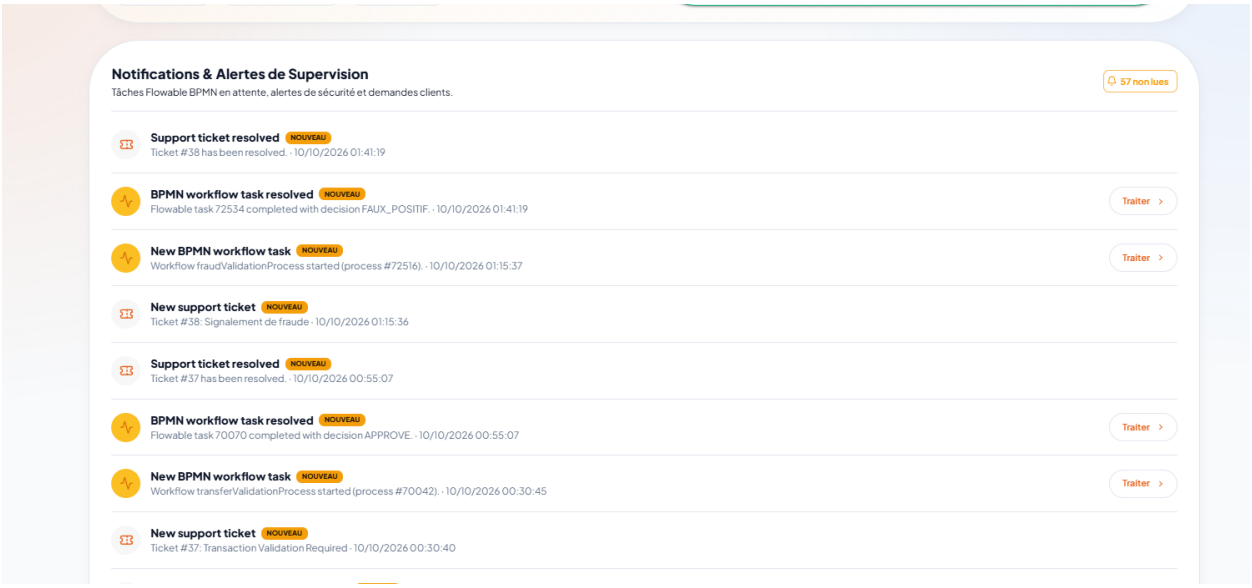


Fig 6.19 : Notifications administratives relatives aux tickets et aux tâches BPMN

Enfin, la traçabilité s'appuie sur deux vues d'administration distinctes des processus BPMN : le journal d'audit, qui conserve les décisions administratives (figure 6.20), et la page des événements de sécurité, qui fournit le contexte de sécurité d'une alerte (figure 6.21).

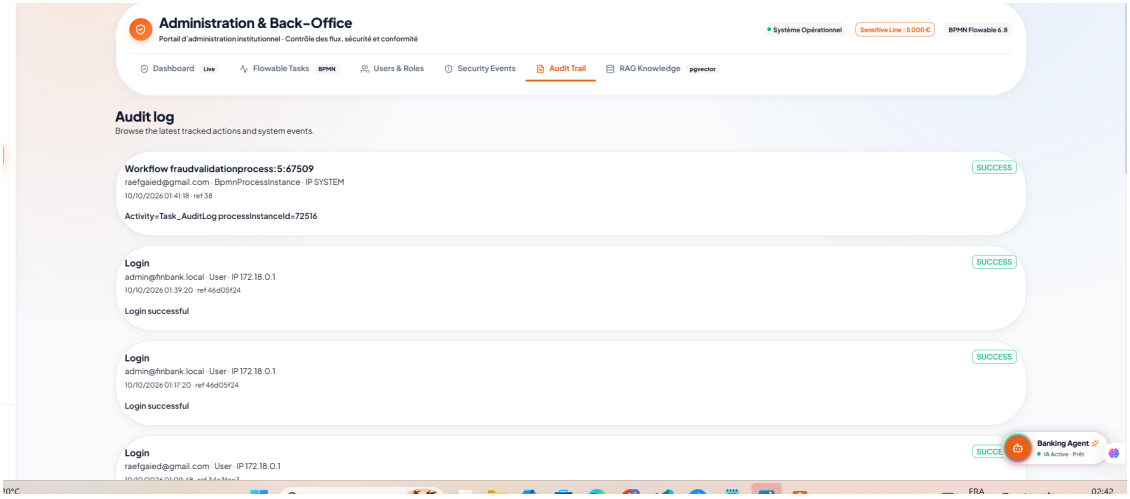


Fig 6.20 : Trace d'audit des décisions administratives

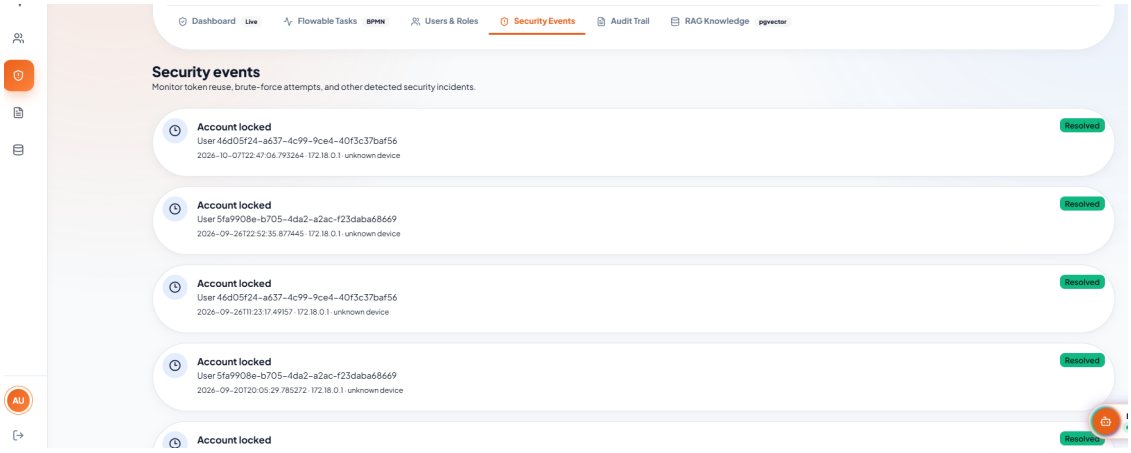


Fig 6.21 : Vue administrateur des événements de sécurité

La supervision rend l'état des processus visible à l'administrateur ; il reste à montrer comment ce même état est restitué au client et comment l'interface gère les situations d'échec.

6.4.6 Suivi des demandes et gestion des états

Côté client, les écrans permettent de consulter les opérations, les notifications, les tickets d'aide et l'état d'une demande. La page d'aide et support présente les demandes soumises et leur évolution (figure 6.22).

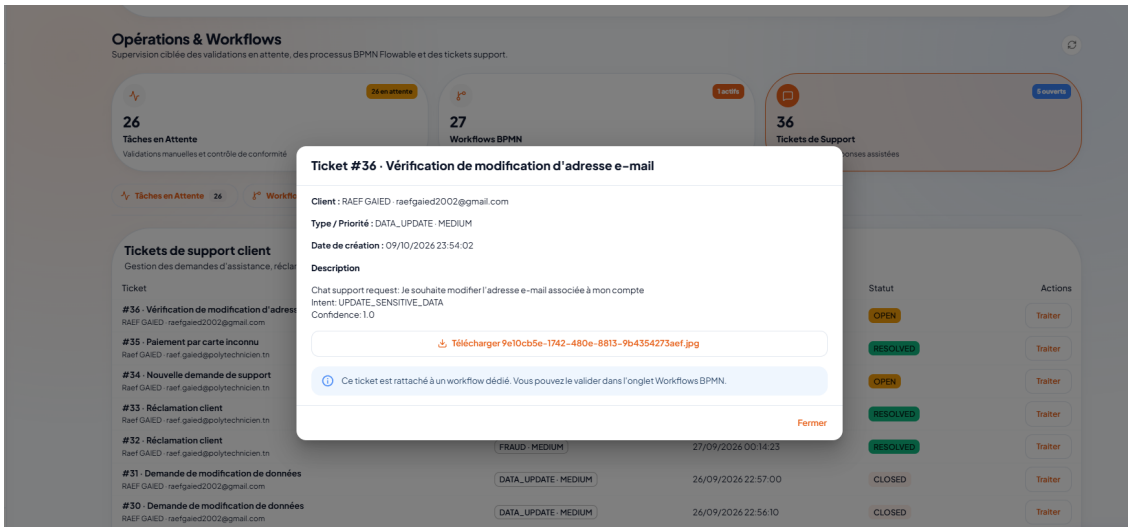


Fig 6.22 : Suivi d'une demande depuis l'espace d'aide et support

Une demande peut être en attente d'informations, en attente d'une décision administrative, approuvée, rejetée, expirée, escaladée ou terminée avec une erreur. Les états de chargement, de réussite, de refus et d'erreur sont présentés à l'utilisateur avec un message compréhensible. L'interface

ne doit pas présenter une opération comme réussie lorsque le moteur n'a pas pu compléter la tâche ; les erreurs techniques sont journalisées côté serveur et un message fonctionnel est affiché au client ou à l'administrateur.

Les délais, les tâches déjà traitées, les décisions non autorisées et les processus indisponibles sont ainsi traités comme des états explicites. Cette gestion garantit la cohérence entre l'état affiché, la tâche Flowable et la demande bancaire suivie.

6.5 Conclusion

Ce chapitre a présenté l'étude et la réalisation du Sprint 4 de FinBank. Après le backlog, la conception a été menée du général au particulier : diagramme de cas d'utilisation, diagramme de classes des demandes et des workflows, descriptions textuelles, diagramme de séquence du traitement d'une tâche, puis modèles BPMN des trois processus exécutables.

La réalisation a intégré Flowable au backend et traité, de bout en bout, les validations de virements importants, les alertes de fraude et les demandes de modification de données sensibles. Pour ces dernières, le processus assure l'examen et la décision, sans appliquer lui-même la modification au profil. Les contrôles, délais, notifications et traces d'audit sont pris en compte, et les interfaces React et TypeScript rendent ces résultats accessibles au client comme à l'administrateur, avec une supervision centralisée et une gestion explicite des états et des erreurs.

Le principe énoncé en introduction est ainsi respecté : le modèle de langage propose, Flowable orchestre, les contrôles métier vérifient et l'administrateur décide. Le chapitre suivant est consacré aux tests et à la validation de l'ensemble de la plateforme.

7

Sprint 5 : Tests, validation et préparation de la livraison

7.1 Introduction

Ce chapitre présente le **cinquième et dernier sprint** de la plateforme FinBank. Les sprints précédents ont livré les composants de la solution : un backend sécurisé, les services bancaires, un assistant conversationnel enrichi par une base de connaissances, des workflows BPMN et les interfaces destinées aux clients et aux administrateurs. Ce dernier sprint ne développe plus de nouvelle fonctionnalité : il vise à **vérifier** que ces composants fonctionnent ensemble, à **corriger** les anomalies détectées et à **préparer la livraison** du produit.

La validation porte sur les services backend, les contrôleurs REST, les fonctionnalités de sécurité, les processus Flowable, l'assistant bancaire et les principales interfaces React. Les parcours utilisateur sont également contrôlés afin de s'assurer que les fonctionnalités développées au cours des sprints précédents restent cohérentes une fois intégrées.

Le chapitre suit l'enchaînement naturel d'un sprint de finalisation. La section 7.2 présente les tâches planifiées. La section 7.3 définit la stratégie de test, dont la mise en œuvre est détaillée à la section 7.4. La section 7.5 montre comment ces vérifications sont intégrées au flux de développement et automatisées. La section 7.6 expose les anomalies corrigées et les résultats obtenus. Enfin, la

section 7.7 décrit la conteneurisation et le déploiement, et la section 7.8 traite de la finalisation de la documentation et de la préparation de la soutenance.

7.2 Backlog du Sprint

Le tableau 7.1 présente les tâches planifiées durant le Sprint 5, avec une estimation de leur durée de réalisation.

Tab 7.1 : Backlog du Sprint 5

ID	Tâches	Estimation
S5-T1	Définir la stratégie et les scénarios de test du projet	2 jours
S5-T2	Exécuter les tests unitaires et d'intégration du backend	4 jours
S5-T3	Vérifier les contrôleurs REST, la sécurité et l'authentification	3 jours
S5-T4	Tester les processus BPMN, l'assistant RAG et les services métier	3 jours
S5-T5	Exécuter les tests frontend et les tests des parcours utilisateur	3 jours
S5-T6	Corriger les anomalies et effectuer les régressions nécessaires	4 jours
S5-T7	Finaliser la documentation technique et fonctionnelle	2 jours
S5-T8	Préparer la livraison (conteneurisation, pipeline CI/CD) et la soutenance du projet	2 jours
	Total	**23 jours**

Ces tâches s'enchaînent en trois temps. Les tâches S5-T1 à S5-T5 définissent puis exécutent les tests ; la tâche S5-T6 traite les anomalies qu'ils révèlent ; les tâches S5-T7 et S5-T8 finalisent la documentation et préparent la livraison. Les sections suivantes reprennent cet ordre. L'automatisation des contrôles prolonge les tâches S5-T2 à S5-T5 et prépare la tâche S5-T8.

7.3 Stratégie de test

La première étape du sprint consiste à définir comment le système sera vérifié (tâche S5-T1). La stratégie de test combine plusieurs niveaux complémentaires. Les tests unitaires vérifient le comportement des services et des composants isolés ; les tests d'intégration examinent leur collaboration avec les contrôleurs, la persistance et les processus. Les tests frontend complètent ces vérifications en contrôlant la compilation de l'application et certains parcours de navigation. Les tests E2E qui requièrent un backend configuré sont distingués des tests de fumée autonomes.

Le tableau 7.2 associe à chaque niveau les outils retenus et l'objectif visé.

Tab 7.2 : Niveaux de test et outils utilisés dans le projet

Niveau	**Outil ou commande**	**Objectif**
Backend	Maven Wrapper, JUnit, Spring Test et Mockito	Vérifier les services, contrôleurs, règles métier, sécurité et workflows.
Frontend	TypeScript et Vite (`npm run build`)	Compiler l'application et vérifier les erreurs de typage.
Parcours navigateur	Playwright	Vérifier le chargement des pages et des parcours automatisés ciblés.
API	Collection Postman et Newman	Exécuter des requêtes HTTP et vérifier les réponses de l'API.
Qualité statique	SonarCloud via GitHub Actions	Analyser le backend et suivre les problèmes de qualité du code.

Les commandes de vérification principales sont `mvnw.cmd test` pour le backend et `npm run build` pour le frontend. Les tests de parcours Playwright se lancent avec `npm run test:e2e`. La figure 7.1 résume l'organisation de ces niveaux.

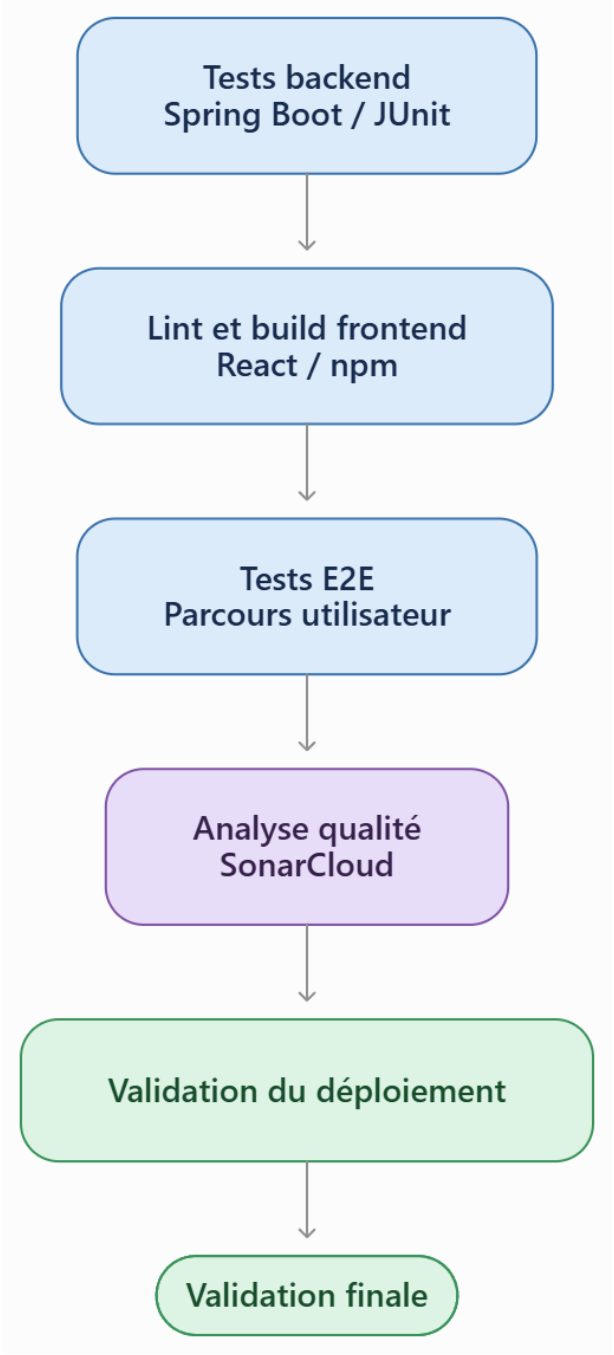


Fig 7.1 : Stratégie de test de la plateforme FinBank

7.4 Mise en œuvre des tests

Cette section présente, niveau par niveau, les vérifications réalisées (tâches S5-T2 à S5-T5). Elle progresse du plus fin au plus global : les services du backend, les contrôleurs et la sécurité, les processus métier et l'assistant, puis les parcours utilisateur dans le navigateur.

7.4.1 Tests unitaires du backend

Les tests unitaires couvrent les services métier et les composants techniques du backend. Ils concernent notamment l'authentification, la validation des transactions, la détection d'intention, la recherche RAG, l'extraction documentaire, les notifications et les traitements des tâches Flowable.

Chaque test vérifie un comportement précis à partir de données contrôlées. Les scénarios nominaux sont complétés par des scénarios d'erreur, par exemple un montant invalide, une décision Flowable interdite ou l'indisponibilité d'un fournisseur d'intelligence artificielle.

7.4.2 Tests des contrôleurs et de la sécurité

Une fois les services vérifiés isolément, les tests des contrôleurs contrôlent leur exposition via l'API REST : accès aux endpoints et forme des réponses. Les fonctionnalités d'inscription, de connexion, de rafraîchissement du jeton, de profil et de double authentification sont testées avec des cas valides et invalides.

Les règles de sécurité sont également vérifiées afin de confirmer qu'un utilisateur ne peut pas accéder aux ressources réservées à l'administrateur et qu'une requête non authentifiée est correctement refusée. Les opérations bancaires sont contrôlées pour s'assurer qu'elles restent liées à l'utilisateur authentifié.

7.4.3 Tests des processus BPMN et de l'assistant

Les tests portent ensuite sur les deux composants les plus spécifiques de FinBank, les workflows et l'assistant (figure 7.2). Les scénarios Flowable vérifient le démarrage des processus, la création des tâches, la validation des décisions et la mise à jour de l'état final. Les scénarios de rejet, d'escalade et de traitement incomplet sont également pris en compte lorsque le processus le prévoit.

Pour l'assistant bancaire, les tests vérifient la détection d'intention, la conservation des messages, la recherche dans la base RAG et la gestion d'une réponse lorsque les fournisseurs d'IA sont indisponibles. L'extraction d'informations depuis un texte ou un document est également vérifiée.

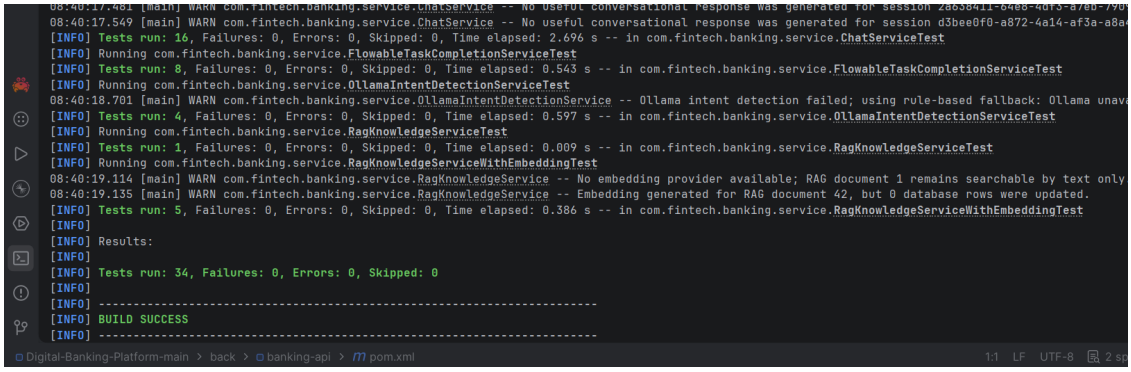


Fig 7.2 : Validation des workflows BPMN et de l'assistant bancaire

7.4.4 Tests frontend et parcours utilisateur

Les tests frontend adoptent enfin le point de vue de l'utilisateur (figure 7.3). Ils vérifient que l'application est accessible et que les pages essentielles se chargent correctement. Les parcours contrôlés comprennent l'affichage de la page de connexion, l'accès au tableau de bord, la navigation vers les opérations bancaires, la conversation avec l'assistant, la création et le suivi d'un ticket de support, ainsi que l'accès aux écrans administrateur.

Ils vérifient également que les routes protégées respectent le rôle de l'utilisateur et que l'interface affiche les états d'erreur lorsqu'une requête échoue.

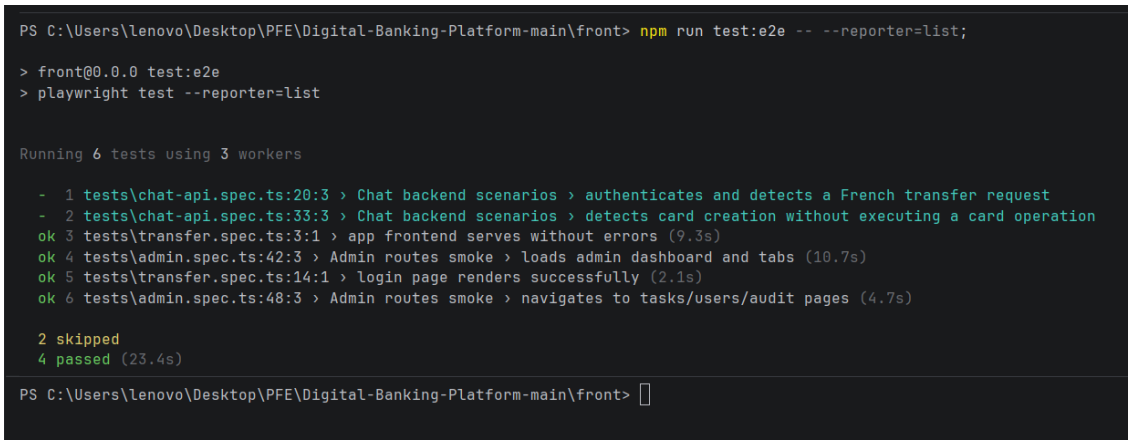


Fig 7.3 : Tests des interfaces et des parcours utilisateur

Pour que ces vérifications restent reproductibles à chaque modification du code, elles sont intégrées au flux de développement et automatisées. C'est l'objet de la section suivante.

7.5 Flux de développement et automatisation

Les tests décrits ci-dessus n'ont de valeur que s'ils sont exécutés régulièrement. Cette section présente la méthode de travail qui les intègre au cycle de développement (Git), leur automatisation (GitHub Actions) et l'analyse statique qui les complète (SonarCloud).

7.5.1 Gestion des versions avec Git

Le code source de FinBank est géré avec **Git**. L'historique du dépôt conserve des changements fonctionnels, des corrections et des évolutions de la documentation sous forme de commits. Cette traçabilité facilite le retour à une version antérieure et permet de relier les modifications aux composants concernés.

Pour limiter les régressions lors de la finalisation, le travail est organisé en changements de portée limitée. Une branche de travail isole une évolution ou une correction de la branche stable. Avant l'intégration, le développeur vérifie les différences, exécute les contrôles adaptés au composant modifié (section 7.3) et consigne le résultat dans un commit explicite. Les modifications liées aux chapitres, aux diagrammes et aux captures sont suivies dans le même dépôt, tout en restant distinctes des changements applicatifs lorsque cela est pertinent.

1. Récupérer la version stable et créer ou sélectionner une branche de travail dédiée.
2. Réaliser une modification ciblée et consulter le diff avec `git diff`.
3. Exécuter les tests backend ou les contrôles frontend concernés.
4. Enregistrer les changements vérifiés dans un commit décrivant leur objectif.
5. Intégrer les changements validés dans la branche stable et contrôler l'état final du dépôt.

Ce flux décrit la méthode de travail retenue pour la validation et la livraison. Il ne suppose pas qu'une revue par pull request a été effectuée pour chaque commit.

7.5.2 Intégration continue avec GitHub Actions

Les workflows GitHub Actions reprennent automatiquement ces contrôles à partir du dépôt Git. Le workflow `ci.yml` exécute les tests backend, la compilation frontend et les tests E2E Playwright lors des pull requests et des poussées sur les branches principales. Lorsque ces contrôles réussissent, il construit ensuite les images Docker de la plateforme et les publie dans GitHub Container Registry (GHCR), comme décrit à la section 7.7.

La figure 7.4 présente l'historique des exécutions. Elle montre aussi le blocage de facturation actuel ; les exécutions antérieures réussies ne prouvent donc pas que le pipeline a été relancé avec succès après les dernières modifications.

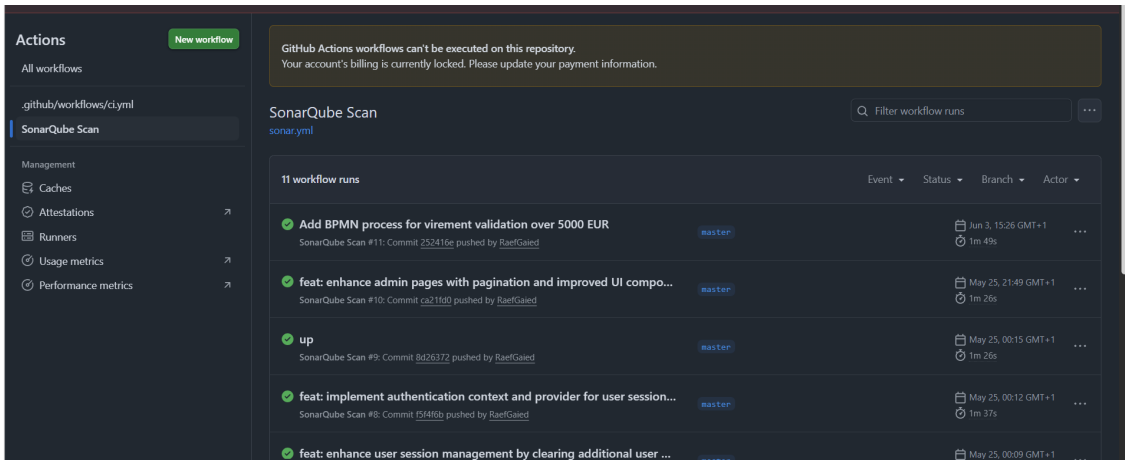


Fig 7.4 : Historique GitHub Actions et blocage actuel de facturation

7.5.3 Analyse statique avec SonarCloud

Les tests vérifient le comportement du code ; l'analyse statique en évalue la qualité. Elle est exécutée séparément par le workflow `sonar.yml` avec SonarCloud, sur le backend. Le jeton `SONAR_TOKEN` doit être défini dans les secrets du dépôt GitHub ; il n'est ni stocké dans le code ni transmis dans les journaux. Cette analyse complète les tests automatisés en signalant les problèmes de qualité et de maintenabilité.

La figure 7.5 montre l'état du tableau de bord au moment de l'analyse.

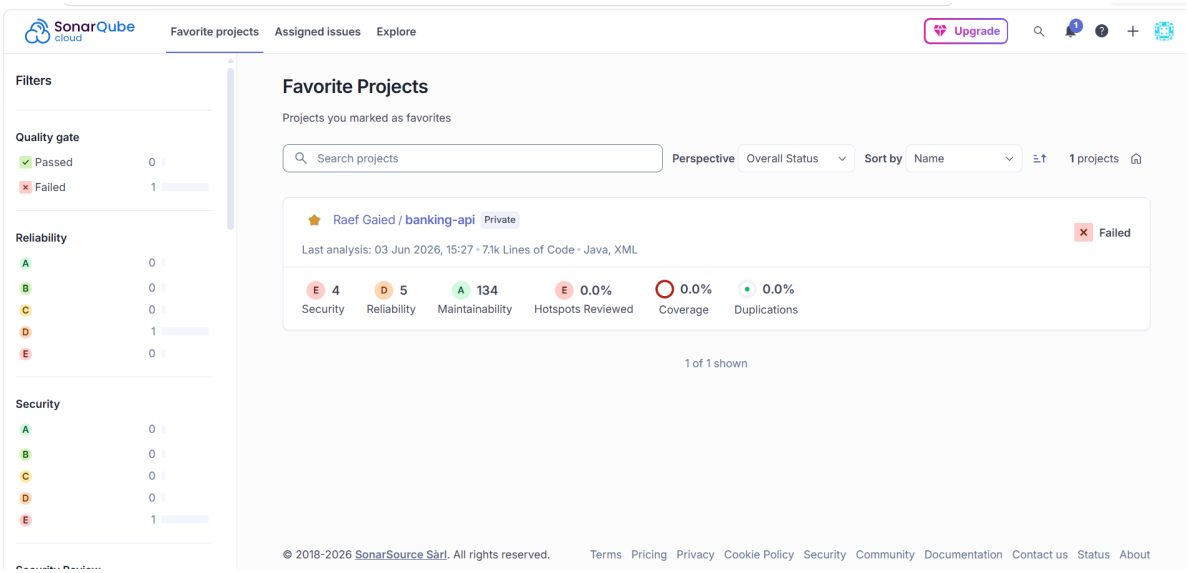


Fig 7.5 : Tableau de bord SonarCloud du projet banking-api

La Quality Gate y est en échec, et des indicateurs de sécurité, de fiabilité, de maintenabilité et de couverture sont affichés. Ces résultats constituent un constat de qualité à examiner ; ils ne doivent pas être interprétés comme une validation complète du projet. Ils alimentent la section suivante, consacrée aux anomalies et aux résultats obtenus.

7.6 Correction des anomalies et résultats de la validation

L'exécution des tests et des contrôles automatisés a mis en évidence plusieurs anomalies. Cette section décrit la démarche de correction (tâche S5-T6), les anomalies traitées, puis les résultats obtenus et leurs limites.

7.6.1 Démarche de correction

Les anomalies sont classées selon leur impact sur la sécurité, les fonctionnalités et l'expérience utilisateur. Les problèmes bloquants sont corrigés en priorité, puis les erreurs de validation, d'affichage et de navigation sont traitées.

Une nouvelle exécution des scénarios concernés est effectuée après chaque correction. Cette démarche permet de vérifier que la correction résout l'anomalie sans introduire de régression dans une fonctionnalité déjà validée.

7.6.2 Anomalies détectées et corrections appliquées

Le tableau 7.3 récapitule les principales anomalies. Elles proviennent de la chaîne d'intégration continue, des tests backend, des tests E2E, du contrôle statique du frontend et de la conteneurisation.

Tab 7.3 : Anomalies détectées et corrections appliquées

Anomalie	**Correction appliquée**
Définitions dupliquées dans le workflow CI	Fusion des étapes en un workflow GitHub Actions unique, avec tests backend, build frontend, E2E et publication conditionnelle vers GHCR.
Recherche textuelle RAG non couverte lorsque l'embedding est indisponible	Correction du mock Mockito de l'argument variable de `JdbcTemplate` et ajout d'une assertion sur le document retrouvé.
Assertions E2E d'administration obsolètes	Alignement des textes attendus sur les libellés réellement affichés par l'interface.
Erreurs ESLint dans le frontend	Correction des règles de hooks, des types et de l'organisation des composants ; le lint et le build frontend ont ensuite été exécutés avec succès.
Image frontend dépendante d'un dossier `dist` préconstruit	Ajout d'une construction multi-étape qui compile le frontend dans Docker avant de copier les fichiers vers Nginx.

7.6.3 Résultats de la validation

Après correction, la suite backend comprend 111 tests réussis. Le frontend passe le lint ESLint et la compilation de production. Les tests E2E Playwright comptent quatre réussites et deux scénarios API ignorés en l'absence de la variable `E2E_API_URL`. La configuration Docker Compose est valide et les images du frontend, du backend, de l'agent IA, de PostgreSQL/pgvector et de pgAdmin ont été construites.

Le tableau 7.4 récapitule les catégories de vérification couvertes par ces résultats.

Tab 7.4 : Catégories de vérification finale

Catégorie	**Éléments vérifiés**
Sécurité	Authentification, autorisations, rôles, 2FA et routes protégées.
Fonctionnel	Comptes, transactions, cartes, investissements, assistant et workflows.
Intégration	Communication frontend/backend, persistence et processus Flowable.
Interface	Navigation, affichage des données, messages d'erreur et responsive design.
Documentation	Endpoints REST, diagrammes, instructions d'installation et utilisation.

7.6.4 Limites de la validation

Pour rester rigoureux, il faut distinguer ce qui a été démontré de ce qui reste à confirmer :

— l'analyse SonarCloud et le déploiement distant n'ont pas été exécutés depuis l'environnement utilisé pour ces vérifications finales : le premier requiert `SONAR_TOKEN`, le second les secrets de déploiement ainsi qu'un hôte accessible ;

— la Quality Gate SonarCloud est en échec (figure 7.5) ;

— deux scénarios E2E de l'API sont ignorés tant que la variable `E2E_API_URL` n'est pas renseignée ;

— l'historique GitHub Actions ne prouve pas la réussite d'une nouvelle exécution du pipeline après les dernières modifications (voir la section 7.7).

7.7 Conteneurisation et déploiement

Une fois la plateforme vérifiée, il reste à la rendre livrable et reproductible sur un autre environnement. Cette section décrit sa conteneurisation, la publication des images puis le déploiement, et enfin ce que les captures permettent d'établir.

7.7.1 Conteneurisation et orchestration des services

Le Dockerfile de l'agent IA définit son environnement d'exécution conteneurisé (figure 7.6), tandis que le fichier Docker Compose décrit les services et leur orchestration (figure 7.7). Docker Compose orchestre notamment PostgreSQL, Ollama, le backend et l'agent IA.

Le composant RAG, intégré au backend Spring Boot, s'appuie sur ces services : PostgreSQL avec pgvector stocke les documents et leurs vecteurs, et Ollama produit les embeddings locaux.

ai-agent > Dockerfile

```
1  FROM python:3.12-slim
2
3  WORKDIR /app
4  COPY requirements.txt ./
5  RUN pip install --no-cache-dir -r requirements.txt
6  COPY . .
7
8  EXPOSE 8001
9  CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8001"]
10
```

Fig 7.6 : Dockerfile de l'agent IA

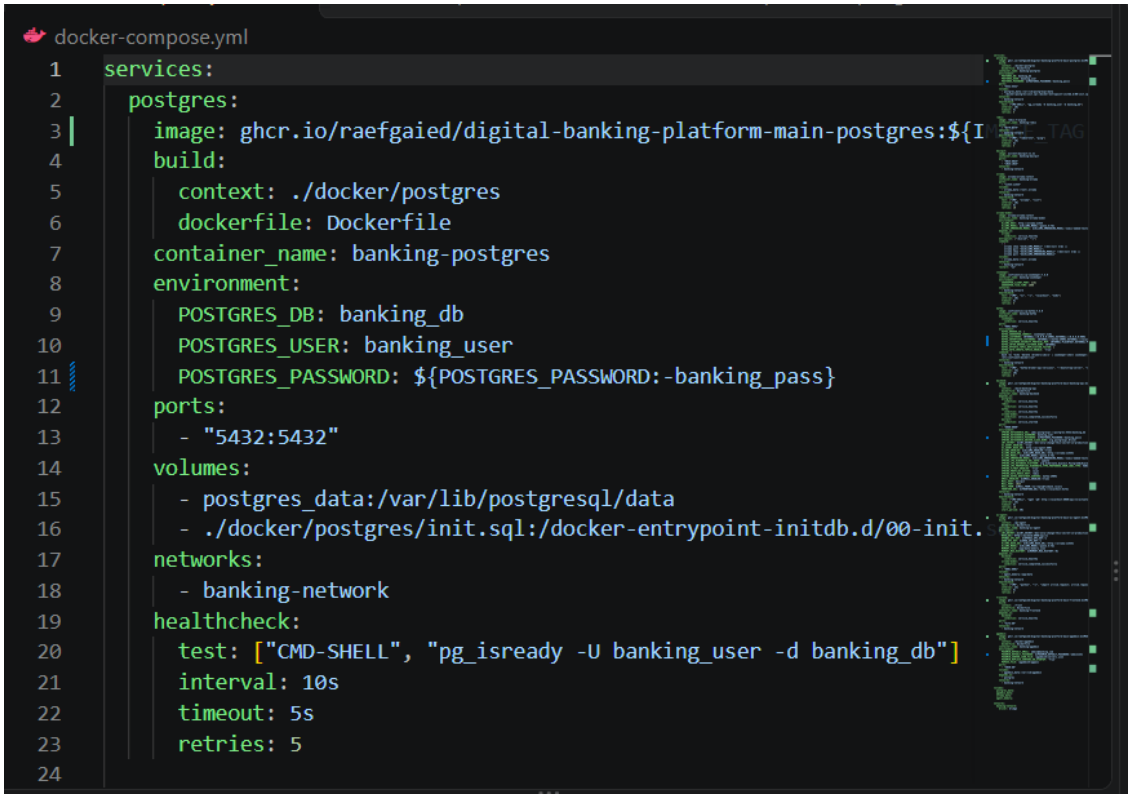


Fig 7.7 : Orchestration des services avec Docker Compose

7.7.2 Publication des images et déploiement distant

Après la réussite des contrôles du workflow `ci.yml` (section 7.5.2), les images Docker du backend, de l'agent IA, du frontend, de PostgreSQL/pgvector et de pgAdmin sont publiées dans GitHub Container Registry (GHCR). Elles sont identifiées par le SHA complet du commit ; l'étiquette `latest` est réservée à la branche principale par défaut.

Le job `deploy` peut ensuite mettre à jour la pile sur l'hôte distant : il synchronise la branche, récupère les images associées au commit et redémarre les services avec Docker Compose, sans reconstruction locale. L'hôte doit disposer du dépôt cloné, de Docker Compose et d'un fichier `.env` contenant la configuration runtime. Avant de démarrer les conteneurs, il vérifie que ce fichier contient un secret JWT et des mots de passe PostgreSQL et pgAdmin suffisamment longs. Ces valeurs restent sur l'hôte et ne sont pas commitées dans le dépôt.

Le déploiement distant est activé en renseignant sept secrets GitHub :

— `DEPLOY_HOST`, `DEPLOY_USER` et `DEPLOY_PATH` pour identifier l'hôte, le compte et le répertoire de déploiement ;

— `DEPLOY_SSH_KEY` et `DEPLOY_SSH_KNOWN_HOSTS` pour la connexion SSH ;

— `DEPLOY_GHCR_USERNAME` et `DEPLOY_GHCR_TOKEN` pour l'accès à GHCR ; ce dernier doit permettre la lecture des images.

Sans ces secrets, la publication des images fonctionne, mais le job de déploiement distant est explicitement ignoré.

7.7.3 Vérification du déploiement

La figure 7.8 montre les conteneurs de la plateforme démarrés localement dans Docker Desktop.

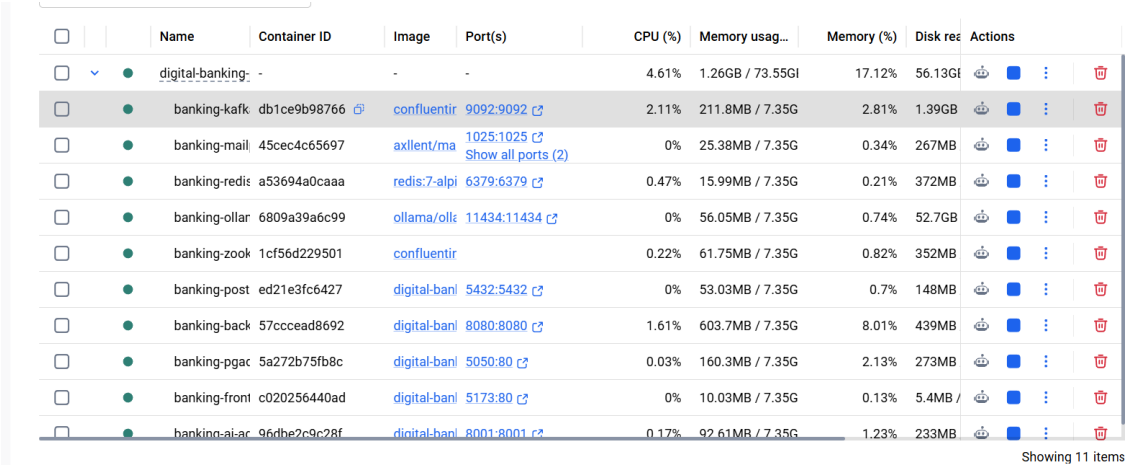


Fig 7.8 : Services FinBank démarrés dans Docker Desktop

7.8 Finalisation et préparation de la soutenance

La dernière étape du sprint (tâches S5-T7 et S5-T8) consiste à rassembler et à vérifier tous les éléments de la livraison. La cohérence des sources, des configurations et de la documentation est contrôlée. Les variables nécessaires à l'exécution sont identifiées, les étapes de démarrage du backend et du frontend sont documentées et les fonctionnalités principales sont regroupées dans une version livrable.

Les diagrammes UML et BPMN sont également révisés afin de rester alignés avec les composants réellement implémentés. Les captures d'écran, les résultats de tests et les éléments nécessaires à la soutenance sont ensuite insérés dans le rapport. La figure 7.9 résume cette préparation.



Fig 7.9 : Préparation de la livraison de la plateforme

7.9 Conclusion

Ce chapitre a présenté le dernier sprint du projet FinBank, qui s'est déroulé en trois temps. Une stratégie de test à plusieurs niveaux a d'abord été définie, puis mise en œuvre sur le backend, la sécurité, les processus BPMN, l'assistant et les interfaces, et enfin automatisée par l'intégration continue et l'analyse statique. Les anomalies révélées ont été corrigées et les scénarios concernés rejoués pour écarter les régressions. La plateforme a ensuite été conteneurisée et sa livraison préparée, avec une documentation et des diagrammes alignés sur les composants réellement implémentés.

Les résultats sont solides sur le backend (111 tests réussis) et le frontend (lint et compilation validés). Ils comportent des réserves explicites : Quality Gate SonarCloud en échec, déploiement distant et nouvelle exécution du pipeline non démontrés, deux scénarios E2E de l'API ignorés (section 7.6.4).

Au terme des cinq sprints, FinBank propose un backend sécurisé, des services bancaires opérationnels, un assistant enrichi par une base de connaissances, des workflows BPMN et des interfaces adaptées aux utilisateurs et aux administrateurs. L'ensemble forme une plateforme bancaire modulaire dont les composants peuvent évoluer indépendamment. La conclusion générale revient sur l'ensemble de ce travail et présente les perspectives d'amélioration.

Conclusion Générale et Perspectives

Ce projet de fin d'études a marqué une étape significative dans mon parcours de formation, en consolidant mes acquis théoriques par une mise en pratique concrète dans un domaine exigeant : les services bancaires numériques. Il m'a permis de me confronter à des problématiques concrètes de sécurité et de validation des opérations sensibles, tout en découvrant et en maîtrisant des technologies modernes utilisées dans le développement d'applications web intelligentes.

Durant ce projet, j'ai eu l'opportunité de concevoir et de développer **FinBank**, une plateforme bancaire centrée sur un agent intelligent de support, capable de répondre rapidement aux demandes courantes tout en protégeant les opérations financières. Cette solution combine les services bancaires classiques, tels que la gestion sécurisée des comptes, des transactions, des virements, des cartes et des investissements, avec des fonctionnalités avancées : un agent conversationnel qui s'appuie sur une base de connaissances grâce au RAG, et trois processus BPMN qui encadrent les opérations sensibles avec une validation humaine lorsque c'est nécessaire.

Grâce à cette expérience, j'ai appris à décomposer un objectif complexe en six sprints, à suivre l'évolution des fonctionnalités et à corriger les problèmes de façon progressive. L'intégration de composants d'intelligence artificielle et de workflows m'a aussi montré l'importance de tenir compte des limites des modèles, de la validation humaine et de la traçabilité des opérations.

Ce projet m'a également permis d'approfondir mes compétences en développement front-end avec React et TypeScript, en développement back-end avec Java et Spring Boot, en gestion de base de données avec PostgreSQL et pgvector, ainsi qu'en Python, en conteneurisation avec Docker Compose et en tests logiciels (111 tests backend réussis). J'ai aussi pu intégrer des composants modernes tels que Flowable pour l'exécution des processus BPMN, Redis pour le cache, Kafka pour les échanges événementiels et Ollama pour l'exécution locale des modèles.

Parmi les perspectives d'évolution du projet, la plus importante consiste à faire évoluer l'agent afin qu'il traite davantage de demandes courantes de manière autonome : questions sur les services, aide à la navigation dans la plateforme, suivi des demandes et préparation de tickets de support à partir des informations fournies par le client. Cette automatisation permettrait de réduire le recours à un conseiller pour les demandes simples, sans supprimer l'intervention humaine, qui resterait nécessaire pour les opérations financières, les litiges et les situations ambiguës.

Une autre perspective consiste à améliorer la qualité et la rapidité des réponses de l'agent. La base de connaissances RAG pourrait être enrichie et mise à jour, avec une évaluation de la pertinence des résultats, tandis que le temps de réponse pourrait être réduit en mesurant puis en optimisant la recherche vectorielle, l'inférence du modèle et la mise en cache des réponses fréquentes. La détection des réponses incertaines et la protection des données personnelles renforceraient aussi la confiance des clients.

Il serait également possible de compléter la plateforme par un suivi plus détaillé des processus Flowable (délais, relances et décisions des conseillers), des tableaux de bord pour l'analyse des opérations et des alertes, ainsi qu'une application mobile.

Enfin, la consolidation technique reste à poursuivre : traiter les problèmes de qualité signalés par SonarCloud, rétablir le pipeline GitHub Actions (facturation et secrets), puis mettre en place un environnement distant sécurisé et, à plus long terme, un déploiement sur Azure Kubernetes Service (AKS). FinBank restant une réalisation académique, un usage bancaire réel exigerait en outre des audits et des tests de sécurité, de charge et de reprise après incident.

En résumé, ce projet de fin d'études clôture mon parcours universitaire et me prépare pleinement à intégrer le monde professionnel. Il a renforcé mes compétences en développement logiciel, en conception de systèmes intelligents et en intégration de services modernes, tout en me dotant d'une base solide pour relever les défis futurs.

Bibliographie

[1] VERMEG. About VERMEG. https://www.vermeg.com/about-us/. Consulté le 3 octobre 2026.

[2] Bankin' SAS. Bankin' : Gestion de budget. https://play.google.com/store/apps/details?id=com.bankeen&hl=fr. Consulté le 5 octobre 2026.

[3] Revolut. L'application financière tout-en-un. https://www.revolut.com/fr-FR/. Consulté le 5 octobre 2026.

[4] Bank of America. Erica : Virtual financial assistant. https://info.bankofamerica.com/en/digital-banking/erica. Consulté le 5 octobre 2026.

[5] Object Management Group. Unified Modeling Language, version 2.5.1. https://www.omg.org/spec/UML/2.5.1/. Consulté le 3 octobre 2026.

[6] Object Management Group. Business Process Model and Notation, version 2.0.2. https://www.omg.org/spec/BPMN/2.0.2/. Consulté le 3 octobre 2026.

[7] Agile Business Consortium. What is MoSCoW prioritization ? https://www.agilebusiness.org/resource/what-is-moscow-prioritization/. Consulté le 3 octobre 2026.

[8] Oracle. JDK 21 documentation. https://docs.oracle.com/en/java/javase/21/. Consulté le 3 octobre 2026.

[9] Spring. Spring Boot reference documentation. https://docs.spring.io/spring-boot/reference/. Consulté le 3 octobre 2026.

[10] Spring. Spring Security reference documentation. https://docs.spring.io/spring-security/reference/. Consulté le 3 octobre 2026.

[11] Michael Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). https://www.rfc-editor.org/rfc/rfc7519. Consulté le 3 octobre 2026.

[12] Apache Software Foundation. Maven documentation. https://maven.apache.org/guides/. Consulté le 3 octobre 2026.

[13] SmartBear. Swagger documentation. https://swagger.io/docs/. Consulté le 3 octobre 2026.

[14] React. React documentation. https://react.dev/. Consulté le 3 octobre 2026.

[15] Microsoft. TypeScript documentation. https://www.typescriptlang.org/docs/. Consulté le 3 octobre 2026.

[16] MUI. Material UI : Overview. https://mui.com/material-ui/getting-started/. Consulté le 3 octobre 2026.

[17] Node.js. Learn Node.js. https://nodejs.org/en/learn. Consulté le 3 octobre 2026.

[18] Vite. Vite : Getting started. https://vite.dev/guide/. Consulté le 3 octobre 2026.

[19] PostgreSQL Global Development Group. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/. Consulté le 3 octobre 2026.

[20] pgvector contributors. pgvector : Open-source vector similarity search for Postgres. https://github.com/pgvector/pgvector. Consulté le 3 octobre 2026.

[21] Redis. Redis documentation. https://redis.io/docs/latest/. Consulté le 3 octobre 2026.

[22] Flowable. Flowable open source documentation : Getting started. https://www.flowable.com/open-source/docs/bpmn/ch02-GettingStarted. Consulté le 3 octobre 2026.

[23] Camunda. Desktop Modeler documentation. https://docs.camunda.io/docs/components/modeler/desktop-modeler/. Consulté le 3 octobre 2026.

[24] Ollama. Ollama documentation. https://docs.ollama.com/. Consulté le 3 octobre 2026.

[25] Spring. Spring AI reference documentation. https://docs.spring.io/spring-ai/reference/. Consulté le 3 octobre 2026.

[26] Python Software Foundation. Python documentation. https://docs.python.org/3/. Consulté le 3 octobre 2026.

[27] Sebastián Ramírez. FastAPI documentation. https://fastapi.tiangolo.com/. Consulté le 3 octobre 2026.

[28] Apache Software Foundation. Apache Kafka documentation. https://kafka.apache.org/documentation/. Consulté le 3 octobre 2026.

[29] Docker. Docker documentation. https://docs.docker.com/. Consulté le 3 octobre 2026.

[30] JetBrains. IntelliJ IDEA : Getting started. https://www.jetbrains.com/help/idea/getting-started.html. Consulté le 3 octobre 2026.

[31] Git contributors. Git documentation. https://git-scm.com/doc. Consulté le 3 octobre 2026.

[32] GitHub. GitHub documentation. https://docs.github.com/. Consulté le 3 octobre 2026.

[33] Postman. Get started in Postman. https://learning.postman.com/docs/getting-started/overview/. Consulté le 3 octobre 2026.

[34] JGraph. draw.io documentation. https://www.drawio.com/docs/. Consulté le 3 octobre 2026.



Poly technique SOUSSE
EUR-ACE®

Campus Polytechnique Sousse,
Sahloul Sousse
(+216) 99 277 877
polytecsousse.tn